

Comparison of the solvers of systems of linear equations: FETI-DP vs BDDC

Porovnání řešičů soustav lineárních rovnic: FETI-DP vs. BDDC

Juraj Musil

Bachelor Thesis

Supervisor: doc. Ing. David Horák, Ph.D.

Ostrava, 2025

Bachelor Thesis Assignment

Student:

Ing. Juraj Musil, Ph.D.

Study Programme:

B0541A170008 Computational and Applied Mathematics

Title:

Comparison of the solvers of systems of linear equations: FETI-DP vs
BDDC

Porovnání řešičů soustav lineárních rovnic: FETI-DP vs. BDDC

The thesis language:

English

Description:

In numerical analysis, there are a number of methods for solving systems of linear equations resulting from the application of the finite element method to elliptic partial differential equations, which today compete to be the fastest.

One of the methods is Finite Element Tearing and Interconnecting (FETI) [1], [2], which is often called an iterative substructuring method and is based on the decomposition of a domain into non-overlapping subdomains. FETI requires the solution of the so-called Neumann problem on each subdomain and also the solution of a coarse problem, which is formed by the gluing matrix and kernels of each subdomain. The FETI-DP variant [3] enforces the gluing of subdomains through dual Lagrange multipliers except for the corner nodes of the subdomains, which are glued by primary global variables called corners. The coarse problem can be enriched by averaging the displacements on the edges of the subregions, which achieves better convergence.

Another method based on domain decomposition is Balancing Domain Decomposition (BDD) [4], [5], which, like FETI, combines the solution of local problems on non-overlapping subdomains with an equally constructed coarse problem or coarse mesh formed in parts by constant functions on subdomains and averages on interfaces. For 4th order problems, it is necessary to extend the coarse problem with special basis functions enforcing the continuity of the solution at the corners of the subdomains, if this is not in multiplicative but in additive form, we speak of BDDC. The dual counterpart to BDD is FETI, which is spectrally equivalent.

The aim of the work is:

- * to get acquainted with the basics of these methods (FETI-DP, BDDC),
- * to thoroughly understand them, to implement them sequentially, e.g. in Matlab/Octave, and to compare the spectra of the resulting operators,
- * to systematically compare parallel implementations of these methods available in the PETSc library on a sufficiently large set of benchmarks, including the convergence rate.

References:

- [1] C. Farhat and F. X. Roux, A method of finite element tearing and interconnecting and its parallel solution algorithm, Internat. J. Numer. Meths. Engrg. 32, 1205-1227 (1991)
- [2] C. Farhat, J. Mandel, and F.-X. Roux, Optimal convergence properties of the FETI domain decomposition method, Comput. Meth. Appl. Mech. Engrg. 115(1994)365-385
- [3] C. Farhat, M. Lesoinne, P. LeTallec, K. Pierson, and D. Rixen, FETI-DP: a dual-primal unified FETI method. I. A faster alternative to the two-level FETI method, Internat. J. Numer. Methods Engrg., 50

(2001), pp. 1523--1544.

[4] J. Mandel, Balancing domain decomposition, Comm. Numer. Methods Engrg., 9 (1993), pp. 233–241. doi:10.1002/cnm.1640090307

[5] L. C. Cowsar, J. Mandel, and M. F. Wheeler, Balancing domain decomposition for mixed finite elements, Math. Comp., 64 (1995), pp. 989–1015. doi:10.1090/S0025-5718-1995-1297465-9

Extent and terms of a thesis are specified in directions for its elaboration that are opened to the public on the web sites of the faculty.

Supervisor: **doc. Ing. David Horák, Ph.D.**

Date of issue: 01.09.2025

Date of submission: 30.04.2026

Study programme guarantor: prof. RNDr. Jiří Bouchala, Ph.D.

In IS EDISON assigned: 10.11.2025 08:51:43

Abstrakt

Táto bakalárska práca porovnáva dual-primal metódy rozkladu oblasti FETI-DP a BDDC určené na riešenie sústav lineárnych rovníc vznikajúcich pri diskretizácii eliptických úloh metódou konečných prvkov. Práca vychádza z modelovej Poissonovej rovnice so zmiešanými Dirichletovými a Neumannovými okrajovými podmienkami na dvojrozmernej štvorcovej oblasti s jednotkovými rozmermi. Po odvodení slabej formulácie a diskretizácii metódou konečných prvkov je diskrétny problém pomocou rozkladu na subdomény a Schurových doplnkov redukovaný na problém v priestore zodpovedajúcom rozhraniu podoblastí, čo umožňuje opísať obe metódy v spoločnom rámci. V tomto rámci sú vytvorené sekvenčné implementácie oboch metód v prostredí Octave a porovnané podľa spektrálnych vlastností predpodmienených operátorov a konvergenzie metódy PCG. Štúdia je doplnená benchmarkovým porovnaním oficiálnych paralelných implementácií v knižnici PETSc. Výsledky ukazujú, že obe metódy sú na testovaných úlohách numericky robustné a veľmi podobné.

Klíčová slova

BDDC, rozklad oblasti, FETI-DP, metóda konečných prvkov, PETSc, Poissonova rovnica, Schurov doplnok

Abstract

This bachelor thesis compares the dual–primal domain decomposition methods FETI-DP and BDDC for solving systems of linear equations arising from finite element discretizations of elliptic problems. The work is based on a model Poisson equation with mixed Dirichlet–Neumann boundary conditions on a two-dimensional unit square domain. After deriving the weak formulation and performing the finite element discretization, the discrete problem is reduced by subdomain decomposition and Schur complements to a problem in the space corresponding to the interface between subdomains, which makes it possible to describe both methods in a common framework. Within this framework, sequential implementations of both methods are developed in Octave and compared according to the spectral properties of the preconditioned operators and the convergence of the PCG method. The study is complemented by a benchmark comparison of the official parallel implementations available in the PETSc library. The results show that both methods are numerically robust and behave very similarly on the tested problems.

Keywords

BDDC, domain decomposition, FETI-DP, finite element method, PETSc, Poisson equation, Schur complement

Acknowledgement

First of all I would like to thank my supervisor for his help and valuable advices and comments. I would also like to thank my entire family for their support and patience throughout my studies.

Contents

List of symbols and abbreviations	8
List of Figures	9
List of Tables	11
1 Introduction	12
2 Weak Formulation of Elliptic Boundary Value Problems	15
2.1 Model problem: Poisson equation with mixed Dirichlet–Neumann boundary conditions	15
2.2 Strong formulation and its limitations	17
2.3 Weak derivatives and Sobolev spaces $\mathbf{W}_2^{(k)}(\Omega)$	20
2.4 Weak formulation of elliptic differential equations	22
2.5 Traces of Sobolev functions and boundary conditions	23
2.6 Weak formulation of a model problem	24
2.7 Existence and uniqueness of weak solution	27
2.8 Variational formulation of boundary value problems: The Ritz method	28
3 The FEM discretization	32
3.1 Ritz approximation of the Poisson problem	32
3.2 Discrete approximation space on the square domain	33
3.3 Discrete Ritz system and stiffness matrix construction	36
3.4 Treatment of boundary conditions	41
3.5 Structure of the resulting linear system	45
4 Substructuring and Interface Framework for DDM	47
4.1 Domain decomposition and local finite element spaces	48
4.2 Subdomain problems, harmonic extension, and Schur complement reduction	54
4.3 Product interface space and continuity constraints	60
4.4 Primal DOFs and coarse continuity enforcement	68

4.5	Position of FETI-DP and BDDC within the unified interface framework	72
4.6	Domain decomposition framework - Octave implementation	73
5	Dual-primal domain decomposition solvers: FETI-DP and BDDC	76
5.1	Common principles and key differences of FETI-DP and BDDC	77
5.2	Common solver infrastructure and implementation tools	84
5.3	FETI-DP solver (multiplier formulation)	87
5.4	BDDC solver (assembled interface formulation)	101
6	Spectral comparison and numerical experiments	111
6.1	Basic notions of spectral theory for domain decomposition operators	112
6.2	Definition of preconditioned iteration operators	115
6.3	Spectral analysis of FETI-DP and BDDC operators – Octave implementation	119
6.4	Results and discussion of spectral comparison and PCG convergence	123
6.5	Parallel PETSc benchmark comparison of FETI-DP and BDDC	136
7	Conclusion	147
	Literatura	151
	Appendices	152
A	Source Code and Reproducibility Archive	153

List of symbols and abbreviations

BDDC	– Balancing Domain Decomposition by Constraints
DDM	– Domain Decomposition Method
DOF	– Degree of Freedom
FEM	– Finite Element Method
FETI	– Finite Element Tearing and Interconnecting
FETI-DP	– Finite Element Tearing and Interconnecting – Dual–Primal
KKT	– Karush–Kuhn–Tucker
PCG	– Preconditioned Conjugate Gradient
PETSc	– Portable, Extensible Toolkit for Scientific Computation
SPD	– Symmetric Positive Definite
B	– jump (constraint) operator on the product interface space
K	– global stiffness matrix
R	– assembly operator from $\widehat{W}$ to W
S	– Schur complement operator
W	– product interface space
Δ	– Laplace operator
Γ_c	– set of primal interface degrees of freedom
Γ_D	– Dirichlet part of the boundary
Γ_N	– Neumann part of the boundary
Ω	– computational domain
Ω_i	– i -th subdomain
λ	– Lagrange multiplier; in spectral context also eigenvalue
$\sigma(T)$	– spectrum of the operator T
$\widehat{W}$	– assembled interface space

List of Figures

2.1	Illustration of the model membrane problem on the unit square. The membrane is shown as a downward deflection under the action of the load f , with the colour indicating the displacement magnitude.	17
3.1	Triangular mesh example	34
3.2	Base Q_6 in triangular mesh	34
3.3	Classification of mesh nodes and boundary parts for the model problem. The left and right sides form the Dirichlet boundary Γ_D , while the top and bottom sides form the Neumann boundary Γ_N	42
3.4	Sparsity pattern of the reduced stiffness matrix K_{FF} obtained after imposing the homogeneous Dirichlet boundary conditions.	46
4.1	Structured non-overlapping decomposition of the computational domain into subdomains, shown with artificial gaps for clarity. Each block represents one subdomain together with its local triangular finite element mesh.	50
4.2	Classification of local degrees of freedom in the exploded subdomain decomposition. Interior nodes, interface nodes, corner/primal nodes, and Dirichlet nodes are distinguished by colour.	53
4.3	Schematic meaning and sparsity pattern of the jump operator B . The product interface space contains local copies of shared interface degrees of freedom, and each row of B enforces equality of two such copies by one $+1$ and one -1 entry.	66
6.1	Spectral comparison of the non-preconditioned FETI-DP and BDDC operators for the case $n = 32$ with 2×2 subdomains	128
6.2	Spectral comparison for the case $n = 32$ with 2×2 subdomains	129
6.3	Sorted spectra of the preconditioned iteration operators for the case $n = 48$ with 4×4 subdomains	131
6.4	Normalized PCG residual histories for the case $n = 48$ with 4×4 subdomains	131
6.5	Spectral distribution and PCG convergence for the case $n = 64$ with 8×8 subdomains	132

6.6	Normalized true residual histories for the representative PETSc case $C-M$	142
6.7	Normalized true residual histories for the representative PETSc case $C-L$	142

List of Tables

6.1	Mesh and reduced finite element dimensions used in the sequential Octave experiments.	124
6.2	Summary of spectral indicators for the non-preconditioned FETI-DP and BDDC operators.	126
6.3	Summary of spectral indicators and PCG convergence for the preconditioned FETI-DP and BDDC operators.	126
6.4	PETSc benchmark design and meaning of the case labels. The active physical DOF count refers to the <code>physical_</code> BDDC branch of the PETSc example.	138
6.5	Summary of PETSc benchmark results for FETI-DP and BDDC.	140
6.6	Auxiliary PETSc timing-event decomposition for representative BDDC runs.	143

Chapter 1

Introduction

The numerical solution of elliptic boundary value problems by the finite element method leads, after discretization, to large sparse systems of linear algebraic equations. For small model problems such systems can be solved directly, but for larger discretizations this approach soon becomes inefficient or even impractical. Efficient solution methods must therefore exploit the structure of the discretized problem, preserve numerical robustness under mesh refinement and increasing problem size, and at the same time be suitable for parallel computation.

Non-overlapping domain decomposition methods form one of the most important classes of methods designed for this purpose. Their basic idea is to decompose the computational domain into smaller subdomains, solve local subdomain problems, and couple the subdomains only through their common interfaces. After eliminating the interior degrees of freedom, the original finite element system can be reduced exactly to an interface problem involving local Schur complements. This reduction makes it possible to separate local computations, which are naturally parallel, from the global interface coupling, which must be treated by a suitable coarse or primal continuity mechanism.

This thesis focuses on two closely related dual–primal domain decomposition methods: FETI-DP and BDDC. Both methods are based on the same substructuring principles: local Schur complement reduction, duplicated interface traces, selected primal degrees of freedom, and a coarse correction ensuring robust global coupling. Their main difference lies in the way the remaining non-primal interface continuity is enforced. In FETI-DP, the continuity constraints are imposed by Lagrange multipliers, which leads to a dual multiplier-space formulation. In BDDC, the coupling is treated from the primal point of view by means of assembled interface corrections and averaging/scaling operators [1, 2]. Because of this common structure and different enforcement mechanism, FETI-DP and BDDC form a natural pair for a comparative study.

The aim of this bachelor thesis is to understand the mathematical and algorithmic foundations of FETI-DP and BDDC, to formulate both methods in a common interface framework, to implement their sequential versions in Octave, and to compare the spectral properties and convergence behavior of the resulting operators. In addition to the sequential implementation developed in the thesis,

the work also includes a compact benchmark comparison of the official parallel implementations available in the PETSc library.

The starting point is a model second-order elliptic problem: the Poisson equation with homogeneous mixed Dirichlet–Neumann boundary conditions on the unit square. The weak and variational formulations of this problem are derived first, including the role of Sobolev spaces, traces, existence and uniqueness of the weak solution, and the Ritz method. The problem is then discretized by the finite element method using piecewise linear basis functions on a structured triangular mesh. Homogeneous Dirichlet boundary conditions are imposed by eliminating the prescribed boundary degrees of freedom, while homogeneous Neumann boundary conditions enter naturally through the variational formulation.

The finite element system obtained in this way is subsequently reformulated in a non-overlapping substructuring framework. The domain is decomposed into structured rectangular subdomains aligned with the finite element mesh. Local stiffness matrices are split into interior and interface blocks, the interior unknowns are eliminated by local solves, and the resulting local Schur complements define an equivalent problem on the subdomain interfaces. To describe the coupling of subdomains explicitly, the thesis introduces the assembled interface space $\widehat{W}$, the product interface space W , the jump operator B , the set of primal degrees of freedom Γ_c , and the decomposition

$$W = W_c \oplus W_\Delta.$$

This common notation provides the basis for the later formulation of both FETI-DP and BDDC.

Within this unified framework, the FETI-DP method is formulated as a reduced multiplier problem in which the constraints on the non-primal interface unknowns are enforced by Lagrange multipliers. The BDDC method is formulated as an assembled interface method using scaled averaging, local Δ -corrections, and a coarse correction generated by primal basis functions. Both solvers are implemented in Octave in a matrix-free style: operator applications are expressed through local subdomain solves, preconditioner actions, and coarse solves rather than by forming all global matrices explicitly. The resulting systems are solved by the Preconditioned Conjugate Gradient method.

The numerical part of the thesis has two complementary components. First, the sequential Octave implementation is used for a detailed spectral comparison. Both non-preconditioned reduced operators and the preconditioned operators actually seen by preconditioned conjugate gradient method are assembled explicitly by probing their matrix-free actions on basis vectors. This makes it possible to compare full spectra, spectral indicators, condition numbers, and residual histories for several mesh and subdomain configurations. Second, the thesis uses the official PETSc tutorial example involving PCBDDC and KSPFETIDP to compare the PETSc implementations of BDDC and FETI-DP on a larger benchmark set. The PETSc benchmark is not intended as a direct timing comparison with the Octave code or as a complete parallel scaling study; rather, it complements the

sequential spectral analysis by iteration counts, PETSc spectral indicators, true residual histories, and representative timing information from an established parallel solver library [3, 4, 5].

The thesis is organized as follows. Chapter 2 introduces the model Poisson problem and derives its weak and variational formulation. Chapter 3 presents the finite element discretization, the construction of the stiffness matrix and load vector, and the treatment of boundary conditions. Chapter 4 develops the non-overlapping substructuring framework, including Schur complements, product interface spaces, continuity constraints, primal degrees of freedom, and the corresponding Octave implementation infrastructure. Chapter 5 specializes this framework to the FETI-DP and BDDC solvers, describes their common and method-specific components, and explains their matrix-free Octave implementation. Finally, Chapter 6 presents the spectral comparison, PCG convergence experiments, and the PETSc benchmark comparison of the official parallel implementations.

In addition to the written thesis, the work is accompanied by an electronic source-code archive. This archive contains the Octave implementation of the finite element assembly, domain decomposition infrastructure, FETI-DP and BDDC solvers, test runners, and scripts/data used for the numerical experiments. The exact version corresponding to this thesis is submitted as an electronic appendix in IS EDISON and is also made available as a tagged public GitHub release. The submitted archive corresponds to the repository https://github.com/Musiljuraj/bc_ddm, release tag `Thesis-final`, and commit `52830ed61a7a75a2b5858b341bdb6332a7ba3fda`. The code archive serves as a reproducibility supplement to the mathematical derivation and numerical results presented in the thesis; a concise description of the electronic appendix is given in Appendix A.

Chapter 2

Weak Formulation of Elliptic Boundary Value Problems

2.1 Model problem: Poisson equation with mixed Dirichlet–Neumann boundary conditions

Given that the objective of this bachelor thesis is the implementation of domain decomposition methods (DDM) based on the general finite element method (FEM), specifically the implementation of Finite Element Tearing and Interconnecting – Dual-primal method (FETI-DP) and Balancing Domain Decomposition method by Constraints (BDDC), it is useful to begin with a simple yet representative mathematical model. DDM are formulated with respect to a specific domain Ω and its boundary partition, the corresponding weak formulation, and the function spaces in which the solution is sought [6]. With a clearly defined problem, it is much easier to introduce the energy functional, derive the weak form, assemble the stiffness matrices, and analyze the method's convergence properties. Moreover, choosing an appropriate model problem at the beginning helps maintain consistent notation throughout the thesis and prevents it from becoming overly abstract.

A classical example of such a model is the deflection (displacement) of a thin, tensioned membrane subjected to a distributed load. In this thesis, a simplified two-dimensional membrane model is considered, in which the vertical displacement is described by a second-order linear elliptic partial differential equation. For a homogeneous membrane with constant tension, this equation reduces to the Poisson equation. This formulation is mathematically straightforward, physically intuitive, and captures all essential features required for the derivation of the finite element method and its variants.

Let $\Omega \subset \mathbb{R}^2$ denote the two-dimensional membrane domain. The goal is to determine the vertical displacement $u : \Omega \rightarrow \mathbb{R}$ of a thin, tensioned membrane subjected to an external load $f : \Omega \rightarrow \mathbb{R}$. Under the assumptions of linear elasticity, small deformations, and constant membrane tension, the

equilibrium configuration of the membrane is governed by the elliptic partial differential equation

$$-\nabla \cdot (T \nabla u) = f \quad \text{in } \Omega.$$

For a homogeneous membrane with isotropic constant tension $T = tI$, where $t > 0$ and I is the identity matrix, the equation becomes

$$-t \Delta u = \tilde{f} \quad \text{in } \Omega,$$

which can be simplified to

$$-\Delta u = f \quad \text{in } \Omega,$$

by setting $f := \frac{\tilde{f}}{t}$. In this way, without loss of generality, we may assume $t = 1$ [6].

A unique and physically meaningful solution requires the prescription of appropriate boundary conditions. In our case, we assume that the boundary $\partial\Omega$ consists of two disjoint parts Γ_D , Γ_N , such that

$$\partial\Omega = \Gamma_D \cup \Gamma_N, \quad \Gamma_D \cap \Gamma_N = \emptyset,$$

and we impose the following boundary conditions:

$$\begin{aligned} u &= 0, & \text{on } \Gamma_D, \\ (t \nabla u) \cdot n &= 0, & \text{on } \Gamma_N, \end{aligned}$$

where n denotes the outward unit vector normal to $\partial\Omega$. The first condition is homogeneous Dirichlet condition corresponding to a fixed boundary, while the second is homogeneous Neumann condition expressing the absence of traction on the free part of the boundary.

Together, the differential equation and the boundary conditions form a mixed Dirichlet–Neumann boundary value problem, which serves as a model problem for all subsequent analysis in this thesis.

The complete formulation of the problem is given below [7]

$$\begin{aligned} -\Delta u &= f \quad \text{in } \Omega, \\ u &= 0 \quad \text{on } \Gamma_D, \\ \nabla u \cdot n &= 0 \quad \text{on } \Gamma_N, \end{aligned} \tag{2.1}$$

where f is the applied load, and the boundary is decomposed as

$$\partial\Omega = \Gamma_D \cup \Gamma_N, \quad \Gamma_D \cap \Gamma_N = \emptyset.$$

To obtain a clear and easily interpretable computational model, a simple geometric configuration

is adopted. Specifically, the domain Ω is taken to be the unit square $[0, 1] \times [0, 1]$. The membrane is fixed along two opposite sides of the square, which together constitute the Dirichlet boundary Γ_D , while the remaining two sides form the Neumann boundary Γ_N . The physical interpretation of this model problem is illustrated in Figure 2.1. The figure shows the membrane as a downward deflection under the applied load f . The connection between the physical membrane model and the Poisson equation introduced above.

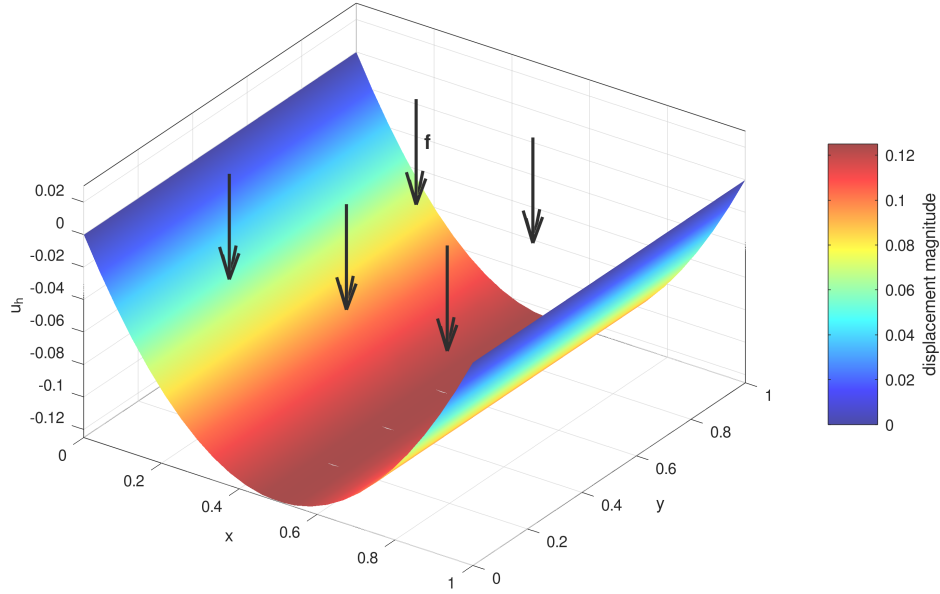


Figure 2.1: Illustration of the model membrane problem on the unit square. The membrane is shown as a downward deflection under the action of the load f , with the colour indicating the displacement magnitude.

2.2 Strong formulation and its limitations

The formulated problem (2.1) belongs to a broad class of frequently occurring problems in mathematical physics, which can be expressed in an abstract operator form. Consider the general equation

$$Au = f, \quad (2.2)$$

where f is an element of a Hilbert space H , and A is a linear operator (typically a positive differential operator) defined on a domain D_A , which is a vector space of functions dense in H [6].

Definition 1 (Scalar product) Let X be a real vector space. A mapping

$$(\cdot, \cdot)_X : X \times X \rightarrow \mathbb{R}$$

is called a scalar product, or inner product, on X if for all $u, v, w \in X$ and all $\alpha, \beta \in \mathbb{R}$ the following properties hold:

$$(\alpha u + \beta v, w)_X = \alpha(u, w)_X + \beta(v, w)_X,$$

$$(u, v)_X = (v, u)_X,$$

and

$$(u, u)_X \geq 0 \quad \text{and} \quad (u, u)_X = 0 \iff u = 0.$$

The scalar product induces the norm

$$\|u\|_X = \sqrt{(u, u)_X}.$$

A vector space equipped with a scalar product is called an inner-product space.

When the underlying space is clear from the context, the subscript X is omitted. In finite-dimensional real vector spaces, unless stated otherwise, we use the Euclidean scalar product $(u, v) = u^T v$.

Definition 2 (Hilbert space) A Hilbert space is a complete inner-product space.

Definition 3 (Dense subset) A subset $D_A \subset H$ is dense if every element of H can be arbitrarily well approximated by elements of D_A .

When A is a differential operator, a natural choice for the underlying Hilbert space is

$$H = L^2(\Omega),$$

and the domain D_A consists of suitable, sufficiently smooth functions on which the differentiation operations prescribed by A are well defined and which satisfy the prescribed boundary conditions.

Definition 4 (The space $L^2(\Omega)$) The space $L^2(\Omega)$ consists of all square-integrable functions on Ω , i.e.,

$$L^2(\Omega) = \left\{ u : \Omega \rightarrow \mathbb{R} \left| \int_{\Omega} |u(x)|^2 dx < \infty \right. \right\}. \quad (2.3)$$

It is a Hilbert space with the scalar product

$$(u, v)_{L^2(\Omega)} = \int_{\Omega} u(x) v(x) dx. \quad (2.4)$$

The corresponding norm is given by

$$\|u\|_{L^2(\Omega)} = \sqrt{(u, u)_{L^2(\Omega)}}. \quad (2.5)$$

The integrals in Definition 4 are understood in the Lebesgue sense. When no confusion is possible, we write simply (u, v) instead of $(u, v)_{L^2(\Omega)}$. This convention will also be used for other scalar products introduced later, with the underlying space being clear from the context.

A function $u_0 \in D_A$ is called a (strong) solution of the operator equation (2.2) if

$$Au_0 = f \quad \text{in } H,$$

i.e., if $Au_0 - f$ is the zero element of H . For example, if $H = L^2(\Omega)$, this means that the equality holds almost everywhere, except possibly on a set of zero measure.

The following classical results hold under these assumptions.

Theorem 2.1 (Uniqueness) *Let D_A be a vector space of functions dense in a Hilbert space H , and let $A : D_A \rightarrow H$ be a linear, symmetric, and positive definite (SPD) operator. Then equation $Au = f$ with $f \in H$ has at most one solution $u \in D_A$.*

Theorem 2.2 (Minimum of a quadratic functional) *Let $A : D_A \rightarrow H$ be the SPD operator and let $f \in H$. Assume that*

$$Au_0 = f, \quad u_0 \in D_A.$$

Then the quadratic functional

$$F(u) = (Au, u) - 2(f, u)$$

reaches its minimum over D_A exactly at u_0 , i.e.,

$$F(u) \geq F(u_0) \quad \text{for all } u \in D_A,$$

with equality if and only if $u = u_0$.

Conversely, if $F(u)$ reaches its minimum at some $u_0 \in D_A$, then u_0 is the solution of equation $Au = f$ in H .

The significance of Theorem 2.2 lies in the fact that it transforms the task of solving the operator equation $Au = f$ into the equivalent task of finding an element $u_0 \in D_A$ that minimizes the quadratic functional $F(u)$ over D_A . This equivalence, however, is conditional. Both formulations are based on the assumption that equation $Au = f$ has a solution in space D_A , or that functional $F(u)$ reaches its minimum in D_A .

In general, neither of these assumptions is guaranteed, not even in very simple cases. This becomes evident already for the Poisson problem (2.1). If the right-hand side f is sufficiently

irregular—for example, if f equals 0 on one subregion of Ω and 1 on the remainder, so that no modification on a set of measure zero can render it continuous — then the problem cannot have a classical solution in sufficiently smooth space. The reason is that every function $u \in D_A$ must be sufficiently regular for the Laplacian Δu to be defined pointwise, meaning that its second derivatives must be continuous in $\bar{\Omega}$. Under such smoothness requirements, the equation

$$-\Delta u = f$$

cannot hold pointwise when f exhibits this type of discontinuity. Consequently, in accordance with Theorem 2.2, the functional $F(u)$ cannot reach its minimum over D_A .

The irregular or discontinuous right-hand sides of this form frequently arise in practical applications. They correspond, for instance, to piecewise constant loads, abrupt changes in material properties, or localized sources. Such examples have perfectly meaningful physical interpretations, and excluding them by requiring excessive smoothness would be undesirable [6].

These difficulties motivate the introduction of a so-called weak (variational) formulation. The essential reason for this reformulation is the lack of regularity of the data, most notably the possible discontinuities of the right-hand side f . When f is only square-integrable or piecewise defined, the differential equation cannot, in general, be satisfied pointwise by a sufficiently smooth function, and classical solutions may fail to exist. To accommodate such irregular data, the solution must therefore be sought in a larger function space with weaker smoothness requirements.

The weak formulation achieves this by interpreting the equation in an integral sense. By testing the equation with smooth test functions and applying integration by parts, derivatives are transferred from the unknown solution to the test functions. For second-order elliptic equations such as the Poisson problem, the weak formulation involves only first derivatives of the solution, which allows solutions to be defined in Sobolev spaces where second derivatives need not exist pointwise. In this way, the weak formulation provides a mathematically consistent notion of solution that remains meaningful even in the presence of discontinuous right-hand sides, and it forms the natural framework for both theoretical analysis and numerical approximation [6].

2.3 Weak derivatives and Sobolev spaces $W_2^{(k)}(\Omega)$

Let Ω be a bounded domain with Lipschitz boundary Γ . Denote by $C^\infty(\bar{\Omega})$ the vector space of all functions $u(x)$ that are infinitely differentiable in the closed domain Ω . Furthermore, denote by $C_0^\infty(\Omega)$ the vector space of all functions with compact support in Ω , i.e., the set of all functions $u \in C^\infty(\bar{\Omega})$ for which $u(x) \equiv 0$ in some neighborhood of the boundary.

Now consider the set $C^\infty(\bar{\Omega})$ and let k be a nonnegative integer. Let us

$$(u, v)_{W_2^k(\Omega)}, \quad u \in C^\infty(\bar{\Omega}), \quad v \in C^\infty(\bar{\Omega})$$

denote the sum of inner products (in $L_2(\Omega)$) of functions $u(x)$ and $v(x)$ and their derivations, always with respect to the same variable, up to and including order k . That is,

$$(u, v)_{W_2^k(\Omega)} = \sum_{|i| \leq k} \int_{\Omega} D^i u D^i v \, dx, \quad u \in C^\infty(\overline{\Omega}), \quad v \in C^\infty(\overline{\Omega}). \quad (2.6)$$

Here

$$D^i u = \frac{\partial^{i_1 + \dots + i_N}}{\partial x_1^{i_1} \dots \partial x_N^{i_N}} u$$

$$|i| = i_1 + \dots + i_N$$

and the summation $\sum_{|i| \leq k}$ means that all distinct multi-indices i satisfying $|i| = i_1 + \dots + i_N \leq k$ are included.

The relation (2.6) defines an inner product in the vector space $C^\infty(\overline{\Omega})$. Consequently, a norm and a metric can be introduced in the usual way, and the vector space $C^\infty(\overline{\Omega})$ together with the inner product (2.6), forms a metric space, which we denote by $S_2^{(k)}(\Omega)$.

Further, it can be proved that integral identity

$$\int_{\Omega} D^i u \phi \, dx = (-1)^{|i|} \int_{\Omega} u D^i \phi \, dx, \quad \phi \in C_0^\infty(\Omega) \quad (2.7)$$

with $1 \leq |i| \leq k$ holds for every function $u \in S_2^{(k)}(\Omega)$ and for every (so-called) test function $\phi \in C_0^\infty(\Omega)$ [6].

The space $S_2^{(k)}(\Omega)$ is not complete; however, it can be completed as follows. Consider a sequence $\{u_n\}$ that is Cauchy in $S_2^{(k)}(\Omega)$ and assume that this sequence does not converge in $S_2^{(k)}(\Omega)$. Since it is Cauchy, each sequence $\{D^i u_n\}$ is Cauchy in $L_2(\Omega)$ and because $L_2(\Omega)$ is complete, each of these sequences has a limit, which we denote by

$$\lim_{n \rightarrow \infty} D^i u_n = v^{(i)} \quad v \in L_2(\Omega). \quad (2.8)$$

For $|i| = 0$ we denote the corresponding limit by

$$\lim_{n \rightarrow \infty} u_n = u \quad u \in L_2(\Omega). \quad (2.9)$$

For functions $v^{(i)}$, $|i| \geq 1$, it cannot be asserted that they are classical derivatives of the limit function u , since we only know that $u \in L_2(\Omega)$. However, it can be shown that these functions satisfy the same integral identity (2.7) as the derivatives of the functions from $S_2^{(k)}(\Omega)$.

The functions $v^{(i)}$ are called weak derivatives of the function u . If a function $u \in L_2(\Omega)$ possesses weak derivatives $v^{(i)}$ ($1 \leq |i| \leq k$), i.e. if it is the limit in $L_2(\Omega)$ of a sequence u_n that is Cauchy in

$S_2^{(k)}(\Omega)$, then these weak derivatives are uniquely determined by u . Moreover, if the function u is sufficiently smooth, (for example, if $u \in S_2^{(k)}(\Omega)$), then the notion of weak derivative coincides with that of the classical derivative.

If, according to the construction above, we add to the space $S_2^{(k)}(\Omega)$ all limit elements defined by (2.9) we obtain the Sobolev space $W_2^{(k)}(\Omega)$ [6].

2.4 Weak formulation of elliptic differential equations

In the previous section, we introduced the concepts of weak derivatives and Sobolev spaces, which provide a functional-analytic framework for the treatment of differential equations whose classical solutions may fail to exist. We now apply this theory directly to the model problem introduced in section 2.1, namely the Poisson equation that describes the equilibrium of a tensioned membrane. The objective of this section is to reformulate the Poisson equation in a weak (variational) sense and to define an appropriate notion of solution that remains meaningful for the right-hand sides with limited regularity. This weak formulation will serve as the basis for all subsequent analysis and numerical methods considered in this thesis.

Consider the Poisson equation

$$-\Delta u = f. \quad (2.10)$$

Let $u \in C^{(2)}(\Omega)$, $f \in C(\Omega)$, and assume that equation (2.10) is satisfied in the domain Ω , so that the function $u(x)$ is a classical solution of this equation in Ω . Multiply equation (2.10) by an arbitrary function $\phi(x)$ with compact support in Ω and integrate the resulting equality over Ω ,

$$-\int_{\Omega} \phi \Delta u \, dx = \int_{\Omega} \phi f \, dx. \quad (2.11)$$

Using the Green's theorem, the relation (2.11) can be transformed into the following identity:

$$\sum_{l=1}^N \int_{\Omega} \frac{\partial \phi}{\partial x_l} \frac{\partial u}{\partial x_l} \, dx = \int_{\Omega} \phi f \, dx. \quad (2.12)$$

Since the function $\phi \in C_0^\infty$ is arbitrary, it follows that if $u(x)$ is a classical solution of equation (2.10) in Ω , then it satisfies the identity (2.12) for every test function $\phi \in C_0^\infty(\Omega)$ [6].

However, if $f \notin C(\Omega)$, equation (2.10) does not admit a classical solution (i.e., a solution in $C^2(\Omega)$), because the left-hand side of (2.10) would then be a continuous function in Ω , while the right-hand side would not. In such a case, the question arises as to how to appropriately generalize the notion of a solution of the equation (2.10), for example, when we only assume that $f \in L_2(\Omega)$.

For $f \in L_2(\Omega)$, the right-hand side of (2.12) is always defined, while the left-hand side is meaningful, provided that $\frac{\partial u}{\partial x_l} \in L_2(\Omega)$. This condition is certainly satisfied if $u \in W_2^{(1)}(\Omega)$ and

derivatives are understood in the weak sense. This observation allows us to introduce the notion of a weak solution of the equation (2.10) in the following way [6]:

Definition 5 (Weak formulation of a Poisson equation) *Let $u \in W_2^1(\Omega)$ and $f \in L_2(\Omega)$. If the function $u(x)$ satisfies identity (2.12) for every test function $\phi \in C_0^\infty(\Omega)$, then $u(x)$ is called a weak solution of equation (2.10).*

A weak solution is thus a function of the space $W_2^1(\Omega)$ which does not need to have second-order derivatives in Ω ; it is sufficient that it has weak first order derivatives. However, if $u \in C^{(2)}(\Omega)$ and $f \in C(\Omega)$, and if $u(x)$ satisfies the identity (2.12) for every $\phi \in C_0^\infty(\Omega)$, then $u(x)$ is also a classical solution of the equation (2.10).

The Poisson equation is a canonical example of a second-order linear elliptic equation. The derivation above illustrates the general principle of weak formulations: by testing the differential equation against smooth functions with compact support and applying an integration identity, derivatives are transferred from the unknown function onto the test function, reducing the regularity requirements on the solution.

It should be emphasized that the weak formulation derived in this section is formulated entirely in the interior of the domain and does not yet incorporate boundary conditions. In order to extend this formulation to boundary value problems, it is therefore necessary to introduce a suitable notion of boundary values for Sobolev functions, which will be done in the next section by means of the trace operator.

2.5 Traces of Sobolev functions and boundary conditions

To formulate boundary conditions in Sobolev spaces, the classical point-wise boundary values $u|_\Gamma$ must be replaced by a suitable generalized notion, since functions belonging to Sobolev spaces generally lack sufficient regularity for well-defined point-wise traces. The so-called trace operator provides a mathematically consistent replacement for classical boundary values by extending the restriction $u \mapsto u|_\Gamma$ from smooth functions to functions in $W_2^{(1)}(\Omega)$.

For every function that is continuous in the closure $\overline{\Omega}$, and in particular for every function $u \in C^\infty(\overline{\Omega})$, the boundary values $u(S)$ for $S \in \Gamma$ are uniquely determined. The function $u(S)$ defined on Γ is called the trace of the function $u(x)$ on the boundary. Since the trace of a smooth function is continuous in Γ , it is in particular square-integrable, and therefore

$$u(x) \in C^\infty(\overline{\Omega}) \implies u(S) \in L_2(\Gamma).$$

The extension of the notion of trace from smooth functions to all functions belonging to the Sobolev space $W_2^{(1)}(\Omega)$, including those that are not continuously extendable on the boundary, is ensured by the following theorem [6].

Theorem 2.3 *Let Ω be a bounded domain with a Lipschitz boundary Γ . Then there exists a unique bounded linear operator $T : W_2^{(1)}(\Omega) \rightarrow L_2(\Gamma)$, such that for every function $u(x) \in C^\infty(\overline{\Omega})$ the equality $Tu(x) = u|_\Gamma$ holds.*

As a consequence of this theorem, each function $u \in W_2^{(1)}(\Omega)$ is assigned a function $u(S) \in L_2(\Gamma)$ called the trace of $u(x)$ on the boundary Γ . This assignment has the following important properties. First, it coincides with the classical notion of boundary values for smooth functions. Second, it is continuous with respect to the norms of the corresponding spaces; if two functions are close in the space $W_2^{(1)}(\Omega)$, then their traces are close in the space $L_2(\Gamma)$.

Since the space $C^\infty(\overline{\Omega})$ is dense in $W_2^{(1)}(\Omega)$, the trace of a function $u \in W_2^{(1)}(\Omega)$ that does not belong to $C^\infty(\overline{\Omega})$ can be interpreted as the limit (in $L_2(\Gamma)$) of the traces of a sequence of smooth functions $u_n(S) \in C^\infty(\overline{\Omega})$ that converge to u in space $W_2^{(1)}(\Omega)$.

The trace operator therefore provides an appropriate mathematical framework for the formulation of Dirichlet boundary conditions in weak formulations of boundary value problems. Its use will be essential in the following section, where weak formulations of elliptic boundary value problems are derived.

2.6 Weak formulation of a model problem

In this section, we derive weak formulations of the Poisson equation for boundary value problems relevant for this thesis. We first treat Dirichlet boundary conditions and then extend the formulation to the mixed Dirichlet–Neumann case introduced as the model problem in section 2.1. The purpose of this approach is to demonstrate how boundary conditions are incorporated into the weak formulation and variational setting and to prepare the ground for the existence and uniqueness analysis and for the numerical discretization developed in the subsequent sections.

2.6.1 Weak formulation of Poisson problem with Dirichlet boundary conditions

Consider the Poisson equation with non-homogeneous boundary condition

$$-\Delta u = f \tag{2.13}$$

$$u = g(S) \quad \text{on} \quad \Gamma. \tag{2.14}$$

Let V denote the set of all functions $v \in W_2^{(1)}(\Omega)$ that satisfy the corresponding homogeneous boundary conditions in the sense of traces, i.e. condition $v = 0$ on Γ . Thus,

$$V = W_2^{(1)}(\Omega),$$

where $W_2^{(1)}(\Omega)$ is a subspace of $W_2^{(1)}(\Omega)$.

Proceeding in a manner analogous to the derivation of the weak formulation in section 2.4 we obtain the integral identity

$$\sum_{l=1}^N \int_{\Omega} \frac{\partial v}{\partial x_l} \frac{\partial u}{\partial x_l} dx = \int_{\Omega} v f dx \quad (2.15)$$

for every $v \in V$.

For later use, we introduce the notation

$$A(v, u) := \sum_{l=1}^N \int_{\Omega} \frac{\partial v}{\partial x_l} \frac{\partial u}{\partial x_l} dx, \quad (2.16)$$

which defines a bilinear form on $V \times V$.

Let $w \in W_2^{(1)}(\Omega)$ be chosen such that its trace on the boundary Γ equals the prescribed boundary data $g(S)$.

A function $u \in W_2^{(1)}(\Omega)$ is called a weak solution of the boundary problem (2.13), (2.14) if

$$u - w \in V \quad (2.17)$$

and if the integral identity

$$A(v, u) = \int_{\Omega} v f dx \quad (2.18)$$

holds for every $v \in V$ [6].

From this definition it follows that a weak solution of the boundary value problem is, at the same time, a weak solution of the Poisson equation in the sense of the earlier definition. Indeed, every function $\phi \in C_0^\infty(\Omega)$ with compact support belongs to the space V . Consequently, the identity (2.18) holds for all such test functions, which is precisely the condition that defines a weak solution of the Poisson equation.

The fulfillment of the boundary condition $u = g(S)$ on Γ is ensured by condition $u - w \in V$. Since the function $w(x)$ has trace $g(S)$ on Γ and the function $u(x) - w(x)$ has trace equal to zero, it follows that the trace of the function $u(x)$ on Γ is equal to $g(S)$.

2.6.2 Weak formulation of a model Poisson problem with mixed Dirichlet - Neumann conditions

We now consider the Poisson equation

$$-\Delta u = f$$

subject to mixed boundary conditions. Let the boundary Γ be decomposed into two disjoint parts,

$$\partial\Omega = \Gamma_D \cup \Gamma_N, \quad \Gamma_D \cap \Gamma_N = \emptyset,$$

where Dirichlet boundary conditions are prescribed on Γ_D and Neumann boundary conditions on Γ_N , as follows

$$\begin{aligned} u|_{\Gamma_D} &= g(S) \\ \frac{\partial u}{\partial \nu} \Big|_{\Gamma_N} &= h(S). \end{aligned}$$

In accordance with the earlier discussion, we define the space

$$V = \{v; v \in W_2^{(1)}(\Omega), Tv = 0 \text{ on } \Gamma_D\}, \quad (2.19)$$

where T denotes the trace operator. Thus, every function in V satisfies the homogeneous Dirichlet boundary condition on Γ_D in the sense of traces, while no restriction is imposed on Γ_N [6].

Proceeding analogously to the derivation of the weak formulation in the previous cases, we multiply the Poisson equation by a test function $v \in V$ and integrate over the domain Ω . Applying the Green's identity and taking into account the Neumann boundary condition on Γ_N , we obtain the integral identity

$$\int_{\Omega} \sum_{l=1}^N \frac{\partial v}{\partial x_l} \frac{\partial u}{\partial x_l} dx = \int_{\Omega} v f dx + \int_{\Gamma_N} v h dS \quad (2.20)$$

for every $v \in V$ [6].

As in the previous section, we introduce the bilinear form

$$A(v, u) := \sum_{l=1}^N \int_{\Omega} \frac{\partial v}{\partial x_l} \frac{\partial u}{\partial x_l} dx$$

defined on $V \times V$.

Let us now fix a function $w \in W_2^{(1)}(\Omega)$ whose trace on the boundary Γ_D equals the prescribed boundary data $g(S)$. A function $u \in W_2^{(1)}(\Omega)$ is called a weak solution of the mixed boundary problem if

$$u - w \in V \quad (2.21)$$

and if the integral identity (2.20) is satisfied for all test functions $v \in V$.

As in the case of purely Dirichlet boundary conditions, this definition ensures that the boundary condition $u = g(S)$ on Γ_D is fulfilled in the sense of traces. At the same time, the Neumann boundary condition in Γ_N enters the weak formulation through the boundary integral and is therefore later naturally incorporated into the variational setting.

The weak formulation derived in this section represents the model boundary value problem considered in this thesis and will serve as the starting point for the theoretical analysis and numerical

methods developed in the following sections.

2.7 Existence and uniqueness of weak solution

Theorem 2.4 (Lax-Milgram theorem [6]) *Let H be a Hilbert space with inner product (u, v) , and let $B(u, v)$ be a bilinear form defined for $v \in H, u \in H$, such that there exists constants $K > 0, \alpha > 0$, independent of v and u , for which*

$$|B(v, u)| \leq K \|v\| \|u\|, \quad (2.22)$$

$$B(v, v) \geq \alpha \|v\|^2.$$

Then every bounded linear functional F on H can be represented in the form

$$Fv = B(v, z), \quad v \in H,$$

where z is an element of the space H , uniquely determined by the functional F . Moreover, the estimate

$$\|z\| \leq \frac{\|F\|}{\alpha},$$

holds, where $\|F\|$ denotes the norm of the functional F .

Definition 6 (V-ellipticity [6]) *Let $A(v, u)$ be a bilinear form defined on a space V . The bilinear form $A(v, u)$ is called V -elliptic if there exists a constant $\alpha > 0$ such that*

$$A(v, v) \geq \alpha \|v\|_V^2.$$

holds for all $v \in V$.

Theorem 2.5 (Existence and uniqueness of weak solution [6]) *Consider the weak formulation of the Poisson problem with mixed Dirichlet-Neumann boundary conditions derived in section 2.6.2.*

Let V be the space defined by (2.19) and let $A(\cdot, \cdot)$ be the bilinear form introduced in (2.16). Assume that the bilinear form $A(\cdot, \cdot)$ is continuous on $V \times V$ and V -elliptic and that the right-hand side defines a bounded linear functional on V .

Then the weak problem has a unique solution $u \in W_2^{(1)}(\Omega)$, and there exists a constant $c > 0$, independent of f, w , and h such that

$$\|u\|_{W_2^{(1)}(\Omega)} \leq c \left(\|f\|_{L_2(\Omega)} + \|w\|_{W_2^{(1)}(\Omega)} + \|h\|_{L_2(\Gamma)} \right). \quad (2.23)$$

The weak formulation leads to an operator equation on a Hilbert space in which the solution is characterized by a bilinear form $A(v, u)$ and a linear functional. Theorem 2.5 therefore guarantees the existence and uniqueness of the weak solution, provided that the bilinear form $A(v, u)$ is V -elliptic in the space V .

Inequality (2.23) expresses the continuous dependence of the weak solution on the given data of the problem. If the function $f(x)$ changes only slightly in the metric of the space L_2 , the function $w(x)$ changes only slightly in the metric of the space $W_2^{(1)}(\Omega)$, and the function $h(S)$ changes only slightly in the metric of the space $L_2(\Gamma)$, then the weak solution $u(x)$ changes only slightly in the metric of the space $W_2^{(1)}(\Omega)$.

In this section, we discuss the Lax–Milgram theorem and introduce the notion of V -ellipticity of bilinear form. These results provide sufficient conditions under which a weak formulation of the problem admits a unique solution that continuously depends on the given data. The Lax–Milgram theorem can be viewed as a generalization of the theorem on the minimum of a quadratic functional discussed earlier, since both results address the existence and uniqueness of solutions under V -ellipticity assumptions. Although the minimum theorem requires that the bilinear form be symmetric and allows the solution to be characterized as a minimizer of an energy functional, the Lax–Milgram theorem does not rely on symmetry and applies to a broader class of problems that may not admit a minimization interpretation. For this reason, the Lax–Milgram theorem constitutes a more general and fundamental analytical tool, forming the basis for the study of weak problems independently of their possible variational characterization.

2.8 Variational formulation of boundary value problems: The Ritz method

In the previous section, we introduced the weak formulation of elliptic boundary value problems in the space V and established (under specific assumptions) the existence and uniqueness of the weak solution (Theorem 2.5).

These results are formulated in terms of bilinear forms and linear functionals and do not, in general, rely on any minimization interpretation of the problem.

The purpose of the present section is to introduce an alternative viewpoint in which the weak problem is characterized through a variational principle based on the minimization of an energy functional. In this section, we assume in addition that $A(\cdot, \cdot)$ is symmetric on V , i.e.

$$A(u, v) = A(v, u), \quad \forall u, v \in V,$$

and show that the weak solution admits an equivalent variational characterization as the minimizer of an associated functional, which naturally leads to the Ritz method.

This variational approach is closely related to the classical Ritz method and plays a central role in the theory of numerical approximation of boundary value problems. Compared with the weak

formulation, the variational formulation provides a more constructive interpretation of the solution and serves as a natural starting point for the development of finite-dimensional approximation schemes, such as the finite element method. It is therefore introduced here as a preparatory step for the numerical methods discussed in the subsequent sections.

2.8.1 Variational principle for the model problem

To introduce the variational characterization in its simplest form, we restrict our attention to the case of homogeneous boundary conditions. This restriction is fully consistent with the model assignment in this thesis. Reduction to homogeneous boundary conditions eliminates boundary terms and leads to a weak problem formulated directly in the space V . This setting makes it possible to interpret the weak solution as the minimizer of an associated energy functional and forms the basis of the classical Ritz method. In what follows, we therefore assume the following

$$\begin{aligned} g(S) &= 0 \quad \text{on} \quad \Gamma_D, \\ h(S) &= 0 \quad \text{on} \quad \Gamma_N. \end{aligned}$$

In this case, the weak formulation of the model Poisson problem reduces to the following problem: find a function $u \in V$ such that

$$A(v, u) = \int_{\Omega} v f \, dx \quad \forall v \in V.$$

Theorem 2.6 (Weak solution as minimum of functional [6]) *Assume that the bilinear form $A(v, u)$ defined in $V \times V$ is symmetric, V -elliptic, and there exists a constant $K > 0$ such that*

$$A(v, u) \leq K \|v\|_V \|u\|_V \quad \forall v, u \in V.$$

Then a function $u \in V$ is a weak solution of the problem

$$A(v, u) = \int_{\Omega} v f \, dx \quad \forall v \in V,$$

if and only if it minimizes the functional

$$J(v) = A(v, v) - 2 \int_{\Omega} v f \, dx$$

over the space V .

Theorem 2.6 transforms the task of finding a weak solution to the boundary value problem into the equivalent task of minimizing a quadratic functional over the space V . This theorem, in contrast to Theorem 2.2 on the minimum of a quadratic functional, has (under stated assumptions)

an unconditional character; the existence and uniqueness of the minimizer are guaranteed here directly by the stated assumptions on the bilinear form.

This part thus completes the theoretical analysis of the weak formulation of the model problem by establishing its equivalence with a variational principle (for the purpose of this thesis only in the homogeneous case). These results provide a solid analytical foundation for the subsequent construction of approximate solutions.

2.8.2 The Ritz method

The variational characterization given in Theorem 2.6 transforms the weak problem into a minimization problem over infinite-dimensional space V . In practical computations, this minimization must be restricted to a finite-dimensional subspace of V . The Ritz method is based on this idea: it seeks an approximate minimizer in the span of a chosen basis of V , leading to a finite system of algebraic equations for unknown coefficients.

Let $V_n \subset V$ be a n -dimensional subspace of V , spanned by linearly independent functions $\{v_1, v_2, \dots, v_n\}$. We seek an approximate solution $u_n \in V_n$ of the form

$$u_n = \sum_{j=1}^n u_j^{(n)} v_j, \quad (2.24)$$

where the coefficients $u_j^{(n)} \in \mathbb{R}$ are to be determined. The Ritz approximation u_n is defined as the minimizer of the functional

$$J(v) = A(v, v) - 2 \int_{\Omega} v f \, dx \quad (2.25)$$

in the subspace V_n [6]. Since J is a quadratic functional, the minimization problem in V_n is equivalent to the condition that the first variation of J vanishes in all directions of V_n , i.e. to the condition

$$\nabla J(u_n) = 0 \quad \text{in } V_n.$$

This condition yields the following system of equations

$$\sum_{j=1}^n A(v_i, v_j) u_j^{(n)} = \int_{\Omega} v_i f \, dx, \quad i = 1, \dots, n.$$

Introducing the notation

$$K_{ij} := A(v_i, v_j)$$

$$f_i := \int_{\Omega} v_i f \, dx,$$

the coefficients $u_j^{(n)}$ are determined by the linear system

$$\sum_{j=1}^n K_{ij} u_j^{(n)} = f_i, \quad i = 1, \dots, n,$$

or, more compactly, in matrix form;

$$\mathbf{K} \mathbf{u}^{(n)} = \mathbf{f},$$

where $\mathbf{K} = (K_{ij})_{i,j=1}^n$ is the so-called stiffness matrix, $\mathbf{u}^{(n)} = (u_1^{(n)}, \dots, u_n^{(n)})^T$ and $\mathbf{f} = (f_1, \dots, f_n)^T$.

Under the assumptions of Theorem 2.6, the matrix $\mathbf{K}$ is SPD, and therefore the system admits a unique solution for each n . Moreover, the following theorem holds for Ritz approximations [6]

Theorem 2.7 (Convergence of Ritz system of equations to weak solution) *Consider the weak problem: find $u \in V$ such that*

$$A(v, u) = \int_{\Omega} v f \, dx, \quad \forall v \in V$$

and assume that the bilinear form $A(v, u)$ satisfies the assumptions of Theorem 2.6.

Let $\{v_1, v_2, \dots\}$ be the basis of the space V , and for each $n \in \mathbb{N}$ let $V_n = \text{span}\{v_1, \dots, v_n\}$.

For each n , let $u_n \in V_n$ be the Ritz approximation defined by

$$u_n = \sum_{j=1}^n u_j^{(n)} v_j,$$

where $u_j^{(n)}$ are the coefficients determined by the linear system

$$\sum_{j=1}^n A(v_i, v_j) u_j^{(n)} = \int_{\Omega} v_i f \, dx, \quad i = 1, \dots, n,$$

or, equivalently, by

$$\mathbf{K} \mathbf{u}^{(n)} = \mathbf{f}.$$

The sequence $\{u_n\}$ then converges to the weak solution $u(x)$ of the weak problem in the space V , i.e.

$$\lim_{n \rightarrow \infty} \|u_n - u\|_V = 0.$$

Chapter 3

The FEM discretization

3.1 Ritz approximation of the Poisson problem

In the final part of Chapter 2, the Poisson problem with homogeneous mixed Dirichlet-Neumann boundary conditions was reformulated as a variational problem in the space

$$V = \{v \in W_2^{(1)}(\Omega); \quad v = 0 \text{ on } \Gamma_D\},$$

and it was shown that its weak solution coincides with the unique minimizer of the quadratic functional

$$J(v) = A(v, v) - 2 \int_{\Omega} v f \, dx, \quad v \in V$$

$$A(v, u) := \sum_{l=1}^N \int_{\Omega} \frac{\partial v}{\partial x_l} \frac{\partial u}{\partial x_l} \, dx.$$

Using the Ritz method, an approximate solution is obtained by restricting the minimization problem to a finite-dimensional subspace $V_n \subset V$. The corresponding Ritz approximation $u_n \in V_n$ is characterized by the condition

$$\nabla J(u_n) = 0 \quad \text{in } V_n. \tag{3.1}$$

At this point, the problem of solving the Poisson equation has been reduced to the task of constructing a suitable finite-dimensional space V_n and solving the variational problem (3.1) in that space. Once a basis $\{\varphi_1, \dots, \varphi_n\}$ of V_n is fixed, the Ritz approximation can be expressed in the form

$$u_n = \sum_{j=1}^n u_j^{(n)} \varphi_j,$$

where the coefficients $u_j^{(n)} \in \mathbb{R}$ represent the unknown degrees of freedom (DOFs) of the discrete problem.

By applying the procedure from section 2.8.2, we obtain as a result the following system of linear equations (with coefficients $u_j^{(n)}$ as unknowns):

$$\mathbf{K} \mathbf{u}^{(n)} = \mathbf{f}.$$

The matrix K (called the stiffness matrix) is SPD as a direct consequence of the properties of the bilinear form associated with the Poisson problem. However, the concrete structure and sparsity of this matrix depend entirely on the choice of the approximation space V_n and its basis functions.

In the following section, the space V_n will be constructed explicitly for the square domain Ω of the model Poisson problem using very specific basis functions, which will lead to a concrete and efficiently computable stiffness matrix suitable for numerical solutions and for the DDM studied in subsequent sections.

3.2 Discrete approximation space on the square domain

Let us now choose the basis for the space V_n in the following specific way. We will cover the domain Ω with a square grid with step size h , where each resulting square will be further divided into two triangles, thus creating a triangular mesh. First, we assign a global numbering to the nodes of the generated mesh. The particular choice of this numbering is arbitrary and does not influence the finite element discretization; it serves only as an index set for the nodal basis functions. In the Figures and illustrative examples below we use one convenient numbering in which interior nodes are numbered first and boundary nodes afterwards, whereas in the Octave implementation we use a simple lexicographic numbering of all grid points. An example of such a resulting mesh for a total of 36 nodes can be seen in following Figure 3.1

Let us now consider an arbitrary interior node of this mesh (let's say r -th node) and construct a "pyramid" J_r (in the rest of a thesis, we will call it "hat"), with height equal to 1 at node r , whose base Q_r is a hexagon formed by six triangles that share a common vertex at node r . An illustrative example of such a base for node $r = 6$ from the above example is shown in the following Figure 3.2.

For the equation of the individual faces of this hat defined as planes in the coordinate system

Let us now define the function $\varphi_r(x, y)$ such that for points belonging to Q_r , we define $\varphi_r(x, y)$ according to the relations above, and everywhere else in the domain $\bar{\Omega}$, we define $\varphi_r(x, y) = 0$. Each of the functions $\varphi_r(x, y)$, $r = 1, \dots, n$, is continuous in Ω and has piecewise continuous first-order partial derivatives. Therefore, these functions belong to the Sobolev space $W_2^{(1)}(\Omega)$ and, since they are equal to 0 on the boundary $\Gamma = \partial\Omega$, they belong to the space V .

It can be easily proven that these constructed functions are linearly independent in V , and that any linear combination of functions $\varphi_r(x, y)$ also belongs to V . If, in addition, we express the general approximate solution (2.24) (of the Ritz method) as:

$$v_n(x, y) = b_1^{(n)}\varphi_1(x, y) + \dots + b_n^{(n)}\varphi_n(x, y)$$

then from the definition of functions $\varphi_r(x, y)$, it follows that at the r -th interior node, the function $v_n(x, y)$ has the value $b_r^{(n)}$. The functions constructed in this manner are called spline functions (continuous and piecewise polynomial functions).

In addition to the interior nodes, we must also consider the nodes on the boundary. The nodal P_1 basis functions are defined in the same geometric way for all mesh nodes: each basis function φ_r equals 1 at node r , equals 0 at all other nodes, is piecewise linear on each triangle and is supported on the patch of triangles incident to r .

For a boundary node this patch is simply truncated by the boundary, i.e., the support is $Q_r \cap \Omega$, and φ_r is set to zero outside this set. Importantly, homogeneous Neumann boundary conditions are not enforced by modifying the basis functions. They are natural boundary conditions: they enter the variational formulation only through the boundary term obtained after integration by parts, and for homogeneous Neumann data this term vanishes. Consequently, the standard hat functions associated with nodes on Γ_N remain part of the discrete space without any additional constraints.

As stated in the previous section, the stiffness matrix is obtained from the bilinear form evaluated on pairs of basis functions. Since the chosen finite element basis functions have local support, only basis functions whose supports overlap contribute nonzero entries. In other words, most interactions between non-neighboring basis functions are zero. As a result, the stiffness matrix is sparse. This sparsity is crucial for large-scale finite element computations, because it substantially reduces the memory required for storing the matrix and makes it possible to apply iterative methods efficiently. However, the actual computational performance depends strongly on the storage format and on the target hardware. In particular, sparse matrix-vector multiplication may be limited by irregular memory access and memory bandwidth, especially on vector architectures and GPUs. Therefore, practical implementations use specialized sparse matrix formats, such as CSR, ELLPACK, HYB, or SELL-C- σ , to improve memory access patterns and parallel efficiency.

In the implementation part of this thesis, the model problem that involves solving the Poisson equation on a two-dimensional square membrane with mixed Dirichlet-Neumann boundary conditions is represented in the square domain $\Omega = [0, 1] \times [0, 1]$. The domain is discretized by uniform

$n \times n$ subdivision (parameter n in the function `mesh_unit_square_P1.m`), which includes a triangulation of Ω used for the construction of the discrete finite element space $P1$ (polynomials of order 1). In this structured setting, the mesh consists of $(n+1)^2$ nodes and $2n^2$ triangular elements; in the domain decomposition part of the thesis, the same mesh is partitioned into $nSubX \times nSubY$ rectangular subdomains aligned with the grid (so $N = nSubX \cdot nSubY$), where n is chosen divisible by both $nSubX$ and $nSubY$. In particular, the mesh is generated in the function `mesh_unit_square_P1.m`, which constructs the array of global node coordinates p containing all grid points on $[0, 1] \times [0, 1]$ and the element connectivity t , where each triangular element is given by the indices of its three vertices. This representation corresponds directly to the theoretical setting of nodal (hat) basis functions, since each global DOF is associated with one mesh node and each triangle carries three local $P1$ basis functions attached to its vertices.

In addition to the geometric description of triangulation, the mesh generator also provides boundary-related index sets that are used to enforce boundary conditions. In particular, the nodes lying on the left and right edges of $\partial\Omega$ are identified as Dirichlet nodes corresponding to Γ_D and stored for subsequent restriction/elimination of the associated DOFs in the discrete system (see section 3.4). The remaining boundary nodes belong to Γ_N ; however, since the Neumann condition in the model problem is homogeneous, it does not produce any additional boundary contribution to the load functional, and therefore no explicit implementation step is required for Γ_N in the assembly. This separation keeps mesh construction purely geometric, while the algebraic enforcement of Dirichlet constraints is handled later during the formation of the reduced linear system.

3.3 Discrete Ritz system and stiffness matrix construction

In section 3.1, the Poisson problem with homogeneous mixed Dirichlet-Neumann boundary conditions was reformulated as a variational problem and approximated using the Ritz method in a finite-dimensional subspace $V_n \subset V$. Furthermore, in section 3.2 a concrete approximation space was constructed on a triangular mesh of the square domain Ω , together with a specific choice of basis functions $\{\varphi_1, \dots, \varphi_n\}$.

In this section, these ingredients are combined to derive the discrete algebraic system corresponding to the Ritz approximation. Special attention is devoted to the assembly procedure of the stiffness matrix.

3.3.1 Discrete Ritz system for the Poisson problem

Let the approximate solution $u_n \in V_n$ be expressed as a linear combination of the basis functions introduced in section 3.2,

$$u_n(x, y) = \sum_{j=1}^n u_j \varphi_j(x, y),$$

where $c_j \in \mathbb{R}$ are unknown coefficients.

Substituting this representation into the Ritz approximation derived in section 2.8.2 yields the system

$$\sum_{j=1}^n A(\varphi_i, \varphi_j) u_j^{(n)} = \int_{\Omega} \varphi_i f \, dx, \quad i = 1, \dots, n,$$

and using the same notation as in this section, specifically:

$$K_{ij} := A(\varphi_i, \varphi_j) = \int_{\Omega} \nabla \varphi_i \cdot \nabla \varphi_j \, dx$$

$$f_i := \int_{\Omega} \varphi_i f \, dx,$$

the discretized Ritz system takes the resulting matrix form:

$$\mathbf{K} \mathbf{u}^{(n)} = \mathbf{f}.$$

From the symmetry and coercivity of the bilinear form $A(\dots, \dots)$, it immediately follows that the stiffness matrix K is SPD.

3.3.2 Element-wise evaluation of stiffness matrix entries

The stiffness matrix entries are defined by

$$K_{ij} = \int_{\Omega} \nabla \varphi_i \cdot \nabla \varphi_j \, dx.$$

In practical computations, this integral is not evaluated over the entire domain, but is decomposed into contributions from individual triangular elements

$$K_{ij} = \sum_{e=1}^{N_e} \int_{T_e} \nabla \varphi_i \cdot \nabla \varphi_j \, dx.$$

Due to the specific choice of the basis functions φ_r (mainly due to their local support), the integrand is nonzero only on the elements T_e that contain both nodes associated with φ_i and φ_j . This property allows us to evaluate the stiffness matrix in an element-wise manner.

To illustrate this process, consider a triangular element $T_{6,7}$ in our specific example, described in the previous section 3.2. The node indices used in this example correspond to the illustrative numbering shown in Figure 3.1 and are used only as an example; the particular global numbering is arbitrary (in the Octave implementation, we use a simple lexicographic numbering of all grid points). This triangle has nodes 6, 7, and 11 adjacent to it, and, consequently, exactly three basis functions φ_6 , φ_7 and φ_{11} are nonzero on this triangle.

Restricting the bilinear form to this element yields the local element stiffness matrix

$$K^{(T_{6,7})} = \begin{pmatrix} \int_{T_{6,7}} \nabla \varphi_6 \cdot \nabla \varphi_6 \, dx & \int_{T_{6,7}} \nabla \varphi_6 \cdot \nabla \varphi_7 \, dx & \int_{T_{6,7}} \nabla \varphi_6 \cdot \nabla \varphi_{11} \, dx \\ \int_{T_{6,7}} \nabla \varphi_7 \cdot \nabla \varphi_6 \, dx & \int_{T_{6,7}} \nabla \varphi_7 \cdot \nabla \varphi_7 \, dx & \int_{T_{6,7}} \nabla \varphi_7 \cdot \nabla \varphi_{11} \, dx \\ \int_{T_{6,7}} \nabla \varphi_{11} \cdot \nabla \varphi_6 \, dx & \int_{T_{6,7}} \nabla \varphi_{11} \cdot \nabla \varphi_7 \, dx & \int_{T_{6,7}} \nabla \varphi_{11} \cdot \nabla \varphi_{11} \, dx \end{pmatrix}.$$

In our specific geometric configuration, we have a uniform step size h (in the x and y directions) and for element T_{67} we can easily derive formulas for the values of φ_6 , φ_7 and φ_{11} on this element, as follows:

$$\begin{aligned} \varphi_6 : z &= 1 - \frac{x - x_6}{h}, \\ \varphi_7 : z &= 1 + \frac{x - x_7}{h} + \frac{y - y_7}{h}, \\ \varphi_{11} : z &= 1 - \frac{y - y_{11}}{h}. \end{aligned}$$

As these formulas are linear, the gradients $\nabla \varphi_r$ are constants on T_{67} . Consequently, each integral reduces to the product of constant gradients and the area of the triangle (which is equal to $\frac{h^2}{2}$), and the local stiffness matrix can be computed explicitly as follows:

$$K^{(T_{6,7})} = \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} & 0 \\ -\frac{1}{2} & 1 & -\frac{1}{2} \\ 0 & -\frac{1}{2} & \frac{1}{2} \end{pmatrix}. \quad (3.3)$$

The essential outcome of this part is that, for a fixed element T_e we obtain a 3×3 matrix $K^{(e)}$ whose entries correspond to interactions of the three basis functions that are nonzero on this triangle. Thus, each triangle T_e contributes a small, 3×3 matrix, which can be computed independently by evaluating integrals on T_e . Importantly, this computation does not yet require any knowledge of where the element is located in the global mesh or how the global DOFs are numbered. Only in the subsequent assembly step do we use the mesh connectivity to assign the entries of $K_{(e)}$ to the appropriate positions in the global stiffness matrix K , and to sum the contributions of all elements.

The evaluation of a single element stiffness matrix is implemented in `triP1_stiffness.m`. The routine takes the three vertex coordinates of a triangle T_e and constructs the gradients of the local $P1$ basis functions $\{\varphi_1, \varphi_2, \varphi_3\}$. As described in the theory above, these basis functions are affine on each triangle, hence $\nabla \varphi_i$ is constant on T_e , and the bilinear form contribution reduces to

$$K_{ij}^{(e)} = \int_{T_e} \nabla \varphi_i \cdot \nabla \varphi_j \, dx = |T_e| (\nabla \varphi_i \cdot \nabla \varphi_j).$$

Consequently, the implementation computes the triangle area and constant gradients from elementary geometry and then forms the 3×3 matrix by inner products of these gradients.

This element routine is independent of global numbering and uses only local geometric information, which matches the theoretical separation between local element integration and the subsequent global assembly step.

Analogously to the element stiffness matrix, the element contribution to the right-hand side is evaluated locally on each triangle. This step is implemented in the `triP1_load.m` function, which computes the local load vector associated with a single triangular $P1$ element. For a triangle T_e with local basis functions $\{\varphi_1, \varphi_2, \varphi_3\}$, the element load vector is defined by

$$f_m^{(e)} = \int_{T_e} \varphi_m(x) f(x) dx, \quad m = 1, 2, 3.$$

As in the stiffness-matrix case, this computation depends only on the local geometry of the element and is independent of the global numbering of DOFs.

In the implementation, the above integrals are approximated using a simple one-point quadrature rule evaluated at the centroid of the triangle. Since the $P1$ basis functions are affine and satisfy $\varphi_m(x_c) = \frac{1}{3}$ at the centroid x_c , the local load vector can be approximated in compact form

$$f^{(e)} \approx |T_e| f(x_c) \frac{1}{3} (1, 1, 1)^T.$$

This choice is consistent with the low-order finite element approximation used in the thesis and allows the element load vector to be efficiently computed using only the element area and a single evaluation of the source term f . The resulting local vector is then passed to the global assembly procedure, where it is combined with contributions from all elements that share the same global basis functions.

3.3.3 Stiffness matrix assembly

In the previous section, the evaluation of the stiffness matrix was carried out at the level of a single triangular element. For each element T_e a local stiffness matrix $K^{(e)}$ of size 3×3 can be derived by restricting the bilinear form to T_e and by integrating the gradients of corresponding local basis functions over this element only. This computation depends only on the geometry of the element and on choice of the local approximation functions, and it can therefore be performed independently for each triangle of the mesh.

The purpose of the present section is to explain how these local element stiffness matrices are combined to form the global stiffness matrix K from the discrete Ritz system. This combination process, usually referred to as the assembly of the stiffness matrix, establishes the connection between the locally computed quantities and the global numbering of DOFs associated with the finite element discretization.

Once the local stiffness matrix $K^{(e)}$ has been calculated for a given triangular element T_e , it must be incorporated into the global stiffness matrix K of the discrete Ritz system. This requires establishing a correspondence between the local DOFs associated with the element and the global DOFs associated with the entire finite element discretization. This correspondence is determined solely by the mesh connectivity, that is, by the global numbering of the nodes forming the element T_e .

Each triangular element T_e has exactly three vertices, which are assigned local indices $m = 1, 2, 3$. At the same time, these vertices correspond to three nodes of the global mesh, whose global indices we denote by i_1, i_2, i_3 . The local stiffness matrix $K^{(e)} = (K_{mn}^{(e)})_{m,n=1}^3$ therefore represents the contributions of the interactions between the basis functions associated with these three global nodes. During the assembly process, each local entry $K_{mn}^{(e)}$ is added to the entry of the global stiffness matrix with indices (i_m, i_n) .

In this way, the global stiffness matrix is constructed by systematically traversing all elements of the mesh and accumulating their local contributions according to the local-to-global index mapping defined by the mesh topology. The mapping itself does not involve any further integration or approximation; it serves only to identify the correct position in the global matrix where the locally computed values must be inserted.

The general assembly principle described above can be illustrated using a concrete example of the triangular element $T_{6,7}$ introduced in the previous section. For this element, the local stiffness matrix $K_{(T_{6,7})}$ has already been computed explicitly by evaluating the corresponding integrals over $T_{6,7}$ (see 3.3).

As the vertices of the triangle $T_{6,7}$ are associated with the global indices $i_1 = 6, i_2 = 7$ and $i_3 = 11$, the local entries in $K^{(T_{6,7})}$ correspond to interactions between the basis functions associated with these three global nodes. All entries in this local stiffness matrix are thus mapped to the corresponding positions of the global stiffness matrix using the local-to-global index correspondence. This example demonstrated how a single local element stiffness matrix contributes to several entries in the global stiffness matrix.

In the global assembly process, each entry in the global stiffness matrix generally receives contributions from several elements. This is a direct consequence of the fact that neighboring finite elements share common nodes, and therefore share basis functions associated with these nodes.

More precisely, a global stiffness matrix entry K_{ij} corresponds to the interaction of the basis functions φ_i and φ_j . According to the element-wise decomposition of the bilinear form, this entry is given by the sum of integrals over all elements that contain nodes i and j ,

$$K_{ij} = \sum_{T_e \subset \Omega} \int_{T_e} \nabla \varphi_i \cdot \nabla \varphi_j dx.$$

Only those elements T_e for which both basis functions φ_i and φ_j are nonzero contribute to this sum. Consequently, the value of K_{ij} is obtained by adding together the corresponding entries of all

local stiffness matrices $K^{(e)}$ associated with such elements.

During the assembly process, the global stiffness matrix is therefore initialized as a zero matrix and is progressively updated by looping over all elements of the mesh. For each element T_e , its local stiffness matrix $K^{(e)}$ is computed, and its entries are added to the global matrix at the position determined by the local-to/global index mapping. If several elements contribute to the same global matrix entry, their contributions are accumulated through summation. Due to the local support of the basis functions, only elements sharing a common node pair can contribute to a given matrix entry. As a result, most of the entries in the global stiffness matrix remain equal to zero, and the matrix has a sparse structure. This sparsity thus arises naturally from the finite element construction itself.

This completes the construction of the global stiffness matrix (and, analogously, the global load vector) for the discrete Ritz system. In order to obtain the final algebraic problem corresponding to the model Poisson equation, we still need to incorporate the prescribed boundary conditions, which determine the admissible discrete space and lead to a reduced linear system.

The global assembly described in this section is carried out by `assemble_stiffness_P1.m` for the stiffness matrix and by `assemble_load_P1.m` for the load vector. Both functions iterate over all triangles T_e , compute the corresponding local quantities $K^{(e)}$ or $f^{(e)}$ by calling the element routines, and then map their entries into the global objects according to the connectivity array $t(e, :)$. The mapping implements exactly the local-to-global correspondence discussed in the text: local basis functions on T_e are identified with global nodal basis functions associated with the triangle's vertex indices.

Since neighboring triangles share nodes, the global entry K_{ij} (or f_i) generally receives contributions from multiple elements. In the implementation, this is achieved by adding each element contribution into the global matrix/vector at the appropriate indices, which corresponds to summing the element integrals that arise from the decomposition $\Omega = \cup_e T_e$.

3.4 Treatment of boundary conditions

We now incorporate the boundary conditions of the model problem into the discrete Ritz system. In particular, we explain how homogeneous Dirichlet conditions on Γ_D and homogeneous Neumann conditions on Γ_N affect the choice of the discrete approximation space, the set of active DOFs, and the final form of the resulting linear system.

To formulate boundary conditions in Sobolev spaces, the classical point-wise boundary values $u|_\Gamma$ must be replaced by a suitable generalized notion, since functions belonging to Sobolev spaces generally lack sufficient regularity for well-defined point-wise traces. The so-called trace operator provides a mathematically consistent replacement for classical boundary values by extending the restriction $u \mapsto u|_\Gamma$ from smooth functions to functions in $W_2^{(1)}(\Omega)$.

The finite element discretization constructed in the previous sections leads to a discrete Ritz system obtained by restricting the continuous variational problem to a finite-dimensional approximation space. In order for this system to correctly represent the model problem, the prescribed boundary conditions must be taken into account at the discrete level. In this section, we explain how homogeneous Dirichlet and homogeneous Neumann boundary conditions influence the choice of the discrete space, the form of basis functions, the assembly process, and the resulting linear system.

For the structured triangular mesh used in the implementation, this classification is shown in Figure 3.3. The figure distinguishes the interior nodes, the nodes on the homogeneous Neumann boundary and the nodes on the homogeneous Dirichlet boundary. This distinction is important because only the Dirichlet nodes are prescribed and removed from the set of unknowns, while the Neumann boundary nodes remain active DOFs of the discrete problem.

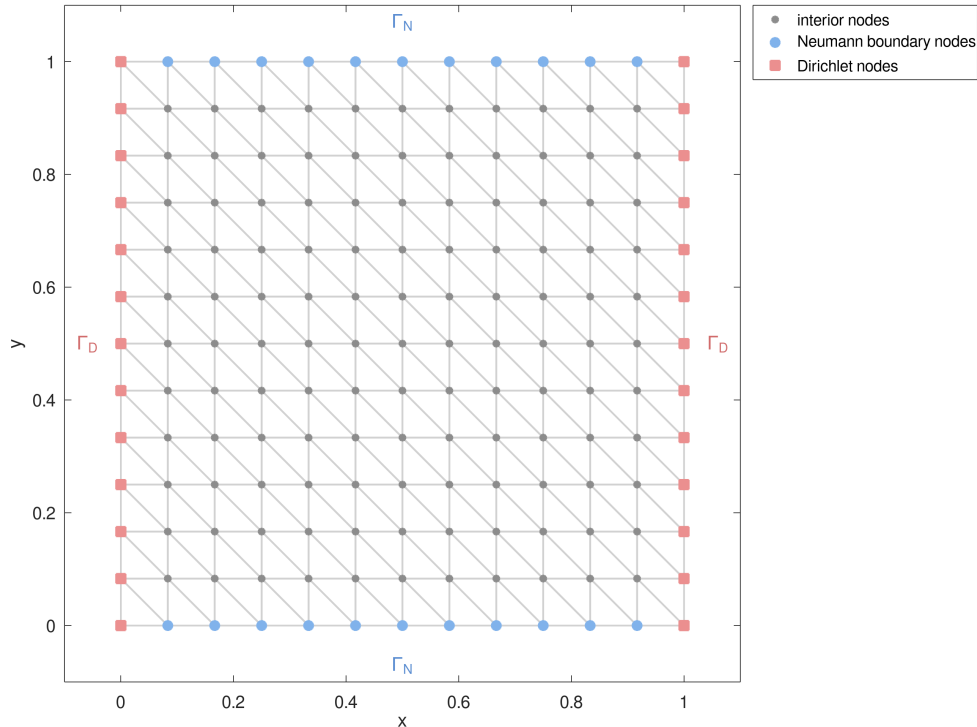


Figure 3.3: Classification of mesh nodes and boundary parts for the model problem. The left and right sides form the Dirichlet boundary Γ_D , while the top and bottom sides form the Neumann boundary Γ_N .

Thus, Figure 3.3 illustrates the discrete counterpart of the boundary decomposition introduced in the continuous model problem. The red square nodes correspond to prescribed homogeneous Dirichlet values and are excluded from the reduced system, whereas the blue Neumann boundary

nodes are treated in the same algebraic way as the gray interior nodes, since the homogeneous Neumann condition produces no additional load contribution.

3.4.1 Discrete space and basis functions

The weak formulation of the Poisson problem is defined in the space

$$V = \{v \in W_2^1(\Omega) : v = 0 \text{ on } \Gamma_D\},$$

where Γ_D denotes the Dirichlet part of the boundary. Consequently, the discrete approximation space V_n is chosen as a finite-dimensional subspace of V ,

$$V_n \subset V,$$

spanned by piecewise linear basis functions associated only with free nodes, i.e. interior nodes and nodes lying on the Neumann boundary.

The nodal hat functions associated with Dirichlet boundary nodes may exist at the mesh level as geometric interpolation functions (for the purpose of a simpler software implementation), but they do not belong to V_n and are therefore excluded from its basis. As a result, the discrete solution is represented as

$$u_n(x) = \sum_{i \in \mathcal{F}} u_i \varphi_i(x),$$

where $\mathcal{F}$ denotes the set of free nodes and $\{\varphi_i\}_{i \in \mathcal{F}}$ is a basis of V_n .

The basis functions included in V_n are standard piecewise linear hat functions. Each such function is nonzero inside the triangle elements sharing its node and vanishes on the boundary of its support. If part of this boundary coincides with Γ_D , the basis function vanishes automatically there.

Inner products defining the stiffness matrix,

$$a(\varphi_i, \varphi_j) = \int_{\Omega} \nabla \varphi_i \cdot \nabla \varphi_j dx,$$

are therefore evaluated only for pairs of basis functions belonging to V_n . Inner products involving hat functions associated with Dirichlet boundary nodes do not appear in the discrete system, since these functions are not part of the basis of V_n .

Importantly, element integrals are still computed on all triangular elements, including those that touch the Dirichlet boundary. However, in the Octave implementation, we first assemble the stiffness matrix and load vector on all mesh nodes and then obtain the reduced system by restricting to the free index set F (see section 3.4.2).

Homogeneous Neumann boundary conditions do not impose additional constraints on the discrete space. In the variational formulation, they enter through boundary integrals that arise after

integration by parts; for a homogeneous Neumann condition, these terms vanish. Consequently, no DOFs are eliminated and no modification of the stiffness matrix is required. Basis functions associated with Neumann boundary nodes are treated in the same way as interior basis functions and remain part of the approximation space V_n .

3.4.2 Assembly and restriction of the discrete system

Let the conceptually assembled stiffness matrix on all mesh nodes be denoted by K . Splitting the DOFs into free nodes $\mathcal{F}$ and Dirichlet nodes $\mathcal{D}$, the full system can be written in block form as

$$\begin{pmatrix} K_{\mathcal{F}\mathcal{F}} & K_{\mathcal{F}\mathcal{D}} \\ K_{\mathcal{D}\mathcal{F}} & K_{\mathcal{D}\mathcal{D}} \end{pmatrix} \begin{pmatrix} u_{\mathcal{F}} \\ u_{\mathcal{D}} \end{pmatrix} = \begin{pmatrix} f_{\mathcal{F}} \\ f_{\mathcal{D}} \end{pmatrix}.$$

Multiplying out the block system gives

$$K_{\mathcal{F}\mathcal{F}} u_{\mathcal{F}} + K_{\mathcal{F}\mathcal{D}} u_{\mathcal{D}} = f_{\mathcal{F}},$$

and

$$K_{\mathcal{D}\mathcal{F}} u_{\mathcal{F}} + K_{\mathcal{D}\mathcal{D}} u_{\mathcal{D}} = f_{\mathcal{D}}.$$

Since the Dirichlet boundary condition is homogeneous, the values at the Dirichlet nodes are prescribed by $u_{\mathcal{D}} = 0$. Therefore, the first block row reduces to

$$K_{\mathcal{F}\mathcal{F}} u_{\mathcal{F}} = f_{\mathcal{F}}. \tag{3.4}$$

The second block row corresponds to equations associated with Dirichlet nodes. These nodes are not unknowns of the reduced problem, since the corresponding basis functions are excluded from the discrete space V_n . Hence, the final linear system is obtained by retaining only the rows and columns associated with the free index set $\mathcal{F}$.

This reduced system corresponds exactly to the assembly of the stiffness matrix directly in the space V_n . Rows and columns associated with Dirichlet nodes do not appear because the corresponding basis functions are excluded from the approximation space [1].

The load vector entries are defined by

$$f_i = \int_{\Omega} \varphi_i f(x) dx, \quad i \in \mathcal{F}.$$

The entries corresponding to Dirichlet boundary nodes do not appear in the discrete system, because the associated basis functions are excluded from the discrete approximation space V_n .

For homogeneous Dirichlet conditions, no additional correction of the right-hand side is required. In the case of non-homogeneous Dirichlet conditions, known boundary values would generate additional contributions to the right-hand side via the eliminated coupling terms.

3.4.3 Resulting discrete problem

After incorporating boundary conditions, the discrete Ritz system takes the final form

$$Ku = f,$$

where K is assembled only on free DOFs, u is the vector of unknown coefficients corresponding to the interior and Neumann boundary nodes and f is the corresponding reduced load vector.

Due to the presence of homogeneous Dirichlet conditions on a nonempty part of the boundary, the stiffness matrix is SPD, and the discrete problem admits a unique solution.

By enforcing homogeneous Dirichlet boundary conditions through the choice of the discrete approximation space, the resulting finite element system is obtained directly in reduced form. This approach is fully consistent with the Ritz method and provides a clear foundation for the numerical solution of the discrete problem developed in the following sections.

The reduction of the discrete system is implemented in `apply_dirichlet_elimination.m`. Conceptually, the routine performs the algebraic restriction implied by choosing the discrete space $V_n \subset V$, i.e., fixing the unknowns at the Dirichlet nodes to zero and retaining only the unknown coefficients associated with the interior and Neumann-boundary nodes. Starting from the assembled system $Ku = f$, the index set is split into free nodes and Dirichlet nodes, and the routine extracts the submatrix $K_{\mathcal{FF}}$ and a subvector $f_{\mathcal{F}}$, which correspond to the reduced system (3.4).

In this way, the implementation follows the same viewpoint as the theory: Dirichlet conditions are enforced not by modifying the differential operator but by restricting the admissible discrete space and solving the resulting reduced linear system for the remaining DOFs.

3.5 Structure of the resulting linear system

The finite element discretization constructed in the previous section leads to a linear system of the form

$$Ku = f,$$

where $K \in \mathbb{R}^{n \times n}$ is the stiffness matrix assembled over the discrete space V_n , $u \in \mathbb{R}^n$ is the vector of unknown coefficients associated with the interior and Neumann boundary nodes, and $f \in \mathbb{R}^n$ is the corresponding load vector. The dimension n equals the number of free DOFs of the discretized domain.

The stiffness matrix K is SPD. Symmetry follows directly from the symmetry of the bilinear form defining the Ritz system, whereas positive definiteness is ensured by the ellipticity of the Poisson operator together with the presence of homogeneous Dirichlet boundary conditions on a nonempty part of the boundary. As a result, the discrete problem admits a unique solution.

Furthermore, K is a sparse matrix. This sparsity is a consequence of the local support of the chosen piecewise linear basis functions: nonzero matrix entries arise only between basis functions whose supports overlap, which corresponds to nodes connected through one or more common triangular elements. Consequently, each row of K contains only a small number of nonzero entries, independent of the total number of DOFs.

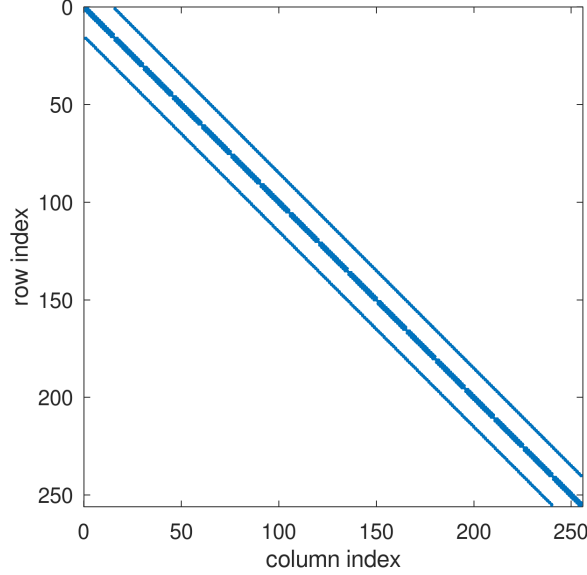


Figure 3.4: Sparsity pattern of the reduced stiffness matrix K_{FF} obtained after imposing the homogeneous Dirichlet boundary conditions.

The sparsity pattern of the reduced stiffness matrix is illustrated in Figure 3.4. The visible banded structure reflects the local connectivity of the structured triangular mesh: each basis function interacts only with basis functions associated with neighboring nodes. These structural properties — symmetry, positive definiteness, and sparsity — are characteristic of finite element discretizations of elliptic problems and play a crucial role in the efficient numerical solution of the resulting linear system.

Chapter 4

Substructuring and Interface Framework for DDM

In many large-scale applications, the solution of elliptic boundary value problems by the finite element method leads, after discretization, to very large sparse systems of linear algebraic equations. The direct solution of these systems becomes rapidly impractical as the number of DOFs increases, and standard iterative methods applied to the global system may converge slowly due to poor conditioning, which typically deteriorates as the mesh is refined. Although the variational formulation and the construction of the conforming finite element space developed in the previous Chapter remain entirely unchanged, the efficient treatment of such systems requires techniques with good numerical scalability and with an algebraic structure suitable for parallel execution. In this thesis, we distinguish between two related but different notions of scalability. Numerical scalability refers to the behavior of the preconditioned operator and of the iterative method when the mesh is refined or when the domain decomposition is changed; it is measured mainly by spectral indicators, condition numbers, and Krylov iteration counts. Parallel scalability, on the other hand, refers to the behavior of the actual implementation on multiple processors; it is measured by wall-clock time, speedup, efficiency, communication overhead, and load balance. In the present Chapter, the emphasis is mainly on the first aspect (i.e. numerical scalability): constructing an interface formulation and a coarse continuity mechanism that lead to robust preconditioned iterations. The second aspect, namely the behavior of actual parallel implementations, is considered later in the PETSc benchmark part of Chapter 6 [8].

In many large-scale applications, the solution of elliptic boundary value problems by the finite element method leads, after discretization, to very large sparse systems of linear algebraic equations. The direct solution of these systems becomes rapidly impractical as the number of DOFs increases, and standard iterative methods applied to the global system may converge slowly due to poor conditioning, which typically deteriorates as the mesh is refined. Although the variational formulation and the construction of the conforming finite element space developed in the previ-

ous Chapter remain entirely unchanged, the efficient treatment of such systems requires techniques with good numerical scalability and a structure suitable for parallel execution (in the context of this thesis, numerical scalability refers to robustness of the preconditioned iteration under refinement and increasing decomposition complexity; parallel scalability refers to efficient use of multiple processors) [8].

Non-overlapping DDM provide a natural framework for this purpose, since they are based on a partition of the computational domain into smaller subregions and allow the discrete problem to be organized in terms of local subdomain contributions coupled only through their common interfaces. This reformulation is therefore motivated not by a modification of the underlying partial differential equation or its finite element approximation but by the need to design solution procedures suitable for large-scale computations [9].

The purpose of this Chapter is to introduce such a reformulation of the discrete finite element problem in a form that is suitable for non-overlapping substructuring. The starting point remains the global Ritz system obtained in Chapter 3; however, instead of treating it as a single monolithic problem, the computational domain is first partitioned into a collection of non-overlapping subdomains. On each subdomain, the finite element space and the corresponding local stiffness operators are considered separately. By an exact elimination of the interior DOFs, the discrete problem is reduced to an equivalent system formulated only on the subdomain interfaces. This interface formulation provides the basis for the introduction of appropriate interface spaces and continuity mechanisms, leading to a unified structural framework in which different non-overlapping DDM can later be formulated and compared.

This Chapter is organized as follows. In section 4.1, the geometric partition of the domain into non-overlapping subdomains is introduced together with the associated decomposition of the global finite element space into local subdomain spaces and interior/interface DOFs. section 4.2 then derives the exact reduction of the discrete problem to subdomain interfaces by eliminating interior unknowns and formulating the corresponding Schur complement system. In section 4.3 and 4.4, the interface setting is further developed by introducing product interface spaces, continuity constraints, and the concept of selected primal DOFs that enforce coarse continuity across subdomains. Finally, section 4.5 explains how the structural framework established in the preceding sections provides a common foundation for both the FETI-DP and the BDDC methods, whose algorithmic formulations will be addressed in later sections. We therefore begin by defining the non-overlapping domain decomposition and the corresponding local finite element spaces. Finally, section 4.6 presents basic characteristics of actual Octave implementation of this topic.

4.1 Domain decomposition and local finite element spaces

We begin by introducing the geometric partition of the computational domain and the corresponding decomposition of the finite element space into local subdomain contributions. This establishes the

notation for subdomains, interfaces, and local DOFs used throughout the remainder of the section.

4.1.1 Non-overlapping decomposition of the domain

Let $\Omega \subset \mathbb{R}^2$ denote the computational domain introduced in the previous sections. We assume that Ω is partitioned into a finite number N of non-overlapping subdomains Ω_i , such that

$$\bar{\Omega} = \bigcup_{i=1}^N \bar{\Omega}_i, \quad \Omega_i \cap \Omega_j = \emptyset \quad \text{for } i \neq j. \quad (4.1)$$

This decomposition is called non-overlapping, since the interiors of distinct subdomains do not intersect, and the subdomains cover the entire domain exactly. The subdomains are assumed to be shape-regular and compatible with the underlying finite element triangulation, meaning that each element of the global mesh belongs to exactly one subdomain [8].

In the present thesis, the model problem is formulated on the square domain $\Omega = [0, 1] \times [0, 1]$ introduced in Chapter 2 and Chapter 3. For simplicity and clarity of presentation, we will therefore restrict our attention to the decompositions of Ω into a structured collection of identical square subdomains. Such a partition is particularly convenient for the uniform triangulations considered in our version of the finite element discretization and allows the interface structure to be described explicitly. Although more general unstructured decompositions are possible, the rectangular setting is sufficient to illustrate all essential concepts of non-overlapping substructuring and interface formulations.

The structured decomposition used as the basic model throughout this thesis is shown schematically in Figure 4.1. The figure is drawn as a schematic separated view: the subdomains are shown with artificial gaps in order to emphasize that, in the substructuring framework, each subdomain carries its own local finite element mesh and its own local DOF numbering.

Although the blocks in Figure 4.1 are displayed separately, they represent a partition of one computational domain. The separation is only visual; mathematically, neighboring subdomains share interface nodes along their common artificial boundaries. These shared boundaries are precisely the interfaces through which the local subdomain problems will be coupled.

The boundaries of the subdomains consist partly of portions lying on the physical boundary $\partial\Omega$, and partly of artificial boundaries created by the decomposition. The latter form the subdomain interfaces, which are the key objects in non-overlapping substructuring methods.

For each subdomain Ω_i , we define its local interface boundary by

$$\Gamma_i = \partial\Omega_i \setminus \partial\Omega,$$

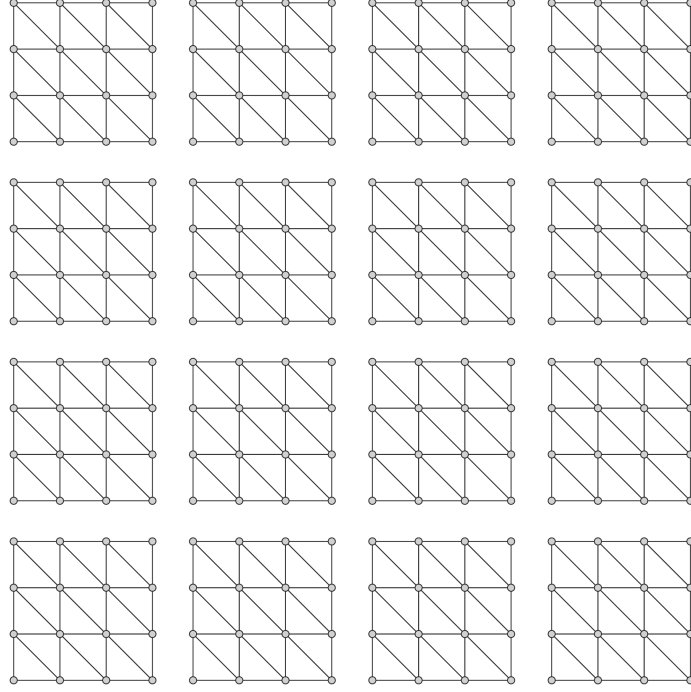


Figure 4.1: Structured non-overlapping decomposition of the computational domain into subdomains, shown with artificial gaps for clarity. Each block represents one subdomain together with its local triangular finite element mesh.

and we denote the global interface of the decomposition by

$$\Gamma = \bigcup_{i=1}^N \Gamma_i.$$

Thus, Γ represents the collection of all boundaries shared by neighboring subdomains.

It is important to emphasize that this domain decomposition does not modify the underlying boundary value problem or its FEM discretization derived in Chapter 3. The global variational formulation remains unchanged; only the algebraic organization of the discrete problem is reformulated in terms of subdomain contributions and their interaction across Γ .

4.1.2 Local finite element spaces induced by restriction to a subdomain

The non-overlapping decomposition introduced in the previous section provides a purely geometric partition of the computational domain Ω . In order to reformulate the discrete finite element problem

in terms of subdomain contributions, it is necessary to introduce the corresponding decomposition of the FEM approximation space.

Recall from Chapter 3 that the finite element discretization of the model Poisson problem was obtained by restricting the variational formulation to a finite-dimensional subspace $V_n \subset V$, spanned by the nodal hat functions associated with the free (non-Dirichlet) mesh nodes. The discrete solution was therefore represented in the form

$$u_n(x, y) = \sum_{j=1}^n u_j \phi_j(x, y),$$

and the corresponding Ritz approximation leads to the global stiffness system,

$$Ku = f.$$

When the domain Ω is partitioned into subdomains $\{\Omega_i\}_{i=1}^N$, each subdomain inherits its local triangulation directly from the global finite element mesh introduced in Chapter 3. More precisely, for each subdomain Ω_i , we define the associated local mesh by

$$\mathcal{T}_h^{(i)} = \{T_e \in \mathcal{T}_h ; T_e \subset \overline{\Omega_i}\},$$

that is, the collection of all triangular elements whose support lies in $\overline{\Omega_i}$. Since the decomposition is assumed to be compatible with the global triangulation, each element belongs to exactly one subdomain, and no re-discretization is introduced.

The restriction of discrete finite element functions to a subdomain then defines the corresponding local approximation space. For each $i = 1, \dots, N$, we introduce

$$V_n(\Omega_i) = \{v|_{\Omega_i} ; v \in V_n\}, \quad (4.2)$$

which consists of continuous piecewise linear functions defined on the local mesh $\mathcal{T}_h^{(i)}$. Thus, the local spaces are not constructed independently, but arise directly as restrictions of the global finite element space.

In this setting, any discrete function $v_n \in V_n$ can be viewed as a collection of local subdomain components,

$$v_n \longleftrightarrow \begin{pmatrix} v_n^{(1)} \\ \vdots \\ v_n^{(N)} \end{pmatrix}, \quad v_n^{(i)} = v_n|_{\Omega_i}.$$

This local viewpoint is fundamental for substructuring methods: the discrete problem will later be expressed through local stiffness operators acting on the subdomain functions $v_n^{(i)}$, while the coupling between subdomains will be enforced only through compatibility conditions on the interface Γ .

4.1.3 Interior and interface DOFs

In the subdomain setting introduced above, the discrete approximation space is represented in terms of local restrictions $V_n(\Omega_i)$. In order to reformulate the global finite element system in a form suitable for substructuring, it is now necessary to distinguish between DOFs that are internal to a subdomain and those that lie on subdomain interfaces.

Recall that the discrete problem constructed in Chapter 3 is formulated in terms of the nodal DOFs associated with the basis functions $\{\varphi_j\}_{j=1}^n$, and the stiffness matrix K was assembled over the set of free nodes, i.e. interior nodes and Neumann boundary nodes. In the present subdomain decomposition, each such node belongs either to exactly one subdomain interior or to the boundary shared by two or more neighboring subdomains.

For a given subdomain Ω_i , we therefore introduce the decomposition of its local index set into two disjoint parts,

$$\mathcal{F}_i = \mathcal{F}_I^{(i)} \cup \mathcal{F}_\Gamma^{(i)}, \quad \mathcal{F}_I^{(i)} \cap \mathcal{F}_\Gamma^{(i)} = \emptyset,$$

where:

- $\mathcal{F}_I^{(i)}$ denotes the set of interior DOFs, corresponding to nodes whose support lies entirely in the interior of Ω_i ,
- $\mathcal{F}_\Gamma^{(i)}$ denotes the set of interface DOFs, corresponding to nodes located on the subdomain boundary Γ_i .

Thus, the interface DOFs represent exactly those unknowns through which neighboring subdomains interact. All coupling between subdomains in the global finite element system is transmitted through these interface nodes.

With this notation, any discrete function restricted to Ω_i can be expressed in terms of interior and interface components. At the algebraic level, the vector of local coefficients on Ω_i can be written in block form as

$$u^{(i)} = \begin{pmatrix} u_I^{(i)} \\ u_\Gamma^{(i)} \end{pmatrix}, \quad (4.3)$$

where $u_I^{(i)}$ collects the unknowns associated with $\mathcal{F}_I^{(i)}$, and $u_\Gamma^{(i)}$ collects those associated with $\mathcal{F}_\Gamma^{(i)}$ [10].

It is important to emphasize that this decomposition introduces two complementary viewpoints. In the global assembled formulation of Chapter 3, each free node corresponds to a single global DOF, and the stiffness matrix K is assembled accordingly. In contrast, in the local subdomain formulation, each subdomain possesses its own local numbering of DOFs. In particular, a node located on the interface Γ belongs simultaneously to several subdomains and therefore appears in each corresponding local space $V_n(\Omega_i)$.

At this stage, the discrete problem is still globally conforming, and interface values are single-valued. However, separation into interior and interface unknowns prepares the key step of substructuring methods: Interior DOFs $u_I^{(i)}$ will later be eliminated locally, leading to an equivalent formulation posed only in terms of interface unknowns $u_\Gamma^{(i)}$.

The resulting local classification of DOFs is summarized in Figure 4.2. In addition to the interior and interface nodes introduced in this subsection, the figure also marks the Dirichlet nodes inherited from the physical boundary and, anticipating Section 4.4, the corner/primal nodes used for coarse continuity enforcement.

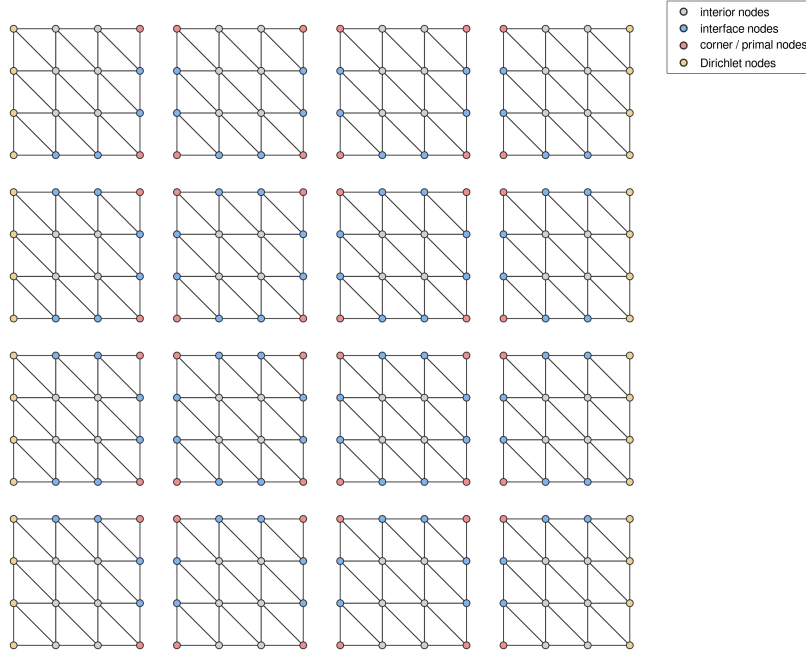


Figure 4.2: Classification of local degrees of freedom in the exploded subdomain decomposition. Interior nodes, interface nodes, corner/primal nodes, and Dirichlet nodes are distinguished by colour.

The important point shown in Figure 4.2 is that the same physical interface node may appear in several local subdomain meshes. This is the geometric origin of duplicated interface unknowns in the product interface space introduced later. The red corner/primal nodes represent the small subset of interface DOFs that will remain globally identified, while the remaining interface DOFs will be treated through explicit continuity constraints.

4.2 Subdomain problems, harmonic extension, and Schur complement reduction

Using the decomposition of DOFs introduced in section 4.1, we now eliminate the interior unknowns by an exact algebraic procedure and obtain the corresponding interface formulation. This reduction leads naturally to discrete harmonic extension and to the local Schur complement operators.

4.2.1 Local subdomain systems and block decomposition

The starting point of the substructuring framework remains the global discrete Ritz system derived in Chapter 3,

$$Ku = f,$$

where K is the stiffness matrix assembled over the finite element space V_n , and u is the vector of nodal coefficients corresponding to the free DOFs. The purpose of the present section is to reinterpret this global system from the viewpoint of the subdomain decomposition introduced in section 4.1 and to formulate the corresponding local algebraic problems on each subdomain.

Let $\{\Omega_i\}_{i=1}^N$ be the non-overlapping partition of Ω defined in (4.1), and let $V_n(\Omega_i)$ denote the local restriction spaces introduced in (4.2). Since the global stiffness matrix is assembled element-wise as a sum of contributions over the triangulation, restriction of the finite element discretization to a subdomain naturally leads to a local stiffness operator associated with Ω_i . More precisely, for each subdomain we may define a local stiffness matrix $K^{(i)}$ and a corresponding local load vector $f^{(i)}$, obtained by assembling only those element contributions that belong to Ω_i .

In this way, the discrete problem can be viewed as a collection of subdomain systems of the form

$$K^{(i)}u^{(i)} = f^{(i)}, \quad i = 1, \dots, N,$$

where $u^{(i)}$ denotes the vector of local nodal unknowns on Ω_i . It is important to emphasize that these local systems do not represent independent discretizations of separate boundary value problems. They arise solely from the restriction of the global finite element system to the elements and DOFs associated with Ω_i , and they remain coupled through the interface unknowns.

The key feature of substructuring methods is that the local coefficient vector $u^{(i)}$ can be decomposed according to the interior/interface splitting introduced in section 4.1.3. Using the notation of (4.3), we write

$$u^{(i)} = \begin{pmatrix} u_I^{(i)} \\ u_\Gamma^{(i)} \end{pmatrix},$$

where $u_I^{(i)}$ collects the DOFs associated with the interior index set $\mathcal{F}_I^{(i)}$, and $u_\Gamma^{(i)}$ collects those associated with the interface index set $\mathcal{F}_\Gamma^{(i)}$.

With respect to this decomposition, the local stiffness matrix takes the corresponding block form

$$K^{(i)} = \begin{pmatrix} K_{II}^{(i)} & K_{I\Gamma}^{(i)} \\ K_{\Gamma I}^{(i)} & K_{\Gamma\Gamma}^{(i)} \end{pmatrix},$$

and the local right-hand side splits analogously as

$$f^{(i)} = \begin{pmatrix} f_I^{(i)} \\ f_\Gamma^{(i)} \end{pmatrix}.$$

The subdomain system can therefore be written explicitly as

$$\begin{pmatrix} K_{II}^{(i)} & K_{I\Gamma}^{(i)} \\ K_{\Gamma I}^{(i)} & K_{\Gamma\Gamma}^{(i)} \end{pmatrix} \begin{pmatrix} u_I^{(i)} \\ u_\Gamma^{(i)} \end{pmatrix} = \begin{pmatrix} f_I^{(i)} \\ f_\Gamma^{(i)} \end{pmatrix}.$$

This block structure reveals the fundamental separation between local interior interactions and interface coupling. The matrix block $K_{II}^{(i)}$ represents the stiffness interactions entirely within the interior of Ω_i , whereas the off-diagonal blocks $K_{I\Gamma}^{(i)}$ and $K_{\Gamma I}^{(i)}$ describe the influence of interface DOFs on the subdomain interior and vice versa. The coupling between different subdomains is therefore transmitted exclusively through the interface components $u_\Gamma^{(i)}$ [10].

The formulation established in this section provides the algebraic basis for the reduction step of substructuring. In the next section, the interior unknowns $u_I^{(i)}$ will be eliminated locally by block Gaussian elimination, leading to an equivalent interface problem expressed only in terms of the interface DOFs.

4.2.2 Elimination of interior unknowns

In the previous section, the local subdomain systems were written in block form with respect to the decomposition of DOFs into interior and interface components. For each subdomain Ω_i , we obtained the discrete system

$$\begin{pmatrix} K_{II}^{(i)} & K_{I\Gamma}^{(i)} \\ K_{\Gamma I}^{(i)} & K_{\Gamma\Gamma}^{(i)} \end{pmatrix} \begin{pmatrix} u_I^{(i)} \\ u_\Gamma^{(i)} \end{pmatrix} = \begin{pmatrix} f_I^{(i)} \\ f_\Gamma^{(i)} \end{pmatrix}.$$

The purpose of this section is to eliminate the interior unknowns $u_I^{(i)}$ by an exact algebraic procedure and thereby reduce the problem to an equivalent formulation posed only on the interface DOFs [1].

We begin by considering the first block row of the local system, which corresponds to the interior equations,

$$K_{II}^{(i)} u_I^{(i)} + K_{I\Gamma}^{(i)} u_\Gamma^{(i)} = f_I^{(i)}.$$

The interior DOFs in the set $\mathcal{F}_I^{(i)}$ are associated with basis functions $\{\varphi_j\}$ whose support lies strictly inside the subdomain Ω_i . In particular, these functions vanish on the artificial boundary Γ_i . As a consequence, the matrix block $K_{II}^{(i)}$ represents the stiffness operator restricted to functions satisfying homogeneous Dirichlet conditions on Γ_i , and it is therefore SPD. Hence, $K_{II}^{(i)}$ is invertible and the interior unknowns can be expressed uniquely in terms of the interface values.

Solving the interior equation for $u_I^{(i)}$ yields

$$u_I^{(i)} = (K_{II}^{(i)})^{-1} (f_I^{(i)} - K_{II}^{(i)} u_\Gamma^{(i)}).$$

This relation shows that, once the interface component $u_\Gamma^{(i)}$ is prescribed, the interior solution on Ω_i can be recovered by solving a local Dirichlet-type problem.

Substituting this expression into the second block row of the system,

$$K_{\Gamma I}^{(i)} u_I^{(i)} + K_{\Gamma \Gamma}^{(i)} u_\Gamma^{(i)} = f_\Gamma^{(i)},$$

we obtain an equation involving only the interface unknowns [9],

$$(K_{\Gamma \Gamma}^{(i)} - K_{\Gamma I}^{(i)} (K_{II}^{(i)})^{-1} K_{II}^{(i)}) u_\Gamma^{(i)} = f_\Gamma^{(i)} - K_{\Gamma I}^{(i)} (K_{II}^{(i)})^{-1} f_I^{(i)}. \quad (4.4)$$

Thus, the effect of eliminating the interior DOFs is to produce a reduced operator acting solely on the interface component $u_\Gamma^{(i)}$, while the influence of the interior force load enters only through the modified right-hand side.

It is important to emphasize that this reduction is purely algebraic and exact: no approximation has been introduced, and the resulting interface formulation is fully equivalent to the original finite element system. Moreover, since the elimination is performed independently on each subdomain, the computation of the interior contributions is completely local.

Once the interface unknowns have been determined, the interior DOFs can be recovered uniquely from the local relation above. This reconstruction viewpoint motivates the concept of discrete harmonic extension, which will be introduced in the next section.

4.2.3 Discrete harmonic extension and the Schur complement

The elimination procedure derived in the previous section shows that, on each subdomain Ω_i , the interior DOFs $u_I^{(i)}$ can be expressed uniquely in terms of the interface component $u_\Gamma^{(i)}$. More precisely, we obtained the relation

$$u_I^{(i)} = (K_{II}^{(i)})^{-1} (f_I^{(i)} - K_{II}^{(i)} u_\Gamma^{(i)}),$$

which demonstrates that the interior solution depends on two contributions: the local load vector $f_I^{(i)}$, representing the distributed load inside the subdomain, and the prescribed interface values

$u_\Gamma^{(i)}$, through which neighboring subdomains interact [1]. Thus, once the interface unknowns are fixed, the interior unknowns are not globally coupled but can be recovered uniquely by solving a local subdomain problem, with the interior load entering only through the local right-hand side.

In order to describe the influence of the interface coupling separately from the influence of the interior loads, it is convenient to first consider the homogeneous interior problem obtained by setting $f_I^{(i)} = 0$. This choice isolates the part of the interior field generated solely by the interface trace and leads naturally to the notion of the discrete harmonic extension.

We therefore consider the homogeneous interior equation

$$K_{II}^{(i)} u_I^{(i)} + K_{I\Gamma}^{(i)} u_\Gamma^{(i)} = 0.$$

Since the block $K_{II}^{(i)}$ is invertible, the corresponding interior solution is given by

$$u_I^{(i)} = -(K_{II}^{(i)})^{-1} K_{I\Gamma}^{(i)} u_\Gamma^{(i)}.$$

This relation defines a linear mapping from the interface DOFs to the interior DOFs. The interior field obtained in this way is called the discrete harmonic extension of the interface values $u_\Gamma^{(i)}$ into the subdomain interior. In other words, for a prescribed trace on Γ_i , the harmonic extension represents the unique discrete solution inside Ω_i of the homogeneous subdomain problem.

More generally, the full subdomain solution can be decomposed into the sum of two components,

$$u^{(i)} = u_p^{(i)} + u_h^{(i)},$$

where $u_p^{(i)}$ denotes a particular solution generated by the interior load vector $f_I^{(i)}$ with vanishing interface trace, and $u_h^{(i)}$ denotes the harmonic correction induced by the interface values. This decomposition separates the influence of the interior loads from the influence of the subdomain interaction across the interface Γ .

The Schur complement operator introduced implicitly in (4.4) can now be interpreted directly in terms of harmonic extension. Indeed, starting from the second block row of the local system,

$$K_{II}^{(i)} u_I^{(i)} + K_{I\Gamma}^{(i)} u_\Gamma^{(i)} = f_\Gamma^{(i)},$$

we substitute the interior solution obtained from the first block row. In the homogeneous case, the interior unknowns are given by the harmonic extension

$$u_I^{(i)} = -(K_{II}^{(i)})^{-1} K_{I\Gamma}^{(i)} u_\Gamma^{(i)}.$$

Inserting this expression into the interface equation yields

$$K_{\Gamma I}^{(i)} \left(-(K_{II}^{(i)})^{-1} K_{I\Gamma}^{(i)} u_\Gamma^{(i)} \right) + K_{\Gamma\Gamma}^{(i)} u_\Gamma^{(i)} = f_\Gamma^{(i)}.$$

Collecting the terms multiplying $u_\Gamma^{(i)}$, we obtain the reduced interface operator called Schur complement

$$S^{(i)} = K_{\Gamma\Gamma}^{(i)} - K_{\Gamma I}^{(i)}(K_{II}^{(i)})^{-1}K_{I\Gamma}^{(i)}.$$

Thus, the Schur complement represents the discrete interface operator obtained after eliminating the harmonic interior extension [8], [9]. It maps prescribed interface values to the corresponding interface response of the subdomain and may be viewed as the algebraic analogue of the continuous Steklov–Poincare operator [7].

The introduction of harmonic extension and Schur complements is essential for the substructuring framework, since both concepts describe two complementary aspects of the same reduction procedure. Harmonic extension provides the functional interpretation: it characterizes how interface values uniquely determine the interior field inside each subdomain. The Schur complement provides the corresponding algebraic representation: after eliminating the interior unknowns, one obtains an operator that acts only on the interface DOFs. Together, they allow the original finite element system to be reduced exactly to an equivalent formulation posed only on the interface, while all interior contributions are handled locally and independently on each subdomain. This reduction forms the basis for the interface spaces and continuity mechanisms introduced in the following part.

4.2.4 Global Schur complement interface problem

In the previous sections, the local subdomain systems were decomposed into interior and interface DOFs, and the interior unknowns were eliminated exactly by block Gaussian elimination. This reduction led to the local Schur complement operators

$$S^{(i)} = K_{\Gamma\Gamma}^{(i)} - K_{\Gamma I}^{(i)}(K_{II}^{(i)})^{-1}K_{I\Gamma}^{(i)},$$

which act only on the interface DOFs $u_\Gamma^{(i)}$. The purpose of the present section is to show how these local reduced operators combine into a global interface problem formulated on the global subdomain interface Γ . This interface problem represents the first complete formulation of the discrete Poisson equation in which the global coupling is carried exclusively by interface unknowns, while all interior DOFs are handled locally.

Recall that the original finite element discretization of the Poisson problem leads to the global linear system

$$Ku = f.$$

After the domain is partitioned into subdomains, the unknown vector on each subdomain is split as

$$u^{(i)} = \begin{pmatrix} u_I^{(i)} \\ u_\Gamma^{(i)} \end{pmatrix}.$$

Eliminating the interior unknowns $u_I^{(i)}$ produces, on each subdomain, the reduced interface equation

$$S^{(i)} u_\Gamma^{(i)} = g^{(i)}, \quad (4.5)$$

where the reduced right-hand side is given by

$$g^{(i)} = f_\Gamma^{(i)} - K_{\Gamma I}^{(i)} (K_{II}^{(i)})^{-1} f_I^{(i)}.$$

This formula makes explicit an important point: the interface problem is influenced not only by interface load contributions $f_\Gamma^{(i)}$, but also indirectly by the interior distributed load $f_I^{(i)}$, whose effect is transmitted to the interface through the correction term arising from the elimination of the interior unknowns. Thus, even though interior unknowns are removed from the global coupling, the interior force load still contributes through the reduced right-hand side $g^{(i)}$.

The interface DOFs are shared between neighboring subdomains. Therefore, the reduced equations (4.5) cannot be solved independently: the interface unknowns must satisfy compatibility and continuity across subdomain boundaries. When the local Schur complement contributions are assembled over all subdomains, one obtains a global reduced system of the form

$$S u_\Gamma = g, \quad (4.6)$$

where u_Γ is the vector of global interface unknowns on Γ , S is the global Schur complement operator assembled from the local contributions $S^{(i)}$, and g is the corresponding global reduced load vector assembled from $g^{(i)}$. This system is fully equivalent to the original finite element problem: solving (4.6) for u_Γ determines the interface trace of the solution, and then the interior fields can be recovered locally.

Once the interface unknowns $u_\Gamma^{(i)}$ are known, the interior unknowns on each subdomain are obtained from the local reconstruction formula

$$u_I^{(i)} = (K_{II}^{(i)})^{-1} (f_I^{(i)} - K_{II}^{(i)} u_\Gamma^{(i)}).$$

This shows that the global solution process can be viewed as consisting of two stages: first, one solves the global interface problem for u_Γ , and second, one recovers the interior solutions independently on each subdomain by local Dirichlet solves. The second stage is completely parallelizable, since each subdomain interior depends only on its own interface values and its own interior load.

At this point, the reduction to the Schur complement system

$$S u_\Gamma = g$$

already represents a valid solution strategy for the discrete Poisson problem: instead of solving the full global system $Ku = f$, one may solve the smaller interface system first and then reconstruct

the interior solution locally. This is the first place in the substructuring framework where the reformulation becomes an explicit computational method. However, although this reduced interface formulation is mathematically exact, it is not yet an efficient method for large-scale computations. The Schur complement matrix S is typically much denser than the original stiffness matrix K , even if K is sparse, and applying S requires repeated local solves with $K_{II}^{(i)}$. Moreover, the conditioning of the interface system deteriorates as the number of subdomains increases, so naive iterative solvers may converge slowly. Thus, the Schur complement system provides the correct interface formulation, but it does not by itself guarantee numerical scalability.

The reduced interface problem (4.6) nevertheless forms the fundamental starting point for modern non-overlapping DDM. Rather than solving it directly, advanced methods introduce additional structure in order to obtain scalable algorithms. In particular, the next sections develop product interface spaces, continuity constraints across subdomains, and primal DOFs and coarse spaces, which lead to practical and efficient solvers like FETI-DP and BDDC. Thus, the Schur complement formulation derived here serves as the essential bridge between the classical finite element system and the modern interface-based domain decomposition algorithms studied in the remainder of the thesis.

4.3 Product interface space and continuity constraints

Although the global Schur complement system derived in section 4.2 provides an exact reformulation of the finite element problem in terms of interface unknowns, the continuity of these unknowns is enforced implicitly through the standard assembly process. In the reduced system

$$Su_\Gamma = g,$$

the interface vector u_Γ is single-valued across the subdomain boundaries, and therefore the coupling between neighboring subdomains is built into the global structure of the assembled interface space. In particular, the unknown satisfies

$$u_\Gamma \in \widehat{W},$$

where $\widehat{W}$ denotes the assembled interface space. Although this formulation is mathematically correct, it does not provide the most convenient setting for the development of modern DDM whose essential feature is the explicit treatment of interface coupling.

In order to obtain a unified framework in which different strategies for enforcing continuity can be described and compared, it is advantageous to introduce a reformulation in which interface DOFs are represented independently on each subdomain. This leads to the concept of a product, or duplicated, interface space

$$W = \prod_{i=1}^N W_i,$$

in which each subdomain carries its own local copy of the interface trace. In the resulting formulation, continuity across the interface is no longer imposed automatically but becomes an explicit condition that can be enforced separately by suitable constraints. The assembled interface space can then be obtained as a constrained subspace of the product space, and the interface coupling mechanism is made fully transparent.

This structural viewpoint plays a central role in non-overlapping substructuring methods. By separating the local interface contributions from the global continuity requirements, it becomes possible to formulate the subdomain interaction in a flexible and solver-independent manner. In particular, the introduction of duplicated interface variables provides the natural setting for constraint-based formulations and for the construction of coarse continuity mechanisms, which are essential ingredients of scalable methods such as FETI-DP and BDDC.

At the same time, this reformulation has important computational consequences. Since each subdomain is associated with its own local interface unknowns and its own local Schur complement operator, many operations remain completely independent and can be performed in parallel. The global problem is then reduced to enforcing compatibility conditions between neighboring subdomains, while the dominant subdomain computations are localized. Thus, the product interface framework not only clarifies the mathematical structure of substructuring methods, but also explains why these methods are suitable for parallel implementation. Their numerical scalability, however, still depends on the additional coarse continuity mechanism introduced through primal DOFs.

4.3.1 Assembled interface space

The reduction developed in section 4.2 shows that, after the elimination of all interior DOFs, the global finite element problem can be reformulated as an interface system defined on the union of the subdomain interfaces. In particular, the Schur complement reduction leads to the assembled interface equation

$$Su_\Gamma = g,$$

where the unknown vector u_Γ represents the trace of the discrete solution on the collection of subdomain interfaces. In order to introduce a more general interface framework, it is useful to first rigorously define the space in which these assembled interface unknowns are defined.

Let the computational domain Ω be decomposed into non-overlapping subdomains Ω_i . This decomposition can be written as

$$\overline{\Omega} = \bigcup_{i=1}^N \overline{\Omega}_i, \quad \Omega_i \cap \Omega_j = \emptyset \quad \text{for } i \neq j.$$

Recall that each subdomain boundary $\partial\Omega_i$ consists of a portion lying on the global physical boundary and a portion shared with neighboring subdomains. The union of all interior subdomain boundaries

forms the global interface, or skeleton, denoted by Γ . More precisely,

$$\Gamma = \bigcup_{i=1}^N \Gamma_i, \quad \Gamma_i = \partial\Omega_i \cap \Gamma.$$

This set contains exactly those edges (in two dimensions) across which the subdomains are coupled, and it provides the geometric support of all interface DOFs.

In the classical assembled finite element formulation, the DOFs associated with nodes on Γ are treated as global unknowns. That is, each physical interface node corresponds to a single DOF, shared by all subdomains that meet at that node. The resulting interface vector u_Γ is therefore single-valued across subdomain boundaries, and the continuity of the discrete solution is implicitly enforced by the standard assembly procedure. From this viewpoint, the interface unknowns belong to an assembled interface space, which may be denoted by $\widehat{W}$, consisting of discrete traces on Γ that take the same value when approached from any adjacent subdomain [11]. This space may be expressed as

$$\widehat{W} = \{ w|_\Gamma : w \in V_h, w \text{ is continuous across } \Gamma \}.$$

Equivalently, interface traces in $\widehat{W}$ satisfy the matching condition

$$w^{(i)}(x) = w^{(j)}(x), \quad x \in \Gamma_i \cap \Gamma_j.$$

for all neighboring subdomains Ω_i and Ω_j .

This assembled interface space provides the natural setting for the reduced Schur complement problem derived in section 4.2. The global interface operator S is obtained by assembling the local Schur complement contributions of all subdomains, that is,

$$S = \sum_{i=1}^N R_i^T S^{(i)} R_i,$$

where $S^{(i)}$ denotes the local Schur complement in subdomain Ω_i and R_i is the restriction operator that maps global interface DOFs to the local interface DOFs on Γ_i (for more detail see later sections). The coupling between subdomains is already incorporated through the identification of shared interface DOFs. Thus, the assembled formulation yields an exact and compact representation of the interface problem, in which continuity across Γ is built into the definition of the unknown space itself.

However, while the assembled interface space is the most direct continuation of the standard finite element viewpoint, it does not explicitly separate the local subdomain contributions from the global continuity requirements. In particular, the mechanism by which neighboring subdomains are coupled remains implicit in the assembled indexing of interface DOFs. For the development of modern non-overlapping DDM, it is advantageous to introduce a reformulation in which interface

unknowns are represented independently on each subdomain, and continuity is imposed as a separate condition. This motivates the introduction of the product, or duplicated, interface space in the next section, where the interface coupling will be expressed explicitly through suitable constraints.

4.3.2 Product (duplicated) interface space

The assembled interface space introduced in the previous section provides the natural setting for the reduced Schur complement formulation, since the interface DOFs are already globally identified and continuity across subdomain boundaries is built into the definition of the unknowns. For the development of modern non-overlapping substructuring methods, however, it is advantageous to introduce an alternative representation in which interface unknowns are described locally on each subdomain and continuity is imposed as an explicit condition. This leads to the concept of a product (or duplicated) interface space.

For each subdomain Ω_i , let Γ_i denote its interface boundary, that is, the portion of $\partial\Omega_i$ shared with neighboring subdomains. Thus,

$$\Gamma_i = \partial\Omega_i \cap \Gamma.$$

The local interface space W_i is defined as the discrete trace space on Γ_i , consisting of all DOFs associated with basis functions supported on Ω_i whose nodes lie on the interface. Thus, an element of W_i represents the interface values of a discrete function as seen from the subdomain Ω_i , expressed in the local subdomain indexing. This local trace space may be written as

$$W_i = \{ w|_{\Gamma_i} : w \in V_h(\Omega_i) \}.$$

The global product interface space is then defined as the Cartesian product of these local trace spaces over all subdomains. More precisely, the product (duplicated) interface space [1] is defined by

$$W := \prod_{i=1}^N W_i = \{ w = (w^{(1)}, \dots, w^{(N)}) \mid w^{(i)} \in W_i \text{ for } i = 1, \dots, N \}.$$

An element $w \in W$ therefore consists of a collection (stack) of local interface vectors, one for each subdomain,

$$w = \begin{pmatrix} w^{(1)} \\ \vdots \\ w^{(N)} \end{pmatrix}, \quad w^{(i)} \in W_i.$$

The dimension of the product space is given by the sum of the numbers of local interface DOFs on all subdomains, that is,

$$\dim(W) = \sum_{i=1}^N \dim(W_i).$$

The essential feature of this construction is that interface DOFs are duplicated. A physical node lying on the interface between two neighboring subdomains gives rise to two distinct local DOFs, one in each of the associated spaces W_i and W_j . These local copies are treated as independent unknowns in the product space, and no continuity requirement is imposed automatically. As a result, the product interface space is larger than the assembled interface space, since it contains interface vectors that may be discontinuous across subdomain boundaries. In particular,

$$\dim(W) > \dim(\widehat{W}).$$

This reformulation provides a clear separation between locality and coupling. In the product space, each subdomain carries its own interface unknowns and its own local Schur complement operator, and these objects remain completely independent. This locality is reflected in the block-diagonal operator

$$S = \text{diag}(S^{(1)}, \dots, S^{(N)}),$$

acting on the product space W . The interaction between neighboring subdomains is no longer built into the assembled indexing but must be introduced explicitly through additional compatibility conditions. This structural viewpoint is particularly convenient for DDM, since it naturally leads to a range of possible strategies for enforcing continuity across the interface.

The assembled interface space $\widehat{W}$ can be viewed as a subspace of the product space consisting of those interface vectors for which all duplicated DOFs agree across the subdomain boundaries. In other words, the globally continuous traces correspond precisely to those elements of the product space whose local components match on shared interface nodes. This matching property may be expressed as

$$\widehat{W} = \{ w \in W : w^{(i)}(x) = w^{(j)}(x) \text{ for all } x \in \Gamma_i \cap \Gamma_j \}.$$

This observation motivates the introduction of explicit continuity constraints, which will be formulated in the next section through a suitable jump operator.

Finally, it is worth noting that the product-space representation has important computational consequences. Since local interface unknowns and local Schur complement operators remain associated with individual subdomains, many operations can be carried out independently and in parallel. The global coupling is reduced to enforcing compatibility between the duplicated interface variables, which provides a natural basis for scalable large-scale solvers.

In the following section, we therefore introduce the continuity constraints that characterize the assembled interface space as a constrained subspace of the product space and make the interface coupling mechanism fully explicit.

4.3.3 Continuity constraints and jump operator B

The product interface space $W = \prod_{i=1}^N W_i$ introduced in the previous section provides a representation in which each subdomain carries its own local copy of the interface DOFs. In this setting, interface unknowns are no longer globally identified and discontinuities across subdomain boundaries are permitted. In order to recover the globally conforming finite element traces, we must therefore impose continuity across the interface explicitly through suitable linear constraints.

Let $x \in \Gamma$ be an interface node shared by two neighboring subdomains Ω_i and Ω_j . For an element $w \in W$, the node x gives rise to two local values, one associated with each subdomain, that is,

$$w^{(i)}(x), \quad w^{(j)}(x).$$

Global continuity requires that these duplicated values coincide, that is,

$$w^{(i)}(x) = w^{(j)}(x).$$

At nodes where more than two subdomains meet, we need several such equalities to ensure that all local copies agree, although we impose only a minimal independent set of constraints [1].

To collect all continuity requirements into a single algebraic relation, we introduce a linear operator

$$B : W \rightarrow \mathbb{R}^m,$$

called the jump operator, where m denotes the total number of independent scalar constraints used to enforce the equality of duplicated interface DOFs [9]. Each row of B represents one such scalar matching condition. More explicitly, let's write an element of the product space as a stacked vector

$$w = \begin{pmatrix} w^{(1)} \\ \vdots \\ w^{(N)} \end{pmatrix}, \quad w^{(i)} \in W_i,$$

where each component of this vector corresponds to one local copy of an interface DOF. Suppose that a physical interface node x is shared by two subdomains Ω_i and Ω_j . In the product space, this single physical DOF is represented by two independent local values, denoted by $w^{(i)}(x)$ and $w^{(j)}(x)$. Continuity requires

$$w^{(i)}(x) = w^{(j)}(x),$$

or equivalently

$$w^{(i)}(x) - w^{(j)}(x) = 0.$$

The corresponding row of B therefore contains exactly two nonzero entries: one entry equal to $+1$ in the column corresponding to the local copy $w^{(i)}(x)$, and one entry equal to -1 in the column

corresponding to the local copy $w^{(j)}(x)$. All other entries in this row are zero.

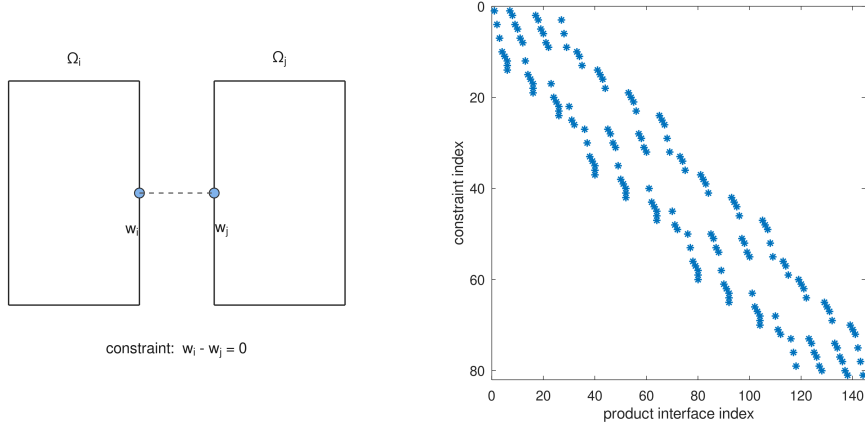


Figure 4.3: Schematic meaning and sparsity pattern of the jump operator B . The product interface space contains local copies of shared interface degrees of freedom, and each row of B enforces equality of two such copies by one $+1$ and one -1 entry.

This construction is illustrated in Figure 4.3. The left part shows one pair of duplicated interface values belonging to two neighboring subdomains and the corresponding constraint $w_i - w_j = 0$. The right part shows the sparsity pattern of the jump operator constructed in the implementation. The two nonzero entries in each row correspond exactly to the two local copies whose equality is being enforced.

If a physical interface DOF is shared by more than two subdomains, say by $\Omega_{i_1}, \dots, \Omega_{i_q}$, then only a minimal independent set of constraints is imposed. For example, one may choose one local copy as a reference and impose

$$w^{(i_1)}(x) - w^{(i_2)}(x) = 0, \quad w^{(i_1)}(x) - w^{(i_3)}(x) = 0, \quad \dots, \quad w^{(i_1)}(x) - w^{(i_q)}(x) = 0.$$

Thus, for q local copies of the same physical DOF, one typically needs $q - 1$ independent scalar constraints. The total number m of rows of B is the sum of these independent constraints over all duplicated interface DOFs.

In this way, the condition

$$Bw = 0$$

expresses precisely the requirement that all duplicated interface values match across the subdomain boundaries. The operator B thus depends only on the decomposition topology and the interface connectivity pattern of the subdomains and on the mapping between local and global interface DOFs, and it is therefore independent of the particular differential operator being discretized (i.e. jump operator B construction uses only the information which local interface DOFs correspond to

the same physical node and it does not use the values of the stiffness matrices, the right-hand side, or the coefficients of the differential operator). Consequently, the same jump operator would be obtained for any differential equation discretized on the same mesh and with the same subdomain decomposition. [1].

Using this operator, we can now characterize the assembled interface space as the subspace of the product space consisting of those elements that satisfy all continuity constraints. In particular, we get

$$\widehat{W} = \{ w \in W \mid Bw = 0 \}.$$

Thus, when we move from the assembled interface space $\widehat{W}$ to the product space W , we separate locality from coupling: local interface unknowns remain independent on each subdomain, while the global conformity requirement is represented entirely through the constraint operator B [8].

From an algebraic viewpoint, an element of the product space can be written as a stacked vector

$$w = \begin{pmatrix} w^{(1)} \\ \vdots \\ w^{(N)} \end{pmatrix}, \quad w^{(i)} \in W_i,$$

so that

$$\dim(W) = \sum_{i=1}^N \dim(W_i).$$

Since the interface nodes are duplicated, the dimension of W is strictly larger than the dimension of the assembled space $\widehat{W}$. The role of the jump operator is to identify those components of w that correspond to the same physical interface DOF. In matrix form, B is sparse and typically contains only entries ± 1 , reflecting differences between neighboring local copies.

It is also useful to relate the constraint formulation back to the classical assembled viewpoint. In the assembled interface space $\widehat{W}$, each physical interface node corresponds to a single global DOF. If we take an assembled interface vector $\widehat{w} \in \widehat{W}$, we can represent it in the product space simply by distributing its values to the corresponding local copies on each subdomain. This distribution is described by an assembly mapping

$$R : \widehat{W} \rightarrow W.$$

Applying R to $\widehat{w}$ produces a product vector $R\widehat{w} \in W$ whose components conform across all interfaces, since they originate from the same single-valued trace. Therefore, every vector in the range of R automatically satisfies the continuity constraints, and we have

$$\text{range}(R) \subseteq \ker(B).$$

Conversely, if a product vector $w \in W$ satisfies $Bw = 0$, then all duplicated interface DOFs match across neighboring subdomains. In this case, the local components define a unique single-

valued interface trace, and we can identify an assembled vector $\hat{w} \in \widehat{W}$ such that $w = R\hat{w}$. This shows that every continuous product vector arises from assembly, and we obtain the structural relation

$$\text{range}(R) = \ker(B).$$

This equivalence expresses that the classical assembled formulation and the constraint formulation describe the same globally continuous interface space, but from two complementary perspectives. It also provides the abstract bridge between primal and dual substructuring approaches: methods such as BDDC emphasize the assembled (primal) viewpoint, whereas methods such as FETI-DP emphasize the constraint (dual) viewpoint.

The formulation of continuity through the constraint equation $Bw = 0$ thus provides a solver-independent mechanism for coupling subdomains in the product interface setting. Modern non-overlapping substructuring methods differ mainly in the way in which these constraints are enforced.

4.4 Primal DOFs and coarse continuity enforcement

4.4.1 Introduction of primal (coarse) interface variables

The product-space formulation of section 4.3 separates locality from continuity enforcement, but this by itself is not sufficient for scalable algorithms. In particular, a formulation based only on duplicated interface unknowns and jump constraints lacks an effective mechanism for the transmission of global interface modes. This motivates the introduction of primal DOFs and a corresponding coarse continuity layer [11].

The complete decoupling is essential, since it allows the reduced interface problem to be expressed in terms of local operators. In particular, after interior elimination, the interface operator takes the block-diagonal form

$$S = \text{diag}(S^{(1)}, \dots, S^{(N)}),$$

where each local Schur complement $S^{(i)}$ acts only on the interface DOFs of a single subdomain.

However, a complete duplication of all interface DOFs also reveals an important limitation of such fully decoupled representation. If all coupling between subdomains is carried only through the constraint equation $Bw = 0$, then the interface problem contains no DOFs that are shared globally by construction. Subdomains communicate only indirectly, through local matching conditions imposed across neighboring interfaces [12].

As a consequence, global information propagates across the overall domain only through successive local interactions of neighboring subdomains. In particular, smooth global components of the interface solution, corresponding to low-frequency modes, cannot be represented or transmitted efficiently by purely local subdomain operators. While local Schur complement solves control oscillatory, high-frequency components effectively, the reduction of global, slowly varying modes requires

a mechanism of global coupling. Without such a mechanism, the decomposed interface formulation lacks a small global backbone that would connect distant parts of the domain directly.

This structural weakness becomes especially apparent as the number of subdomains increases: constraint-only coupling leads to increasingly weak global connectivity, and the treatment of global modes becomes inefficient. In addition, in the presence of floating subdomains, local Neumann-type problems may possess nontrivial kernels, so additional global conditions are required in order to ensure uniqueness and stability of the interface formulation [11], [12].

These observations indicate that, while duplication of interface unknowns is a powerful tool, a fully decoupled representation is not sufficient nor desirable on its own. Modern dual-primal substructuring methods therefore employ a partial decoupling strategy: most interface DOFs remain duplicated and are coupled weakly through constraints, but a small selected subset is kept globally identified and continuous by construction.

This leads to the introduction of primal DOFs, which form a coarse continuity layer. We therefore split the interface DOFs into two disjoint sets,

$$\Gamma = \Gamma_c \cup \Gamma_\Delta, \quad \Gamma_c \cap \Gamma_\Delta = \emptyset,$$

where Γ_c denotes the primal (coarse) interface nodes, and Γ_Δ denotes the remaining interface nodes. Continuity on Γ_c is enforced strongly through global identification, while continuity on Γ_Δ remains imposed through constraint relations [1].

These observations motivate the introduction of a coarse continuity layer through selected primal interface DOFs. We now formalize this construction.

4.4.2 Primal DOFs and coarse interface space

Let Γ denote the global interface of the decomposition, and recall that the product interface space is given by

$$W = \prod_{i=1}^N W_i,$$

so that an element $w \in W$ consists of local interface traces,

$$w = \begin{pmatrix} w^{(1)} \\ \vdots \\ w^{(N)} \end{pmatrix}, \quad w^{(i)} \in W_i.$$

In this product-space representation, a physical interface DOF is represented by separate local copies on all subdomains sharing the corresponding interface node. If all interface DOFs were treated in this way, continuity would have to be enforced only through constraint relations.

Motivated by the limitations of such a fully duplicated representation discussed in the previous section, we select a subset

$$\Gamma_c \subset \Gamma$$

of interface DOFs that are kept globally identified and continuous by construction. These DOFs are called primal, or coarse, DOFs. The remaining interface DOFs are collected in the complementary set

$$\Gamma_\Delta = \Gamma \setminus \Gamma_c.$$

Accordingly, the interface DOFs are split into two disjoint sets,

$$\Gamma = \Gamma_c \cup \Gamma_\Delta, \quad \Gamma_c \cap \Gamma_\Delta = \emptyset.$$

The defining property of primal variables is that their continuity is enforced strongly by global identification. Thus, if a primal node $x \in \Gamma_c$ is shared by subdomains Ω_i and Ω_j , then the corresponding value is represented by a single global unknown, and the local traces satisfy

$$w_c^{(i)}(x) = w_c^{(j)}(x).$$

In other words, duplication is restricted to the non-primal part Γ_Δ , whereas the DOFs in Γ_c remain assembled and single-valued by construction. Continuity on Γ_Δ is still imposed through constraint relations, while continuity on Γ_c is built directly into the choice of unknowns [10].

We denote by $W_c \subset W$ the subspace associated with the primal DOFs on Γ_c , and by $W_\Delta \subset W$ the subspace associated with the remaining duplicated interface variables on Γ_Δ [1]. This induces direct decomposition

$$W = W_c \oplus W_\Delta.$$

It is important to distinguish between the product-space representation of primal variables and their assembled counterpart. The space W_c consists of the primal components written in subdomain-local coordinates, that is, as collections of local interface vectors on each Ω_i . In contrast, the assembled coarse space is denoted by $\widehat{W}_c$ and consists of globally identified primal traces, where each primal node corresponds to a single DOF shared across all neighboring subdomains. Thus, $\widehat{W}_c$ is the single-valued coarse interface space, whereas W_c represents the same variables in the product (subdomain-wise) setting. The assembled coarse space $\widehat{W}_c$ is defined in direct analogy with the assembled interface space $\widehat{W} \subset W$ introduced in section 4.3.1; however, it involves only the primal DOFs and therefore has a significantly smaller dimension [8].

In the model setting considered in this thesis, where Ω is a two-dimensional square domain decomposed into identical square subdomains, a natural and simple choice of primal variables is given by the corner nodes of the decomposition. These are the interface nodes at which several subdomains meet. Enforcing continuity at such crosspoints yields an effective pointwise coupling

across the decomposition and ensures that the subdomain partition remains globally connected.

This choice, however, is not the only admissible one. In dual-primal methods, the primal variables may also be chosen as suitable averages or moments over subdomain interfaces. In two-dimensional problems, common alternatives include averages over subdomain edges, or a combination of corner values and edge averages. In three-dimensional problems, one may additionally use averages over faces, and for elasticity problems the primal constraints must be chosen so that the rigid body modes of floating subdomains are controlled. Enriching the primal set usually improves the robustness of the preconditioned iteration, but it also increases the size and cost of the coarse problem. Thus, the choice of primal variables is a compromise between a small coarse space and strong global coupling.

The remaining interface DOFs, associated with edge nodes, remain duplicated in the product space, and their continuity is imposed weakly through constraint relations.

This splitting of interface unknowns into primal and remaining parts induces a corresponding block structure of the local Schur complement operators, which will be exploited in the formulation of FETI-DP and BDDC in later sections.

Thus, the selection of primal DOFs defines the coarse interface space that underlies dual-primal substructuring methods. Both FETI-DP and BDDC are based on the same coarse continuity mechanism, while differing only in the treatment of the remaining interface constraints.

4.4.3 Interface space decomposition and reduced continuity constraints

The introduction of primal DOFs leads to a structural modification of the interface formulation. Since primal variables are globally identified and continuous by construction, continuity constraints are no longer imposed on the entire product interface space but only on the remaining duplicated DOFs.

Recall that the product interface space admits the decomposition

$$W = W_c \oplus W_\Delta,$$

where W_c corresponds to the primal variables and W_Δ contains the remaining interface unknowns. The primal component is assembled into the single-valued coarse space $\widehat{W}_c$, whereas the DOFs in W_Δ remain duplicated across subdomains.

As a consequence, the continuity constraints expressed by the jump operator reduce to the remaining part Δ . Thus, the global condition

$$Bw = 0$$

will change into reduced form;

$$B_\Delta w_\Delta = 0,$$

where B_Δ denotes the restriction of the jump operator to the interface DOFs on Γ_Δ . Continuity is therefore enforced strongly on Γ_c through assembly and only weakly on Γ_Δ through constraint relations.

The assembled interface space can therefore be characterized as the space of all interface vectors whose primal component belongs to $\widehat{W}_c$ and whose remaining component satisfies the reduced constraints. In this sense, the interface framework combines a coarse globally continuous layer with a constrained product-space representation of the remaining interface unknowns.

This decomposition provides the final structural setting common to modern dual-primal substructuring methods. The primal variables define the coarse coupling mechanism, while the remaining constraints determine the dual interface interaction. The subsequent distinction between FETI-DP and BDDC lies primarily in the way the reduced constraint system on W_Δ is enforced, which will be addressed in the following sections.

4.5 Position of FETI-DP and BDDC within the unified interface framework

The purpose of section ?? has been to reformulate the finite element discretization of the model elliptic problem into an interface framework suitable for non-overlapping substructuring methods. Starting from a decomposition of the computational domain into subdomains, we introduced local finite element spaces and the interior/interface splitting of DOFs. By eliminating interior unknowns, the global problem was reduced to an interface formulation expressed in terms of local Schur complement operators acting on the subdomain traces.

In order to separate locality from coupling, we then moved from the assembled interface space $\widehat{W}$ to the product interface space $W = \prod_{i=1}^N W_i$, where interface DOFs are duplicated across subdomains, and continuity is imposed explicitly by the constraint relation $Bw = 0$. This formulation makes the interaction between neighboring subdomains entirely transparent and provides a solver-independent characterization of the assembled space as $\widehat{W} = \ker(B)$.

In view of the limitations of a fully duplicated interface representation discussed in section 4.4.1 in particular the absence of an effective mechanism for the transmission of global modes across the decomposition, we introduced a coarse continuity layer by selecting a subset Γ_c of primal interface DOFs. These variables are globally identified and assembled into the coarse space $\widehat{W}_c$, while the remaining interface unknowns remain duplicated in W_Δ and are coupled through reduced continuity constraints $B_\Delta w_\Delta = 0$. The resulting decomposition combines strong continuity on Γ_c with constraint-based continuity enforcement on the remaining interface.

This abstract interface setting constitutes the common structural foundation of modern dual-primal substructuring methods. In particular, the present thesis is concerned with two representative approaches, FETI-DP and BDDC, which are among the most widely used scalable methods for

large-scale elliptic problems. For this reason, one of the purposes of this section was to build up a common interface framework within which both FETI-DP and BDDC can be naturally formulated, and on which both methods are in fact built.

Both FETI-DP and BDDC are indeed built upon the same ingredients established in this section: the non-overlapping decomposition, the Schur complement reduction to the interface, the product space formulation with explicit continuity constraints, and the introduction of primal DOFs defining a coarse globally continuous subspace. The distinction between the two methods lies primarily in the mechanism by which continuity on the remaining interface DOFs is enforced [11]. In FETI-DP, the reduced constraint system on W_Δ is imposed by means of Lagrange multipliers, leading to a dual formulation. In contrast, BDDC achieves the coupling through primal assembly together with suitable averaging operators acting on the duplicated interface variables [1].

The subsequent sections will specialize this common framework into the concrete formulations of FETI-DP and BDDC and will present the corresponding algorithmic constructions within the common setting developed here.

4.6 Domain decomposition framework - Octave implementation

This section introduces a unified algebraic domain decomposition framework (local subdomain spaces, interior elimination, product interface space, continuity constraints, and a primal- Δ split). The Octave implementation follows this structure closely: the code is organized as a short build pipeline that constructs the discrete objects from sections 4.1 – 4.4 and exposes them through small, composable operator-application routines.

4.6.1 Implementation principles

The implementation is written in a modular, “data + operators” style. All bookkeeping is stored in plain Octave `struct` objects and arrays, and computational kernels are implemented as functions that either (i) enrich these structures during setup or (ii) apply the relevant operators to vectors. Sparse matrices are used for global maps on the interface (assembly/distribution and constraints), while the dominant subdomain computations are kept local. The Schur complement reduction is implemented in a matrix-free manner: local interior blocks are factorized once and reused to apply local Schur complements without forming dense global operators.

4.6.2 Data structures

The main theory objects are represented by the following structures:

- `ddm`: global decomposition bookkeeping (numbers of subdomains, mapping between global mesh nodes and free DOFs after Dirichlet elimination, and basic mesh sizes).

- **sub**: an array of subdomain structs. Each entry stores element membership, local free-DOF sets and maps, the interior/interface split, local stiffness/load contributions, and the data needed for local Schur-complement actions.
- **iface**: global interface bookkeeping corresponding to the assembled interface space $\widehat{W}$ (list of shared global free DOFs and mappings between global free-DOF indices and assembled “hat” indices).
- **prod**: product-space bookkeeping for $W = \prod_i W_i$ (product indexing of duplicated interface DOFs, and the maps between product indices and assembled hat indices).
- **primal**: description of the primal set Γ_c and the induced split $W = W_c \oplus W_\Delta$ (indices and maps needed to identify primal and Δ components consistently across $\widehat{W}$ and W).

4.6.3 Setup pipeline and code map

The implementation is distributed in the directory `ddm/` and is grouped into four submodules whose roles mirror the section structure:

- **Partitioning (`ddm/partition`)**. The routine `build_subdomains_structured.m` partitions the unit-square triangulation into a rectangular grid of subdomains and constructs, for each subdomain, its element set and the corresponding local set of free DOFs (consistent with the reduced FEM system after Dirichlet elimination from section 3).

The routine `identify_interface_dofs.m` then classifies free DOFs into interior and interface DOFs by multiplicity across subdomains and constructs the assembled interface bookkeeping `iface`, including the mapping between global free DOFs and the assembled interface indexing of $\widehat{W}$.

- **Local subdomain operators (`ddm/local`)**. Local stiffness matrices and load vectors restricted to each subdomain are assembled by `assemble_subdomain_matrices_P1.m` using the same P_1 element technology as in Chapter 3, but applied on subdomain element sets and local-to-global free-DOF maps. The block form associated with the interior/interface split,

$$K^{(i)} = \begin{pmatrix} K_{II}^{(i)} & K_{I\Gamma}^{(i)} \\ K_{\Gamma I}^{(i)} & K_{\Gamma\Gamma}^{(i)} \end{pmatrix},$$

is extracted by `extract_subdomain_blocks.m`.

Interior elimination data for the Schur reduction are prepared by `setup_local_schur.m` (factorization of $K_{II}^{(i)}$ and reduced right-hand side), and the matrix-free action of a local Schur complement is provided by `apply_local_schur.m`.

- **Product interface space and coupling operators (ddm/interface).** The product interface indexing for $W = \prod_i W_i$ is constructed by `build_product_interface.m`. This routine assigns a unique product index to every local interface DOF on each subdomain and stores the resulting local product segments back into `sub`. Based on the product bookkeeping, `build_assembly_operator_R.m` constructs the sparse operator $R : \widehat{W} \rightarrow W$ connecting the assembled and product interface spaces. In the forward direction, R acts as a distribution operator: it takes one single-valued assembled interface vector $\widehat{w} \in \widehat{W}$ and copies each of its values to all corresponding local interface copies in the product space W . Thus, if a physical interface DOF is shared by several subdomains, the corresponding column of R contains ones in all product-space rows associated with these local copies. The transpose R^T performs the opposite operation: it gathers product-space quantities and sums all contributions belonging to the same assembled interface DOF. For this reason, the implementation name `build_assembly_operator_R.m` refers to the assembly/distribution pair represented by R and R^T , although the matrix R itself acts as the distribution operator. Continuity in the product space is represented by the sparse jump/constraint operator `build_jump_operator_B.m`, which encodes the equalities between multiple copies of the same physical interface DOF. Finally, `apply_blockdiag_S.m` applies the block-diagonal product-space operator $S = \text{diag}(S^{(1)}, \dots, S^{(N)})$ by calling the local Schur action on each subdomain block.
- **Primal set and the primal- Δ split (ddm/coarse).** The primal DOFs Γ_c are selected by `select_primal_dofs.m`. In the model setting of this thesis (structured rectangular decomposition), primal DOFs are chosen as the corner/crosspoint nodes of the subdomain partition. The routine `build_primal_maps.m` then constructs consistent maps for the decomposition $W = W_c \oplus W_\Delta$ across assembled and product representations (including masks and index maps that identify Δ -unknowns in the product space). These maps form the interface “infrastructure” that is reused later by the solvers in Chapter 5.

4.6.4 Traceability

To keep the implementation aligned with the structure, terminology and formulations of actual thesis, each core routine contains a short header comment that points to the corresponding section(s) and equation(s) in this section. This provides a direct link between the mathematical objects ($\widehat{W}$, W , R , B , Schur complements, and the primal- Δ split) and their concrete Octave representation and usage in the build pipeline above.

Chapter 5

Dual–primal domain decomposition solvers: FETI-DP and BDDC

The purpose of Chapter 5 is to turn this abstract interface framework into concrete solvers implemented in Octave, focusing on the two representative dual–primal methods FETI-DP and BDDC. The emphasis is not on introducing new discretization principles, but on describing how the same reduced interface objects from Chapter 4 (local Schur complements, product-space traces, primal constraints, and reduced continuity on W_Δ) are converted into practical iterative algorithms and reusable implementation tools. In particular, this Chapter explains which global unknown is iterated on in each method, how continuity on W_Δ is enforced (by multipliers in FETI-DP versus primal averaging/scaling in BDDC), and how interface and interior values are reconstructed from the iterates by local subdomain solves.

Chapter is organized as follows. section 5.1 summarizes the common principles of FETI-DP and BDDC and isolates their key conceptual difference within the unified notation of section 4; it also states the minimal symmetry and positive definiteness requirements that justify the use of the Preconditioned Conjugate Gradient (PCG) method in the later implementation. section 5.2 introduces the shared solver infrastructure needed by both methods in Octave, including the common PCG wrapper, the matrix-free application of interface operators based on local subdomain solves, and the treatment of the primal/coarse component induced by Γ_c . sections 5.3 and 5.4 then present the method-specific formulations and implementations: first the FETI-DP solver in a multiplier (dual) formulation and then the BDDC solver in an assembled/primal interface formulation, each expressed consistently in the same interface notation and implemented using the common tools developed earlier in the section.

5.1 Common principles and key differences of FETI-DP and BDDC

We begin by summarizing the interface objects that are common to both FETI-DP and BDDC and by stating the main structural difference between the two methods. The detailed operator derivations are deferred to sections 5.3 and 5.4.

From the structural viewpoint, both FETI-DP and BDDC follow the same solution pipeline implied by the substructuring reduction of Chapter 4. First, interior unknowns are eliminated locally so that the global coupling is expressed purely in terms of interface traces and local Schur complement operators, collected into the block-diagonal operator $S = \text{diag}(S^{(i)})$ acting on the product interface space $W = \prod_{i=1}^N W_i$. Second, global conformity is represented explicitly by the jump constraints $Bw = 0$ (equivalently $\widehat{W} = \ker(B)$), and scalability is ensured by selecting primal DOFs Γ_c , which induces the decomposition $W = W_c \oplus W_\Delta$ and reduces the remaining continuity requirement to $B_\Delta w_\Delta = 0$. Third, the actual iterative solver can be understood as an interface-level iteration coupled with local subdomain solves: at each step, global information is updated on the interface, while interior values are recovered by independent local subdomain solves [11].

With this common framework fixed, the essential distinction between the two methods can be stated in one sentence: FETI-DP enforces the reduced continuity constraints $B_\Delta w_\Delta = 0$ on W_Δ by Lagrange multipliers (a dual enforcement mechanism), whereas BDDC enforces continuity on W_Δ in a primal manner through assembled interface corrections combined by averaging/scaling operators acting on the duplicated interface variables [8], [13].

5.1.1 Reused interface framework from Chapter 4

We use the notation of Chapter 4 without further modification. In particular, after interior elimination each subdomain Ω_i contributes a local Schur complement operator $S^{(i)}$, and these local operators form the block-diagonal product-space operator

$$S = \text{diag}(S^{(1)}, \dots, S^{(N)})$$

acting on

$$W = \prod_{i=1}^N W_i,$$

where W_i denotes the local trace space on Γ_i . In this representation, a vector $w \in W$ is a stacked collection of local interface vectors

$$w = \begin{pmatrix} w^{(1)} \\ \vdots \\ w^{(N)} \end{pmatrix}, \quad w^{(i)} \in W_i.$$

and the locality of the subdomain contributions is explicit: applying S to w reduces to independent actions of $S^{(i)}$ on each $w^{(i)}$.

The price of this locality is that the product space W contains, by construction, multiple copies of the same physical interface DOF. Consequently, global continuity is no longer implicit (as it is in the classical assembled interface space) but must be expressed as an explicit compatibility requirement between neighboring subdomains. This is captured by the jump operator

$$B : W \rightarrow \mathbb{R}^m,$$

whose rows encode differences of duplicated interface values on shared nodes/edges, so that the continuity requirement is expressed compactly as

$$Bw = 0.$$

The assembled interface space $\widehat{W}$ is therefore characterized as the constrained subspace $\widehat{W} = \ker(B)$. To relate the assembled and product viewpoints, Chapter 4 also introduced the assembly mapping $R : \widehat{W} \rightarrow W$, which distributes a single-valued interface vector $\hat{w} \in \widehat{W}$ into its consistent local copies on each subdomain. This mapping satisfies the structural identity $\text{range}(R) = \ker(B)$, i.e., product vectors are continuous exactly when they arise by assembly from $\widehat{W}$.

A second structural ingredient, common to both FETI-DP and BDDC, is the introduction of primal (coarse) interface DOFs $\Gamma_c \subset \Gamma$. These are selected interface nodes (in the model setting of this thesis represented by the crosspoints/corners of the decomposition) that are kept globally identified and continuous by construction. The remaining interface DOFs are collected in $\Gamma_\Delta = \Gamma \setminus \Gamma_c$. On the level of interface spaces, this selection induces the decomposition

$$W = W_c \oplus W_\Delta,$$

where W_c corresponds to primal variables and W_Δ contains the remaining duplicated interface variables. Since primal variables are assembled, continuity constraints are only needed for the remaining part, which leads to the reduced constraint system

$$B_\Delta w_\Delta = 0,$$

where B_Δ is the restriction of the jump operator to the DOFs in W_Δ . This split is the point at which the “coarse/primal layer” enters both methods: both rely on the same selection Γ_c and the same strong continuity enforcement on W_c , while differing only in the mechanism by which the reduced constraints on W_Δ are enforced.

Finally, it is useful to state explicitly why this Chapter 4 framework is an appropriate starting point for the solver formulations in the rest of this section. In the product-space representation,

the operator S is block diagonal, hence purely local, and the only global coupling is represented by the continuity condition (in reduced form) $B_\Delta w_\Delta = 0$. This separation makes it natural to view the interface solution process as the interaction of two components: a local component given by applications of $S^{(i)}$ (or their inverses in the sense of local subdomain solves) and a global coupling component that enforces the interface compatibility. In later sections, FETI-DP will realize this coupling by treating $B_\Delta w_\Delta = 0$ as an explicit constraint enforced by Lagrange multipliers, while BDDC will realize it by producing assembled interface corrections through averaging/scaling of product-space quantities. At this stage, the key structural point is that both approaches start from the same decoupled local operators on W and from the same reduced continuity requirement on W_Δ ; only the chosen enforcement mechanism differs.

5.1.2 Dual vs primal enforcement on W_Δ

The split $W = W_c \oplus W_\Delta$ introduced in Chapter 4 isolates the part of the interface where continuity is enforced strongly by construction (the primal component on Γ_c) from the remaining duplicated interface DOFs on Γ_Δ . In this section, the terms “primal” and “dual” are used in a precise and limited sense: “primal” refers to enforcing continuity by working with single-valued (assembled) interface unknowns, whereas “dual” refers to enforcing continuity constraints by introducing Lagrange multipliers. Importantly, the set Γ_Δ itself is not “dual”; it is simply the subset where continuity is not enforced by global identification, and it can be handled either dually (FETI-DP) or primally (BDDC). Both methods share the same choice of Γ_c and the same decomposition $W = W_c \oplus W_\Delta$; the distinction arises only in the mechanism used to impose the reduced continuity requirement $B_\Delta w_\Delta = 0$.

To motivate the dual (multiplier) viewpoint, it is useful to connect the interface formulation to the quadratic minimization principle already used in Chapter 3. There, a linear SPD system was related to the stationarity of a quadratic energy functional. After substructuring reduction, the same idea may be applied on the interface: once interior unknowns are eliminated, the remaining interface problem is governed by Schur complement operators and can be described by a quadratic functional in the interface trace. In the product-space representation, locality is expressed by the block-diagonal operator $S = \text{diag}(S^{(i)})$ acting on W , while continuity across interfaces is expressed separately by the linear constraint $Bw = 0$. This leads naturally to a constrained quadratic minimization problem of the form

$$\min_{w \in W} \frac{1}{2} w^T S w - g^T w \quad \text{subject to} \quad Bw = 0,$$

where g denotes the interface right-hand side obtained after elimination. The constraint is exactly the algebraic statement that all duplicated copies of each physical interface DOF coincide; equivalently, w must lie in the assembled subspace $\widehat{W} = \ker(B)$. Writing the problem in constrained form is convenient because it separates two roles that are central in non-overlapping domain decomposi-

tion: the energy term contains only local subdomain contributions (through the block-diagonal S), while the constraint contains the global coupling.

A standard algebraic way to enforce such linear constraints in a quadratic minimization problem is to introduce Lagrange multipliers. One forms the Lagrangian

$$\mathcal{L}(w, \lambda) = \frac{1}{2} w^T S w - g^T w + \lambda^T (Bw),$$

where λ is a vector of multipliers, one component per continuity constraint (equivalently, per independent jump) [14]. The solution is characterized by the stationarity of $\mathcal{L}$ with respect to both variables, which yields the coupled system

$$Sw - g + B^T \lambda = 0, \quad Bw = 0.$$

Equivalently, this can be written in block form.

$$\begin{bmatrix} S & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} w \\ \lambda \end{bmatrix} = \begin{bmatrix} g \\ 0 \end{bmatrix}.$$

This is a typical saddle-point problem: the block matrix is symmetric but, in general, indefinite, so it is not directly suitable for solvers designed for SPD operators such as PCG. Historically, this is one of the motivations for extending purely dual tearing methods (classical FETI) by a primal/coarse component: the dual-primal construction improves robustness and scalability by treating selected interface DOFs strongly and by separating out global modes. In the present thesis, this is reflected in the choice of Γ_c and the split $W = W_c \oplus W_\Delta$; the resulting SPD operators used in the actual PCG iterations are described later in sections 5.1.3, 5.3 and 5.4.

The second equation enforces continuity. The first equation shows how λ enters the equilibrium of the decoupled local interface problems: the term $B^T \lambda$ acts as an additional interface load that corrects the independently computed subdomain traces so that they satisfy compatibility. This interpretation is also the key to understanding the term “dual.” The interface trace w is a primal (displacement/trace) quantity living in the trace space, while λ is the variable conjugate to the constraint Bw (it pairs with the jump in the Lagrangian). In mechanical terms, λ represents interface reaction forces required to enforce continuity; in a flux/traction interpretation it can be viewed as a discrete interface traction-like quantity whose action, via $B^T \lambda$, balances the local subdomain interface equations so that neighboring traces match. This is the sense in which λ differs from primal interface variables: w describes interface values themselves, while λ describes the “forces” needed to make the independently computed interface values compatible.

In FETI-DP, this constrained viewpoint is applied precisely on the remaining interface part W_Δ . The primal component on W_c is kept single-valued by construction, and the reduced continuity

requirement

$$B_{\Delta}w_{\Delta} = 0$$

is enforced by multipliers associated with the reduced jump operator B_{Δ} . Conceptually, one may view the method as follows: the product-space unknown w_{Δ} is still present as the local trace on each subdomain, but the global coupling is carried by the multiplier λ , and the iterative method is organized so that the main global unknown updated by PCG is (a suitable form of) λ , together with a coarse/primal correction induced by Γ_c . For a given multiplier vector λ , the corresponding interface traces are obtained by local subdomain operations (applications of the local Schur complements and their constrained inverses), and the role of λ is to drive these local traces toward compatibility by contributing the reaction term $B_{\Delta}^T\lambda$. The detailed elimination steps leading to the explicit dual operator and the precise form of the FETI-DP preconditioner are deferred to section 5.3; at this stage, the essential point is that continuity on W_{Δ} is imposed as an explicit constraint and λ is the variable that enforces it.

In BDDC, the same reduced continuity requirement on Γ_{Δ} is achieved without multipliers by working with assembled (single-valued) interface corrections obtained by averaging/scaling of duplicated quantities. The basic mechanism is that each physical interface DOF has several local copies (one per adjacent subdomain), and BDDC constructs a single continuous correction by a weighted average of these local copies. The weights satisfy a partition-of-unity property at each shared interface DOF (the weighted contributions from all neighboring subdomains sum to one), so that the averaged correction is single-valued and therefore belongs to the assembled interface space. In the simplest multiplicity scaling, the weight is proportional to the inverse of the number of subdomains sharing the DOF; in coefficient- or stiffness-based scalings, the weights incorporate local information so that the averaging better reflects heterogeneous parameters. A crucial operational point is that this averaging/scaling is not a one-time preprocessing step: it appears repeatedly as part of the BDDC preconditioner action during the iterative solve. In each application of the preconditioner, local subdomain problems produce product-space (duplicated) interface corrections, and the averaging/scaling operator combines them into one assembled correction that is applied to update the global (single-valued) interface iterate.

To keep the later derivations in sections 5.3 and 5.4 readable, we summarize the iteration logic of both methods in a minimal schematic form. In FETI-DP, the PCG iteration is organized around the multiplier variable λ . In each iteration:

- (i) the action of the dual operator on a multiplier increment is evaluated using local subdomain/interface solves together with constraint operations involving B_{Δ} and B_{Δ}^T ;
- (ii) from the current λ , the corresponding interface trace is reconstructed together with the coarse/primal component on W_c ;
- (iii) once the interface trace is available, interior values on each subdomain are recovered independently by the same local reconstruction as in Chapter 4.

In BDDC, the PCG iteration is performed on an assembled (single-valued) interface quantity. In each iteration:

- (i) local subdomain solves produce interface corrections in the product space, that is, in duplicated form;
- (ii) these local corrections are combined into a single assembled correction by averaging and scaling, using partition-of-unity weights;
- (iii) the assembled correction updates the global interface iterate, and interior values are again recovered by local reconstruction after the interface update.

At this stage, the essential message is that both methods start from the same reduced interface setting of Chapter 4—local Schur complements on the product space and the reduced continuity requirement on W_Δ —but they enforce continuity on W_Δ in two fundamentally different ways. FETI-DP treats $B_\Delta w_\Delta = 0$ as an explicit constraint, introduces multipliers λ , and organizes the iteration around these dual variables, which represent interface reactions enforcing compatibility. BDDC avoids multipliers and enforces continuity by producing single-valued interface corrections through primal assembly based on averaging/scaling [1], [11]. The following sections build directly on this distinction: section 5.3 makes the multiplier (dual) operator and its application explicit, while section 5.4 introduces the assembled/primal operator and the averaging/scaling machinery used in BDDC.

5.1.3 SPD operators and applicability of PCG method

The solvers in the remainder of this section are implemented as iterative Krylov subspace methods. A Krylov method constructs a sequence of approximations by repeatedly applying the system operator to the current residual and by combining these vectors in a low-dimensional search space generated from the initial residual. More precisely, after k steps the iterate is sought in an affine space of the form $x_0 + \mathcal{K}_k(A, r_0)$, where $\mathcal{K}_k(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{k-1}r_0\}$ is the k -th Krylov subspace associated with the operator A and the initial residual $r_0 = b - Ax_0$. The practical advantage of such methods in domain decomposition is that they only require the ability to apply the operator A to a vector; the operator itself need not be assembled explicitly. In the present setting, one application of the interface operator can be realized by a small number of local subdomain solves together with constraint or averaging/scaling operations on interface quantities, which makes Krylov methods a natural choice for the Octave implementation.

In this thesis, we use the PCG method, which is a Krylov method tailored to SPD operators (with respect to a suitable inner product). Its role is to accelerate convergence by combining two complementary ingredients: the Krylov subspace iteration, which captures global error components through repeated operator applications, and a preconditioner, which provides a computationally cheap approximate inverse and reduces the disparity of scales in the problem. In each iteration, the method produces a new search direction that is conjugate with respect to the system operator, and

the next iterate is obtained by minimizing the quadratic energy over the current affine Krylov space; this minimization viewpoint connects naturally to the variational formulation used in Chapter 3. In practical terms, the PCG method is attractive here because (i) it relies on symmetric operations, (ii) it can be implemented in a matrix-free manner, and (iii) the dominant cost per iteration is determined by local subdomain solves that can be reused across both FETI-DP and BDDC.

The reason why SPDness is the decisive property can be stated in the same variational language already used in Chapter 3. Consider a quadratic functional of the form

$$J(x) = \frac{1}{2}x^T A x - b^T x.$$

If A is SPD, then J is strictly convex and therefore has a unique minimizer; the first-order optimality condition $\nabla J(x) = 0$ is equivalent to the linear system $Ax = b$. Conversely, strict convexity of J is equivalent to positive definiteness of the symmetric part of A , and in the symmetric case this reduces exactly to A being SPD. Thus, whenever the reduced interface problem can be formulated as the minimization of a strictly convex quadratic energy functional on an admissible space, the corresponding linear operator is SPD and is a valid target for the PCG method.

Within the interface framework of Chapter 4, the natural energy operator is the block-diagonal Schur complement $S = \text{diag}(S^{(i)})$ acting on the product interface space $W = \prod_i W_i$. The physical interface solution, however, does not live in the full product space: it must satisfy the continuity constraints (or their reduced form) and incorporate the primal/coarse component induced by Γ_c . Once these admissibility conditions are enforced, the interface problem corresponds to minimizing the interface energy of discrete harmonic extensions, and in the present model setting, the minimizer is unique. This is the structural reason why the later formulations are designed so that the actual Krylov iteration is performed on a SPD operator acting on an admissible (assembled or reduced) interface space, rather than on the indefinite saddle-point system that appears before elimination and coarse correction.

For the FETI-DP method, the coupled formulation obtained by introducing Lagrange multipliers is symmetric but indefinite, so it is not used directly as the target of the PCG method. Instead, after enforcing the primal layer and eliminating the remaining local trace variables, the method is reformulated as a reduced problem acting on a Lagrange-multiplier-type variable associated with reduced continuity constraints $B_\Delta w_\Delta = 0$. In this reduced form, the operator used in the Krylov iteration is SPD on the relevant multiplier subspace, while the primal/coarse component induced by Γ_c is incorporated through a coarse correction. The precise definition of this reduced dual operator and the corresponding preconditioner is given in section 5.3; in this section, it is sufficient to know that FETI-DP applies the PCG method to a reduced SPD dual operator, not to the original saddle-point system.

For the BDDC method, the iteration is organized directly on a primal (assembled) interface unknown that is single-valued on the primal set Γ_c and updated by assembled corrections obtained

from local subdomain solves combined by averaging/scaling on Γ_Δ . In this viewpoint, the operator targeted by the PCG method is a SPD operator on an assembled interface space, and the preconditioner is built symmetrically from local constrained corrections, a consistent averaging/scaling assembly, and the same coarse/primal correction. The detailed form of the assembled operator and the preconditioner action is deferred to section 5.4; here the key point is that BDDC stays in a purely primal SPD setting, whereas FETI-DP achieves symmetry and positive definiteness by reducing the formulation to the multiplier space, and in both cases the primal/coarse component induced by Γ_c is the structural ingredient that makes the final iterated operator compatible with the PCG method.

Finally, this section also clarifies the division of responsibilities inside this section. The present comparison section only states which SPD operator is targeted by the PCG method in each approach and why the primal/coarse component is necessary for well-posedness. The explicit definitions of the reduced dual operator and preconditioner (FETI-DP), and of the assembled operator and averaging/scaling preconditioner (BDDC), together with their matrix-free Octave realization, are presented in section 5.3 and 5.4, while section 5.2 collects and explain the common implementation tools shared by both methods.

5.2 Common solver infrastructure and implementation tools

The goal of this section is to collect the minimal solver infrastructure that is shared by both FETI-DP and BDDC in the Octave implementation. Even though the two methods iterate on different global unknowns (multipliers for FETI-DP versus an assembled interface vector for BDDC), both solvers are implemented in the same matrix-free spirit: one Krylov iteration consists of a small number of local subdomain operations (based on the Schur complements from Chapter 4), combined with sparse interface mappings and a small coarse/primal solve. For this reason, the shared implementation layer is organized around (i) a thin wrapper of Octave’s built-in `pcg` routine in function-handle form and (ii) a small set of reusable helper routines (problem builder, scaling weights, and deterministic RNG).

5.2.1 PCG requirements and stopping criteria

In the remainder of Chapter 5 we use the PCG method (PCG). PCG is applicable when the iterated operator is SPD on the space in which the iteration is performed and when the preconditioner defines a SPD action with respect to the same inner product. As explained in section 5.1.3, both FETI-DP and BDDC are formulated so that the Krylov iteration targets a SPD operator on the appropriate reduced space.

In the implementation, PCG is called with a user-prescribed relative tolerance `tol` and a maximum iteration count `maxit`. For every run we record the standard outputs returned by Octave: the

convergence flag, the achieved relative residual norm, the number of performed iterations, and the residual history. Concretely, the wrapper returns a structure

```
stats.flag, stats.relres, stats.iter, stats.resvec.
```

If x_j denotes the approximation after the j -th Krylov iteration and $r_j = b - Ax_j$ denotes the corresponding residual, then `resvec(1)` contains $\|r_0\|_2$, while `resvec(j+1)` contains $\|r_j\|_2$. The value `stats.relres` is the final residual norm relative to the initial residual norm, i.e.

$$\text{stats.relres} = \frac{\|r_{\text{final}}\|_2}{\|r_0\|_2}.$$

5.2.2 Matrix-free operators and PCG wrapper

A key practical advantage of Krylov methods in domain decomposition is that they only require the ability to apply the operator to a vector. In the present setting, one application of an interface operator is realized by local subdomain operations (Schur complement applications and local reconstructions) together with sparse interface-level mappings and constraints. Therefore, the operators used in sections 5.3–5.4 are applied in a matrix-free manner and are not assembled as global matrices.

To enforce a uniform calling convention, we wrap Octave’s `pcg` into a thin helper routine `pcg_wrap.m` (directory `common/krylov/`). The wrapper is intentionally minimal and uses the interface

```
[x, stats] = pcg_wrap(applyA, b, tol, maxit, applyM, x0),
```

where `applyA` is a function handle evaluating $y = Ax$ and `applyM` is an optional function handle evaluating $z = M^{-1}r$. If `applyM` is empty (or omitted), PCG is run without preconditioning. The initial guess x_0 is optional; if not provided, the wrapper uses the zero vector. The wrapper produces no printing and returns only the solution and the structure `stats` described above.

5.2.3 Primal/coarse correction via Γ_c and $W = W_c \oplus W_\Delta$

The interface framework of Chapter 4 introduces the primal set Γ_c and the decomposition $W = W_c \oplus W_\Delta$, where W_c is assembled (single-valued) by construction and W_Δ remains duplicated and is coupled only through the reduced constraints. This split is exactly the point at which the coarse/primal layer enters both methods. In the implementation, the primal selection and all related index maps are constructed once (in the Chapter 4 build pipeline) and then reused by both solvers.

From the viewpoint of the common infrastructure, the coarse/primal correction is treated as a small auxiliary solve that is prepared in the solver-specific setup phase and then reused inside the matrix-free operator/preconditioner applications. In order to keep a stable function signature at the shared layer, the file `common/coarse/apply_coarse_hook.m` provides a placeholder routine

with the interface

$$z_c = \text{apply_coarse_hook}(\text{data}, r),$$

which returns a zero vector. The actual coarse actions are implemented in the solver-specific modules and are defined in the later method-specific sections:

- in FETI-DP, the coarse solve is based on the coarse matrix denoted later by K_{cc}^* , defined in section 5.3.3. In the implementation, this matrix is stored under the field name `data.Kcc`, and its Cholesky factor is used inside `solve_tildeS.m` and in the reconstruction step;
- in BDDC, the coarse solve is based on the coarse matrix denoted later by K_0 , defined in section 5.4 when the BDDC coarse basis Ψ is introduced. In the implementation, the factorized matrix is used inside `applyM_bddc.m`.

This separation keeps the common layer lightweight while ensuring that both solvers consistently reuse the same primal maps and the same (c, Δ) split created in Chapter 4.

5.2.4 Multiplicity scaling and interface averaging

Both solvers use the same baseline scaling information derived from the product/assembled interface bookkeeping. For each assembled (hat) interface DOF there are k product-space copies, one for each adjacent subdomain. Multiplicity scaling assigns the weight $1/k$ to each copy. In the implementation, these weights are computed by

$$\omega = \text{multiplicity_scaling}(\text{prod}),$$

where `prod` is the product-interface structure from Chapter 4 (file

`common/scaling/multiplicity_scaling.m`). The vector ω has length `prod.nProd` and is defined so that the weights within each hat-group sum to one.

In BDDC, ω is used to implement the scaled distribution/averaging operators $R_D = DR$ and $R_D^T = R^T D$ without forming D explicitly (scaling is applied by pointwise multiplication in product space). In FETI-DP, the same weights are restricted to the Δ -unknowns and stored as `DeltaWeights`; these are then used in the baseline multiplier-space preconditioner (section 5.3.4).

5.2.5 Shared diagnostics and experiment harness

To keep the solver setup deterministic and consistent across experiments, the file

`common/diagnostics/build_problem_data.m` provides a thin problem builder

$$\text{data} = \text{build_problem_data}(n, nSubX, nSubY, f),$$

which reuses the Chapter 3 FEM assembly and the Chapter 4 decomposition/interface pipeline. For a structured unit-square mesh with parameter n , the builder creates:

- the mesh `p,t` and boundary information `bn`,
- the global FEM matrices `K,F` and the Dirichlet-reduced system `Kff,Ff` together with the free index set `free`,
- the structured subdomain decomposition `ddm,sub` and interface bookkeeping `iface,prod`,
- the primal selection/maps `primal` and the local Schur data on each subdomain (including explicit local Schur matrices `sub(i).S`).

The structured partition requires n divisible by both $nSubX$ and $nSubY$ and uses at least two subdomains.

Finally, for deterministic auxiliary tests and experiments, the file `common/utis/rng_deterministic.m` provides `rng_deterministic(seed)`, which sets the random states of `rand` and `randn`. This allows all randomized runs (when used) to be reproduced exactly.

This completes the shared solver infrastructure used by both methods. We now turn to the method-specific formulation of FETI-DP.

5.3 FETI-DP solver (multiplier formulation)

Chapter 4 reduced the discrete Poisson problem to an equivalent interface formulation by eliminating interior DOFs independently on each subdomain. In the product interface space $W = \prod_{i=1}^N W_i$, this reduction separates locality from coupling: all subdomain contributions are represented by the block-diagonal Schur operator $S = \text{diag}(S^{(1)}, \dots, S^{(N)})$, while global conformity is expressed only by explicit jump constraints $Bw = 0$.

The purpose of this section is to turn this interface setting into the concrete FETI-DP solver used later in the Octave implementation. The method enforces the reduced continuity requirement on the non-primal interface by Lagrange multipliers, and the Krylov iteration is organized around these multipliers rather than around a primal interface unknown. This requires (i) an energy/constrained-minimization viewpoint on the reduced interface problem and (ii) a dual-primal split in which selected “primal” interface DOFs are assembled strongly, while the remaining interface DOFs are constrained weakly [11].

5.3.1 Reduced interface energy and dual-primal split

After interior elimination, each subdomain Ω_i contributes a (local) Schur complement operator $S^{(i)}$ acting on the trace space W_i . In the product viewpoint, all local traces are stacked into

$$w = \begin{pmatrix} w^{(1)} \\ \vdots \\ w^{(N)} \end{pmatrix}, \quad w^{(i)} \in W_i,$$

$$S = \text{diag}(S^{(1)}, \dots, S^{(N)}),$$

so applying S is purely local (one independent action per subdomain).

Even if the original stiffness matrices are sparse, Schur complements $S^{(i)}$ are, in general, denser than the original blocks because they represent the effect of eliminating interior DOFs. (This is one practical reason why the implementation in Chapter 5 uses matrix-free applications of Schur operators via local solves rather than explicitly forming large dense matrices.)

The product space duplicates interface DOFs: the same physical interface node has one copy in each adjacent subdomain trace W_i . Continuity is therefore not automatic and must be imposed explicitly by a jump operator

$$B : W \rightarrow \mathbb{R}^m, \quad Bw = 0,$$

whose rows encode differences of duplicated values across shared interfaces. The assembled (single-valued) interface space $\widehat{W}$ can be characterized as $\widehat{W} = \ker(B)$. The link between the assembled and the product representations is given by the assembly mapping $R : \widehat{W} \rightarrow W$, which distributes a single-valued interface vector into consistent local copies; structurally, $\text{range}(R) = \ker(B)$ [13].

This interface reduction admits the same quadratic energy interpretation that was used earlier in the thesis for SPD systems. In the product variables, the reduced interface energy has the form

$$J(w) = \frac{1}{2} w^T S w - g^T w,$$

where $g \in W$ is the reduced interface right-hand side obtained after eliminating interior DOFs (it collects the interface contributions from the original load vector and the effect of imposed boundary conditions). In the assembled setting, the physically relevant interface trace is required to be continuous, i.e. $w \in \ker(B)$. Therefore, the reduced interface problem can be written equivalently as the constrained minimization problem

$$\min_{w \in W} \frac{1}{2} w^T S w - g^T w \quad \text{subject to} \quad Bw = 0. \quad (5.1)$$

This formulation is useful because it separates the two central roles in non-overlapping domain decomposition: the quadratic term contains only local subdomain contributions through the block-diagonal operator S , while the constraint contains the entire global coupling [8].

The remaining ingredient needed for dual-primal methods is the selection of primal (coarse)

interface DOFs $\Gamma_c \subset \Gamma$, which induces a decomposition of the product interface space

$$W = W_c \oplus W_\Delta.$$

Here W_c contains the components associated with Γ_c , and W_Δ contains all remaining interface DOFs (still duplicated across subdomains). In the model experiments of this thesis, Γ_c is chosen as the set of subdomain crosspoints/corners, but the dual–primal construction itself does not require identical square subdomains; it only requires a non-overlapping partition and a consistent choice of primal DOFs that “connects” neighboring subdomains through a small globally assembled space.

The practical meaning of this choice is tied to scalability. In this section, scalability is meant primarily in the numerical sense: after suitable primal constraints are chosen, the condition number of the preconditioned interface operator and the number of PCG iterations should not deteriorate significantly when the mesh is refined or when the number of subdomains is increased in a controlled way. This is distinct from parallel scalability, which concerns wall-clock time, speedup, efficiency, communication overhead, and load balance when the algorithm is executed on multiple processors. Parallel scalability depends additionally on the implementation of local solves, coarse problem solution, communication patterns, and the distribution of subdomains among processors.

The primal layer Γ_C is therefore introduced mainly to ensure numerical scalability: it provides a small global coarse space that transmits low-frequency interface components across the decomposition; without such a coarse component, purely local corrections coupled only through constraints typically lead to iteration counts that grow strongly with the number of subdomains [13]. This is why Chapter 4 introduces Γ_c as a structural prerequisite for modern dual–primal methods.

To express the split algebraically, reorder each local trace vector as

$$w^{(i)} = \begin{bmatrix} w_c^{(i)} \\ w_\Delta^{(i)} \end{bmatrix}, \quad S^{(i)} = \begin{bmatrix} S_{cc}^{(i)} & S_{c\Delta}^{(i)} \\ S_{\Delta c}^{(i)} & S_{\Delta\Delta}^{(i)} \end{bmatrix},$$

where the block structure is defined purely by which local interface indices belong to Γ_c (primal) and which belong to $\Gamma_\Delta = \Gamma \setminus \Gamma_c$ (non-primal). The global product operator $S = \text{diag}(S^{(i)})$ inherits the same block partitioning:

$$S = \begin{bmatrix} S_{cc} & S_{c\Delta} \\ S_{\Delta c} & S_{\Delta\Delta} \end{bmatrix}, \quad S_{**} = \text{diag}(S_{**}^{(1)}, \dots, S_{**}^{(N)}),$$

so each block S_{cc} , $S_{c\Delta}$, $S_{\Delta c}$, $S_{\Delta\Delta}$ is itself block diagonal with respect to subdomains (there is no cross-subdomain coupling inside S ; all coupling remains in the constraint operator) [10].

Because primal variables are intended to be single-valued by construction, continuity constraints are needed only for the non-primal component. Writing the jump constraints in split form yields

the reduced constraint system

$$B_\Delta w_\Delta = 0, \quad (5.2)$$

where B_Δ is the restriction of B to the DOFs in W_Δ . This is exactly the reduced continuity requirement already stated in section 5.1.1: W_c is treated by strong assembly, and only W_Δ is coupled by explicit constraints.

With these ingredients, the constrained minimization problem (5.1) can be rewritten in the dual–primal variables (w_c, w_Δ) as

$$\min_{w_c \in W_c, w_\Delta \in W_\Delta} \frac{1}{2} \begin{bmatrix} w_c \\ w_\Delta \end{bmatrix}^T \begin{bmatrix} S_{cc} & S_{c\Delta} \\ S_{\Delta c} & S_{\Delta\Delta} \end{bmatrix} \begin{bmatrix} w_c \\ w_\Delta \end{bmatrix} - \begin{bmatrix} g_c \\ g_\Delta \end{bmatrix}^T \begin{bmatrix} w_c \\ w_\Delta \end{bmatrix} \quad \text{subject to} \quad B_\Delta w_\Delta = 0.$$

This is the point at which the FETI-DP formulation begins: the next step is to enforce (5.2) by Lagrange multipliers λ , leading to an explicit saddle-point system whose elimination produces the SPD operator in the multiplier space used by PCG.

5.3.2 Lagrangian formulation and saddle-point system

The constrained minimization problem obtained in section 5.3.1 has the standard structure: “quadratic energy + linear constraints”. In the present interface framework, this is particularly natural, the quadratic term is built from local Schur complements and is therefore also local (block diagonal in the product space), while the coupling between subdomains is carried entirely by the reduced continuity requirement on Γ_Δ .

To write the saddle-point system in a form that matches the dual–primal split, the interface unknowns are ordered as

$$w = \begin{bmatrix} w_c \\ w_\Delta \end{bmatrix} \in W_c \oplus W_\Delta, \quad g = \begin{bmatrix} g_c \\ g_\Delta \end{bmatrix}, \quad S = \begin{bmatrix} S_{cc} & S_{c\Delta} \\ S_{\Delta c} & S_{\Delta\Delta} \end{bmatrix}. \quad (5.3)$$

Here w_c collects the (still product-space) traces on Γ_c (one copy per adjacent subdomain), and w_Δ collects the remaining traces on Γ_Δ . The block decomposition in (5.3) is obtained by applying the same (c, Δ) -ordering on each subdomain interface and then stacking the subdomains in the product space; consequently, each block $S_{cc}, S_{c\Delta}, S_{\Delta c}, S_{\Delta\Delta}$ is itself a block diagonal with one local block per subdomain (the “off-diagonal blocks” $S_{c\Delta}, S_{\Delta c}$ couple c - and Δ -variables only within the same subdomain, never across different subdomains).

The reduced continuity requirement on Γ_Δ is

$$B_\Delta w_\Delta = 0,$$

where B_Δ is the reduced jump operator introduced in Chapter 4 (with dependent rows removed so that it has full row rank, as stated in the introduction to section 5.3).

The reduced constrained energy minimization can, therefore, be enforced by Lagrange multipliers. Define the Lagrangian

$$\mathcal{L}(w_c, w_\Delta, \lambda) = \frac{1}{2} \begin{bmatrix} w_c \\ w_\Delta \end{bmatrix}^T \begin{bmatrix} S_{cc} & S_{c\Delta} \\ S_{\Delta c} & S_{\Delta\Delta} \end{bmatrix} \begin{bmatrix} w_c \\ w_\Delta \end{bmatrix} - \begin{bmatrix} g_c \\ g_\Delta \end{bmatrix}^T \begin{bmatrix} w_c \\ w_\Delta \end{bmatrix} + \lambda^T (B_\Delta w_\Delta).$$

The multiplier vector λ has one component per independent reduced jump constraint (per row of B_Δ). In the usual mechanics interpretation λ represents interface reaction forces required to enforce continuity; algebraically, λ is “conjugate” to the jump because it appears paired with the jump quantity $B_\Delta w_\Delta$ in the bilinear term $\lambda^T (B_\Delta w_\Delta)$ [13].

The solution (w_c, w_Δ, λ) is characterized by the stationarity of $\mathcal{L}$ with respect to all variables. Since $\mathcal{L}$ is quadratic in (w_c, w_Δ) and linear in λ , the partial derivatives are explicit:

$$\frac{\partial \mathcal{L}}{\partial w_c} = S_{cc}w_c + S_{c\Delta}w_\Delta - g_c, \quad \frac{\partial \mathcal{L}}{\partial w_\Delta} = S_{\Delta c}w_c + S_{\Delta\Delta}w_\Delta - g_\Delta + B_\Delta^T \lambda, \quad \frac{\partial \mathcal{L}}{\partial \lambda} = B_\Delta w_\Delta. \quad (5.4)$$

Setting all equations (derivatives) in (5.4) to zero yields the three coupled equations

$$S_{cc}w_c + S_{c\Delta}w_\Delta = g_c, \quad (5.5)$$

$$S_{\Delta c}w_c + S_{\Delta\Delta}w_\Delta + B_\Delta^T \lambda = g_\Delta, \quad (5.6)$$

$$B_\Delta w_\Delta = 0. \quad (5.7)$$

Equation (5.7) is the reduced continuity condition (the “jump constraint”). Equation (5.6) shows how the multiplier enters the local interface equilibrium: the term $B_\Delta^T \lambda$ is an additional interface load acting on the product-space Δ -trace unknowns, adjusting the independently computed subdomain traces so that the jumps vanish.

Equations (5.5)–(5.7) can be written compactly as the so called Karush-Kuhn-Tucker saddle point system (KKT system):

$$\begin{bmatrix} S_{cc} & S_{c\Delta} & 0 \\ S_{\Delta c} & S_{\Delta\Delta} & B_\Delta^T \\ 0 & B_\Delta & 0 \end{bmatrix} \begin{bmatrix} w_c \\ w_\Delta \\ \lambda \end{bmatrix} = \begin{bmatrix} g_c \\ g_\Delta \\ 0 \end{bmatrix}. \quad (5.8)$$

The block matrix in (5.8) is symmetric but indefinite (it is a saddle-point matrix), so it is not directly a valid target for PCG. This is the point where the “dual–primal” design becomes essential: continuity on Γ_c will be imposed strongly by switching from the duplicated w_c to assembled primal

unknowns, and the remaining variables will then be eliminated to obtain an operator that is SPD in the multiplier space [15].

A technical condition that will be used in the elimination step is the invertibility of subdomain $\Delta\Delta$ -blocks after the primal layer is enforced. In the Poisson setting of this thesis, choosing a primal set Γ_c that removes the local kernel of floating subdomains (rigid-body modes) makes the resulting constrained local interface operators positive definite; in particular, the corresponding $\Delta\Delta$ -blocks become SPD and hence invertible. This is the algebraic reason why the subsequent reduction to a dual SPD operator is well defined [15].

5.3.3 Primal enforcement and elimination (coarse correction)

The previous section rewrote the reduced interface problem in dual-primal variables (w_c, w_Δ) and enforced the reduced continuity constraint $B_\Delta w_\Delta = 0$ by Lagrange multipliers λ . In the notation of section 5.3.1, w_c represents the primal part of the product-interface trace, w_Δ the remaining (duplicated) part, and the reduced Schur operator is partitioned accordingly to (5.3) with B_Δ containing only the jump constraints on the non-primal component. With multipliers, the stationarity of the Lagrangian yields the saddle-point KKT system (5.8). This system is symmetric but indefinite (a standard feature of constrained quadratic minimization), so it is not directly a target for PCG.

The purpose of the present section is to incorporate the primal layer Γ_c “strongly” (by global identification) and to eliminate (w_c, w_Δ) so that the remaining operator in λ becomes SPD on the reduced multiplier space described in the introduction of section 5.3.

Assembled primal unknowns and the injection R_c

The primal set Γ_c is meant to be single-valued by construction: primal DOFs are globally identified and therefore do not require jump constraints. Algebraically, this is enforced by switching from the product representation w_c (containing copies on each subdomain that touches a primal node) to an assembled primal vector u_c that contains exactly one unknown per physical primal DOF.

This switch is expressed by an injection (distribution) operator

$$R_c : \widehat{W}_c \rightarrow W_c, \quad w_c = R_c u_c,$$

where $\widehat{W}_c$ is the assembled primal space (single-valued on Γ_c), and W_c is its representation inside the product interface space. Conceptually, R_c “copies” each assembled primal value into all subdomains that share the corresponding primal node. Its transpose R_c^T performs the reverse operation: it assembles product primal vectors by summing contributions of their local copies into the unique assembled primal entry. (If an averaged assembled value is desired, one composes R_c^T with a scaling; in this thesis multiplicity scaling is introduced for this purpose in section 5.2.4.)

The two consequences of using u_c instead of w_c are important.

- 1) Primal continuity is built into the variable itself (no multipliers or constraints are needed on Γ_c).
- 2) The dimension of the primal unknown becomes the number of global primal DOFs, rather than the number of their product copies, which is typically larger.

Substitution into the KKT system

Substituting $w_c = R_c u_c$ into the first two block rows of (5.8) gives equations that still live partly in the product coordinates. To obtain an equation purely in assembled primal coordinates, the primal equilibrium equation is assembled by applying R_c^T to the first row. The transformed KKT system, therefore, reads

$$\begin{bmatrix} R_c^T S_{cc} R_c & R_c^T S_{c\Delta} & 0 \\ S_{\Delta c} R_c & S_{\Delta\Delta} & B_\Delta^T \\ 0 & B_\Delta & 0 \end{bmatrix} \begin{bmatrix} u_c \\ w_\Delta \\ \lambda \end{bmatrix} = \begin{bmatrix} R_c^T g_c \\ g_\Delta \\ 0 \end{bmatrix}. \quad (5.9)$$

This system represents the same stationarity conditions as (5.8), but now with primal continuity enforced strongly through the assembled unknown u_c .

Eliminating w_Δ and the coarse (primal) correction

The next step is to eliminate the trace variables (u_c, w_Δ) in a way that makes the role of the primal layer explicit. Consider the second block row of (5.9),

$$S_{\Delta c} R_c u_c + S_{\Delta\Delta} w_\Delta + B_\Delta^T \lambda = g_\Delta.$$

Assuming $S_{\Delta\Delta}$ is invertible on the non-primal interface unknowns, this yields

$$w_\Delta = S_{\Delta\Delta}^{-1} (g_\Delta - S_{\Delta c} R_c u_c - B_\Delta^T \lambda). \quad (5.10)$$

In the present Poisson setting, local Schur complements are SPD whenever the underlying subdomain problem is properly anchored by boundary conditions; if a subdomain is “floating” (pure Neumann locally), the primal layer is precisely the mechanism that removes the corresponding kernel and restores uniqueness at the reduced level. This is one of the structural motivations for the introduction of Γ_c .

Substituting (5.10) into the assembled primal equation (first row of (5.9)),

$$R_c^T S_{cc} R_c u_c + R_c^T S_{c\Delta} w_\Delta = R_c^T g_c,$$

gives a closed equation for the assembled primal unknown u_c :

$$\left[R_c^T (S_{cc} - S_{c\Delta} S_{\Delta\Delta}^{-1} S_{\Delta c}) R_c \right] u_c = R_c^T (g_c - S_{c\Delta} S_{\Delta\Delta}^{-1} g_\Delta) + \left[R_c^T S_{c\Delta} S_{\Delta\Delta}^{-1} B_\Delta^T \right] \lambda. \quad (5.11)$$

This is the point where the “coarse correction” appears naturally: it is the solution of a small global system posed only in the assembled primal space $\widehat{W}_c$. Define

$$K_{cc}^* := R_c^T (S_{cc} - S_{c\Delta} S_{\Delta\Delta}^{-1} S_{\Delta c}) R_c, \quad h_c := R_c^T (g_c - S_{c\Delta} S_{\Delta\Delta}^{-1} g_\Delta), \quad C := R_c^T S_{c\Delta} S_{\Delta\Delta}^{-1} B_\Delta^T. \quad (5.12)$$

Then (5.11) becomes the compact “coarse problem”

$$K_{cc}^* u_c = h_c + C\lambda. \quad (5.13)$$

Several remarks clarify its meaning and its role inside the multiplier iteration.

1) K_{cc}^* is a Schur complement: it is the effective stiffness seen by the primal unknowns after the non-primal interface variables have been eliminated locally. Because the primal unknowns are globally assembled, K_{cc}^* is a global (coupling) operator even though it is built from local pieces.

2) The right-hand side depends on λ . Therefore, whenever the dual iteration requests an operator application involving a new λ , an updated coarse right-hand side $h_c + C\lambda$ must be formed and the coarse system (5.13) must be solved to obtain the corresponding primal component $u_c(\lambda)$. This is exactly what is commonly called the “coarse correction” in FETI-DP: a small auxiliary solve that couples all subdomains and propagates global information efficiently during PCG .

3) Although (5.13) is solved repeatedly during PCG, the matrix K_{cc}^* itself does not change between iterations. It depends only on the decomposition, the primal set Γ_c , and the local Schur operators. Consequently, it is assembled (or represented operator-wise) once, and its factorization (or a small direct solve setup) can be reused; only the right-hand side changes from iteration to iteration.

With $u_c(\lambda)$ obtained from (5.13), the non-primal trace $w_\Delta(\lambda)$ is then recovered from (5.10). In other words, for any multiplier vector λ , the pair $(u_c(\lambda), w_\Delta(\lambda))$ is the solution of the first two block rows of the KKT system under strong primal assembly, and the remaining constraint equation $B_\Delta w_\Delta(\lambda) = 0$ is what produces the final reduced operator in multiplier space.

In summary, enforcing the primal layer means (i) switching to assembled primal unknowns u_c via the injection R_c , and (ii) introducing the coarse system (5.13), whose solution provides the primal component required to define a well-posed (and ultimately SPD) operator acting on the multipliers [10].

5.3.4 Reduced multiplier system and preconditioning

The transformed KKT system with strongly enforced primal continuity is

$$\begin{bmatrix} R_c^T S_{cc} R_c & R_c^T S_{c\Delta} & 0 \\ S_{\Delta c} R_c & S_{\Delta\Delta} & B_\Delta^T \\ 0 & B_\Delta & 0 \end{bmatrix} \begin{bmatrix} u_c \\ w_\Delta \\ \lambda \end{bmatrix} = \begin{bmatrix} R_c^T g_c \\ g_\Delta \\ 0 \end{bmatrix}.$$

section 5.3.3 already eliminated w_Δ from the second block row (assuming $S_{\Delta\Delta}$ invertible on the non-primal interface variables) and obtained the “coarse problem” (5.13) for the assembled primal unknown u_c , with K_{cc}^* , h_c and C defined according to (5.12).

The key point is that K_{cc}^* is independent of λ and can therefore be assembled/factorized once, while only the right-hand side $h_c + C\lambda$ changes from PCG iteration to iteration [1].

This repeated small global solve is the “coarse correction” in FETI-DP: it injects global coupling into an otherwise local (block-diagonal) interface process, which is one of the main mechanisms behind numerical scalability of the dual-primal family.

Coarse corrected inverse and the derivation of the reduced SPD system

In order to write the multiplier operator compactly, it is convenient to introduce the (coarse-corrected) inverse acting on W_Δ ,

$$\tilde{S}^{-1} := S_{\Delta\Delta}^{-1} + S_{\Delta\Delta}^{-1} S_{\Delta c} R_c (K_{cc}^*)^{-1} R_c^T S_{c\Delta} S_{\Delta\Delta}^{-1}. \quad (5.14)$$

It is convenient to view this inverse, which is in fact mapping “right-hand side $\mapsto w_\Delta$ ” as a single corrected inverse on the non-primal interface. That is, the action of $S_{\Delta\Delta}^{-1}$ is augmented by the coarse correction so that the resulting w_Δ is consistent with the assembled primal equilibrium. This corrected inverse is what enters the reduced dual operator derived in the next part.

The assumption that $S_{\Delta\Delta}$ is invertible is the same technical condition already stated in section 5.3.3: in Poisson problems, local Schur complements are SPD when the local problem is properly anchored; if a subdomain is floating, the primal layer (corner values) removes the local kernel and restores uniqueness at the reduced/interface level. This is the standard dual-primal rationale: make the remaining constrained interface operator well-defined and suitable for PCG.

To derive a coarse corrected inverse and resulting reduced SPD system, we must first start from the eliminated form of the second block row (already obtained in section 5.3.3),

$$w_\Delta = S_{\Delta\Delta}^{-1} (g_\Delta - S_{\Delta c} R_c u_c - B_\Delta^T \lambda).$$

Insert the coarse solution $u_c = (K_{cc}^*)^{-1} (h_c + C\lambda)$ from (5.13) and impose the constraint $B_\Delta w_\Delta = 0$ (third block row of (5.9)). This gives

$$B_\Delta S_{\Delta\Delta}^{-1} (g_\Delta - S_{\Delta c} R_c (K_{cc}^*)^{-1} h_c) - B_\Delta S_{\Delta\Delta}^{-1} (B_\Delta^T + S_{\Delta c} R_c (K_{cc}^*)^{-1} C) \lambda = 0. \quad (5.15)$$

Using the definition of C in (5.12), the multiplier-dependent factor can be rewritten as

$$S_{\Delta\Delta}^{-1} (B_\Delta^T + S_{\Delta c} R_c (K_{cc}^*)^{-1} C) = (S_{\Delta\Delta}^{-1} + S_{\Delta\Delta}^{-1} S_{\Delta c} R_c (K_{cc}^*)^{-1} R_c^T S_{c\Delta} S_{\Delta\Delta}^{-1}) B_\Delta^T = \tilde{S}^{-1} B_\Delta^T,$$

where $\tilde{S}^{-1}$ is exactly (5.14). Therefore, (5.15) becomes the reduced multiplier system

$$A_{FETI-DP}\lambda = d, \quad (5.16)$$

with

$$A_{FETI-DP} := B_{\Delta}\tilde{S}^{-1}B_{\Delta}^T, \quad d := B_{\Delta}S_{\Delta\Delta}^{-1}\left(g_{\Delta} - S_{\Delta c}R_c(K_{cc}^*)^{-1}h_c\right). \quad (5.17)$$

This is the concrete “SPD operator in multiplier space” used by FETI-DP: the unknown is the multiplier vector λ , and one PCG iteration consists of applying $A_{FETI-DP}$ (and a chosen preconditioner) to vectors in the multiplier space [1].

To ensure $A_{FETI-DP}$ is strictly positive definite (not merely semidefinite), B_{Δ} must have full row rank; in practice this is enforced by taking a non-redundant set of jump constraints (dependent constraints are removed in the construction of B_{Δ}). Under this standard assumption and with $S_{\Delta\Delta}$ and K_{cc}^* SPD, the operator $A_{FETI-DP} = B_{\Delta}\tilde{S}^{-1}B_{\Delta}^T$ is SPD on the multiplier space and is therefore a valid target for PCG.

Operator-application form

The reduced FETI-DP system is posed in the multiplier space as (5.16), (5.17). In the Octave implementation, neither $A_{FETI-DP}$ nor the corrected inverse $\tilde{S}^{-1}$ is formed explicitly as a matrix. Instead, the PCG method is implemented in the standard matrix-free form: Iteration only requires the ability to evaluate, for a given multiplier vector x , the product

$$y = A_{FETI-DP}x = B_{\Delta}(\tilde{S}^{-1}(B_{\Delta}^T x)).$$

The operator $\tilde{S}^{-1}$ itself is applied by the local-coarse decomposition described in the previous section: its action consists of local subdomain solves on the non-primal trace space combined with a single coarse correction solve with K_{cc}^* . Therefore, each PCG iteration can be realized using only sparse applications of B_{Δ} and B_{Δ}^T , local subdomain solves, and the reusable coarse solve with K_{cc}^* . The detailed realization of these operator applications, including the construction of the right-hand side d and the chosen preconditioner, is given in the implementation and validation section.

Preconditioning of the reduced multiplier system

In this thesis, the PCG iteration in the FETI-DP solver is applied to the reduced SPD system in Lagrange multipliers (5.16), (5.17), so the relevant question is how to precondition this operator $A_{FETI-DP}$ to obtain a practical and scalable iteration.

Preconditioning is the standard technique used to accelerate Krylov methods (here PCG) by replacing the original linear system with an equivalent one whose spectrum is more favorable for iteration. Specifically, instead of working with $A_{FETI-DP}$ alone, PCG repeatedly applies a pre-

conditioner $M_{FETI-DP}^{-1}$ that approximates $A_{FETI-DP}^{-1}$ at low cost. If $M_{FETI-DP}$ is SPD, then the preconditioned operator has a reduced effective condition number, which typically decreases the number of PCG iterations needed to reach a given tolerance.

For FETI-type methods, common choices include lumped preconditioners and Dirichlet preconditioners, often combined with an interface scaling D . The lumped family uses relatively cheap local interface operators (involving the Schur complement blocks) and is attractive for simple, fully matrix-free implementations. Dirichlet variants are typically stronger (often fewer iterations) but require additional local Dirichlet-type solves and extra implementation machinery. Scaling D is used to balance contributions of duplicated interface unknowns; a basic and robust choice is multiplicity scaling, where interface copies are weighted according to how many subdomains share the corresponding physical DOF [10].

In this thesis, to keep the implementation compact and fully consistent with the matrix-free operator model (only “apply-to-vector” routines), we use the scaled lumped preconditioner

$$M_{FETI-DP}^{-1} := B_{\Delta} D S_{\Delta\Delta} D B_{\Delta}^T.$$

Here, $D : W_{\Delta} \rightarrow W_{\Delta}$ is a diagonal scaling assembled from the multiplicity weights introduced earlier, restricted to the Δ -indices. Its action is inexpensive and does not require any coarse solve. For a multiplier residual r , the preconditioned vector $z = M_{FETI-DP}^{-1}r$ is evaluated purely by operator applications:

$$t := B_{\Delta}^T r, \quad t \leftarrow Dt, \quad u := S_{\Delta\Delta} t, \quad u \leftarrow Du, \quad z := B_{\Delta} u.$$

This is exactly the kind of preconditioner that fits the overall PCG workflow used in Chapter 5: it relies only on sparse constraint operators B_{Δ}, B_{Δ}^T , diagonal scaling, and a local Schur-block application $S_{\Delta\Delta}(\cdot)$, without explicitly forming any dense matrices.

5.3.5 PCG method for the reduced multiplier system

The reduced FETI-DP solver of this section is based on the SPD multiplier system (5.16), (5.17) derived in the previous sections. Since $A_{FETI-DP}$ can be ill-conditioned, the system is solved iteratively by the PCG method (PCG) together with the preconditioner $M_{FETI-DP}$ introduced in the preceding section.

As was already explained in the previous section, PCG requires only two operator actions: the matrix-vector product $y = Ax$ and the preconditioner application $z = M^{-1}r$ for a given residual r . In this thesis both actions are implemented in a matrix-free way: $A_{FETI-DP}$ is applied as the composition $x \mapsto B_{\Delta}(\tilde{S}^{-1}(B_{\Delta}^T x))$, and $M_{FETI-DP}^{-1}$ is applied by the corresponding multiplier-space preconditioner routine.

To state the algorithm in a form directly usable later, let λ_k be the current multiplier iterate and define the residual

$$r_k = d - A_{FETI-DP}\lambda_k,$$

together with its preconditioned image

$$z_k = M_{FETI-DP}^{-1}r_k.$$

All inner products are Euclidean, $(u, v) = u^T v$. The left-preconditioned PCG iteration reads [7], [16]:

Initialization (typically $\lambda_0 = 0$):

$$r_0 = d - A_{FETI-DP}\lambda_0, \quad z_0 = M_{FETI-DP}^{-1}r_0, \quad p_0 = z_0.$$

For $k = 0, 1, 2, \dots$ until convergence:

1) operator application:

$$q_k = A_{FETI-DP}p_k,$$

2) step length:

$$\alpha_k = \frac{(r_k, z_k)}{(p_k, q_k)},$$

3) update iterate:

$$\lambda_{k+1} = \lambda_k + \alpha_k p_k,$$

4) update residual:

$$r_{k+1} = r_k - \alpha_k q_k,$$

5) stopping test (relative residual):

$$\|r_{k+1}\| \leq \text{tol} \|r_0\| \quad (\text{or stop after maxit iterations}),$$

6) preconditioner application:

$$z_{k+1} = M_{FETI-DP}^{-1}r_{k+1},$$

7) direction update coefficient:

$$\beta_k = \frac{(r_{k+1}, z_{k+1})}{(r_k, z_k)},$$

8) update search direction:

$$p_{k+1} = z_{k+1} + \beta_k p_k.$$

After PCG converges and the multiplier vector λ is obtained, the corresponding interface traces are reconstructed by the elimination relations derived earlier. First, compute the assembled primal

component u_c from the coarse problem (5.13), (5.12). Then recover the non-primal product-space trace using (5.10) and distribute the primal copies

$$w_c = R_c u_c.$$

The full product-space interface vector $w = (w_c, w_\Delta)$ then determines the interior unknowns by independent local subdomain reconstructions (harmonic extension with the known interface trace), exactly as in the interface reduction of Chapter 4.

5.3.6 FETI-DP solver - Octave implementation

Implementation principles

The Octave realization of FETI-DP is built around a matrix-free reduced problem in multiplier space. Neither the reduced operator $A_{FETI-DP}$ nor the preconditioner $M_{FETI-DP}^{-1}$ is assembled as a global matrix. The PCG solver receives only two operator actions, $x \mapsto A_{FETI-DP}x$ and $r \mapsto M_{FETI-DP}^{-1}r$, implemented as function handles and passed to the common wrapper `pcg_wrap.m`. Multiplications by the reduced jump operator are sparse; the dominant cost in $A_{FETI-DP}$ is the application of $\tilde{S}^{-1}$, realized via independent local solves (one per subdomain) and one coarse/primal solve with a reused factorization.

Module layout and main entry points

The solver-specific code is located in the directory `feti_dp/` and is organized into three parts:

- `feti_dp/setup/setup_fetidp.m`: builds all FETI-DP data structures and factorizations needed for multiplier-space PCG;
- `feti_dp/operators/`: matrix-free operator and preconditioner actions (`applyA_lambda.m`, `applyM_lambda.m`, `solve_tildeS.m`);
- `feti_dp/solve/`: PCG driver and reconstruction (`solve_fetidp.m`, `reconstruct_fetidp_solution.m`).

All routines assume that the Chapter 3–4 pipeline has already produced the common problem structure `data`, typically by calling `build_problem_data.m` (section 5.2.5).

Setup phase (`setup_fetidp.m`)

The setup routine extends `data` by the objects required by the dual-primal formulation:

- *Packed Δ indexing*. The global list of product-space Δ -unknowns is taken from the primal maps (`primal.delta_idx`) and stored as `data.delta_idx_prod`. For efficient block-local

work, each subdomain stores a consecutive range `data.delta_range{i}` describing where its local Δ unknowns are placed inside the packed vector w_Δ .

- *Local Schur blocks and factorizations.* The explicitly assembled local Schur matrix `sub(i).S` (requested already in `build_problem_data`) is split into the local primal/ Δ blocks $S_{cc}^{(i)}, S_{c\Delta}^{(i)}, S_{\Delta c}^{(i)}, S_{\Delta\Delta}^{(i)}$ using the local index sets `sub(i).idx_c` and `sub(i).idx_d`. The block $S_{\Delta\Delta}^{(i)}$ is factorized by Cholesky and the triangular factor is stored as `data.Sdd_R{i}` for repeated applications of $S_{\Delta\Delta}^{(i)-1}$.
- *Reduced jump operator on Δ .* The full jump operator B is constructed from the product interface by `build_jump_operator_B(prod)` and restricted to Δ columns. To obtain a full-row-rank operator (the reduced multiplier space), the code performs a pivoted QR-based row selection and stores the reduced operator as `data.Bd` together with `data.BdT = Bd'` and the selected row indices `data.lambda_rows`. The number of multipliers is `data.nLambda = size(Bd,1)`.
- *Multiplicity scaling on Δ .* The product-space multiplicity weights ω (section 5.2.4) are restricted to the packed Δ indices and stored as `data.DeltaWeights`. These weights implement the diagonal scaling D used in the baseline preconditioner.
- *Coarse/primal operator.* The coarse matrix K_{cc} (assembled from local contributions using the split blocks and the stored Cholesky factors of $S_{\Delta\Delta}^{(i)}$) is stored as `data.Kcc` and factorized once; the triangular factor is stored as `data.Kcc_R`. The corresponding coarse right-hand side contribution is stored as `data.hc`.

Matrix-free operator actions

The reduced multiplier operator $A_{FETI-DP}$ is applied by `applyA_lambda.m` in the operational sequence

$$t = B_d^T x, \quad w_\Delta = [\tilde{S}^{-1}(0, t)]_\Delta, \quad y = B_d w_\Delta,$$

where $\tilde{S}^{-1}$ is applied by `solve_tildeS.m`. The routine `solve_tildeS.m` implements the block-structured action of $\tilde{S}^{-1}$ on a right-hand side split into primal and Δ parts (r_c, r_Δ): it performs independent solves with $S_{\Delta\Delta}^{(i)}$ (using `data.Sdd_R{i}`), assembles the coarse right-hand side, solves the coarse system with the stored factorization `data.Kcc_R`, and returns both the coarse vector u_c and the packed Δ vector w_Δ .

The baseline preconditioner action $M_{FETI-DP}^{-1}$ is applied by `applyM_lambda.m` using the recipe

$$t = B_d^T r, \quad t = Dt, \quad u = S_{\Delta\Delta} t, \quad u = Du, \quad z = B_d u,$$

where $S_{\Delta\Delta}$ is the block-diagonal operator with blocks $S_{\Delta\Delta}^{(i)}$ stored in `data.Sdd{i}`, and the diagonal scaling D is implemented by the elementwise product with `data.DeltaWeights`.

PCG driver and reconstruction

The routine `solve_fetidp.m` forms the multiplier right-hand side using the full condensed interface right-hand side split into primal and Δ parts:

$$b = B_d[\tilde{S}^{-1}(g_c, g_\Delta)]_\Delta,$$

where g_Δ is assembled into the packed Δ vector using `data.delta_range` and g_c is assembled into the coarse vector using `sub(i).c_ids`. Then PCG is run via

```
[lambda, stats] = pcg_wrap(@(x)applyA_lambda(x,data), b, tol, maxit,
    @(r)applyM_lambda(r,data), x0).
```

After PCG, the routine `reconstruct_fetidp_solution.m` reconstructs the coarse/primal vector w_c , the packed Δ trace w_Δ , and the global solution on free DOFs. The reconstruction reuses the stored factorizations of $S_{\Delta\Delta}^{(i)}$ and K_{cc} , and recovers subdomain interior unknowns by local back-substitution (using the interior factors stored in the Chapter 4 Schur's setup). Finally, the global free-DOF vector is assembled by averaging repeated contributions on overlapping interface DOFs.

5.4 BDDC solver (assembled interface formulation)

Section 5.3 derived the FETI-DP solver by enforcing the reduced continuity requirement

$$B_\Delta w_\Delta = 0$$

through Lagrange multipliers and then eliminating local trace variables to obtain a reduced SPD problem in multiplier space, suitable for PCG. In this section we derive the second solver of this thesis—BDDC—within exactly the same interface framework of Chapter 4 and the same Octave-oriented workflow of this section: the local Schur complements remain fully decoupled in the product space $W = \prod_i W_i$, the primal set Γ_c is kept globally identified, and the only essential difference is again the mechanism used to enforce continuity on the remaining interface DOFs on Γ_Δ .

In contrast to FETI-DP, BDDC avoids multipliers and stays entirely in a primal (assembled) frame: the Krylov iteration is performed on a single-valued assembled interface quantity, and continuity on Γ_Δ is enforced by producing assembled interface corrections through averaging/scaling of product-space quantities. The application of the assembled interface operator is performed in the same matrix-free spirit as in section 5.2: an assembled interface vector is distributed to consistent local copies by the assembly mapping $R : \widehat{W} \rightarrow W$, local Schur actions are applied independently

on each subdomain through the block-diagonal operator $S = \text{diag}(S^{(i)})$, and the resulting product-space reactions are gathered back by R^T . This realizes the assembled operator $A_{BDDC} := R^T S R$ on the assembled interface space $\widehat{W} = \ker(B)$ without ever forming a global dense matrix [1].

The second central ingredient of BDDC is the preconditioner, which is built symmetrically from (i) local constrained corrections computed by independent subdomain solves and (ii) an explicit coarse (primal) correction induced by Γ_c [12]. The need for this coarse layer is the same structural point emphasized already in Chapter 4 and reused throughout this section: purely local subdomain operators cannot transmit smooth global interface components efficiently as the number of subdomains grows, and the globally assembled primal set provides the required communication backbone and (in particular) removes the kernels associated with floating subdomains. In BDDC, this global coupling enters through the preconditioner (rather than through a reduced multiplier operator as in FETI-DP), while the iterated operator remains the assembled primal interface operator.

The remainder of this section follows the same principles as sections 5.2–5.3: we keep the derivation and notation consistent with Chapter 4 objects (S , R , Γ_c , the split into primal and non-primal interface parts) and we describe the resulting algorithm in a form that maps directly to the shared Octave tools (matrix-free operator application, the common PCG wrapper, and the already introduced multiplicity scaling used for interface averaging).

5.4.1 Assembled interface operator

In the BDDC method we remain entirely in the primal (assembled) interface viewpoint: the Krylov iteration is performed directly on a single-valued interface trace, and continuity is built into the space of iterates. This is consistent with the structural setting of Chapter 4, where the assembled interface space is characterized as $\widehat{W} = \ker(B)$ and related to the product space W by the assembly mapping $R : \widehat{W} \rightarrow W$.

The subdomain contributions are represented by the same block-diagonal Schur operator as in the previous parts of this section. After the interior DOFs are eliminated, each subdomain Ω_i contributes its local Schur complement $S^{(i)}$ acting on the subdomain interface DOFs, and the global operator on the product interface space is

$$S = \text{diag}(S^{(1)}, \dots, S^{(N)}) \quad \text{acting on } W.$$

The BDDC iteration is formulated on the assembled interface space $\widehat{W}$. Let $\widehat{u} \in \widehat{W}$ denote the assembled interface unknown (the global interface trace). Its product-space representation is obtained by distribution

$$w = R\widehat{u} \in W,$$

so that each assembled interface value is copied into all its local subdomain interface copies. Apply-

ing the block-diagonal Schur operator produces the corresponding product-space interface reactions

$$q = Sw = S(R\hat{u}) \in W,$$

and these reactions are then assembled back to the single-valued interface indexing by

$$A_{BDDC}\hat{u} := R^T q = R^T S R \hat{u}.$$

This defines the assembled interface operator [1]

$$A_{BDDC} := R^T S R \quad \text{on } \widehat{W}.$$

In the Octave implementation, this operator is applied matrix-free: one distributes by R , applies the local Schur complements independently on each subdomain through the block-diagonal operator S , and gathers the resulting product-space reactions by R^T .

As in Chapter 4, interior elimination also produces reduced load vectors $g^{(i)}$ on subdomain interfaces. It is convenient to collect these into the product-space vector $g = (g^{(1)}, \dots, g^{(N)}) \in W$ and assemble the corresponding right-hand side by the same gathering operation,

$$\hat{f} := R^T g.$$

The BDDC Krylov iteration then targets the assembled interface equation

$$A_{BDDC}\hat{u} = \hat{f}, \quad \hat{u} \in \widehat{W}. \quad (5.18)$$

In the Poisson “membrane” interpretation used throughout the thesis, $\hat{u}$ represents the interface trace of the deflection. The distributed vector $w = R\hat{u}$ is the same deflection trace represented independently on each subdomain boundary, and the vector $q = S(R\hat{u})$ is the collection of subdomain interface reactions (discrete fluxes/forces) induced by that imposed trace after the interior has equilibrated. The gathering operation R^T sums these subdomain reactions into a single net reaction per global interface DOF; this is the appropriate assembly-by-summation mechanism, analogous to standard finite element assembly. With this interpretation, the residual

$$\hat{r} = \hat{f} - A_{BDDC}\hat{u}$$

measures the equilibrium defect associated with the current interface trace: it is the imbalance of assembled interface reactions with respect to the assembled reduced loads. In other words, convergence corresponds to enforcing equilibrium of the assembled interface reactions induced by the trace $\hat{u}$ with the reduced forcing represented by $\hat{f}$. In FETI-DP, the multipliers provide an explicit representation of interface reaction forces enforcing continuity; in the present primal formulation

the same physical content is expressed implicitly through the Schur reactions and the residual of (5.18), rather than through an additional unknown λ .

The operator $A_{BDDC} = R^T S R$ is symmetric by construction when the local Schur complements are symmetric. Under the same standing assumptions used in this section—namely an elliptic Poisson model with a nonempty Dirichlet boundary ensuring global coercivity, and a primal set Γ_c sufficient to remove kernels associated with locally floating subdomains—the assembled interface system (5.18) is SPD on the assembled interface space and is therefore a valid target for the PCG method.

5.4.2 Coarse basis and BDDC preconditioner

In this section we will introduce the actual BDDC preconditioner used together with the assembled operator from the previous section. The starting point is the assembled interface system on the hat space, with operator $A_{BDDC} = R^T S R$, solved by PCG. In each PCG step, the current assembled interface iterate $\hat{u} \in \widehat{W}$ produces an assembled residual $\hat{r} = \hat{f} - A_{BDDC} \hat{u}$.

In the present substructuring frame, this residual can be interpreted as an interface equilibrium defect: the vector $A_{BDDC} \hat{u} = R^T S(R\hat{u})$ is obtained by distributing the single-valued trace $\hat{u}$ to consistent local copies $R\hat{u} \in W$, applying the independent subdomain Schur actions $S^{(i)}$ (which return the subdomain reactions/fluxes induced by this trace), and finally assembling these reactions by R^T .

On a purely interior interface (no external boundary load at that interface DOF), the correct equilibrium means that the reactions contributed by neighboring subdomains cancel when assembled; in the presence of body loads or Neumann boundary segments, the assembled right-hand side $\hat{f}$ represents the corresponding net interface load. Therefore, the norm of $\hat{r}$ measures how far the current trace is from producing globally balanced interface reactions, and the PCG stopping criterion based on $\|\hat{r}\|$ is precisely a convergence test for this assembled interface equilibrium.

The role of the BDDC preconditioner is to map an assembled residual $\hat{r} \in \widehat{W}$ to an assembled correction $z = M_{BDDC}^{-1} \hat{r} \in \widehat{W}$ by combining (i) inexpensive local subdomain corrections that effectively reduce oscillatory (subdomain-scale) error components and (ii) an explicit coarse (primal) correction that transmits the slowly varying global components across the decomposition. This decomposition into local and coarse layers mirrors the structural purpose of the primal set Γ_c already emphasized in sections 5.2–5.3: without a coarse communication mechanism, purely local Schur-type inverses cannot propagate global information robustly as the number of subdomains grows, and floating subdomains introduce nullspaces that must be controlled. In BDDC, this global coupling enters through the preconditioner (rather than through a reduced multiplier operator, as in FETI-DP).

A crucial practical point is that the assembled operator A_{BDDC} uses the unscaled assembly R^T because the stiffness contributions must be summed when forming the global equilibrium. By

contrast, the preconditioner must average the interface corrections to avoid overcounting at shared interface DOFs. This is where the multiplicity scaling from section 5.2.4 enters. Let $D = \text{diag}(\omega)$ be the diagonal product-space scaling matrix, where each product copy of $\widehat{DOF}$ receives weight $1/k$ if the $\widehat{DOF}$ has k adjacent subdomains. Using this scaling, we define the scaled distribution and averaging operators as follows:

$$R_D := D R, \quad R_D^T := R^T D.$$

The partition-of-unity property of multiplicity scaling implies that $R^T D R = I$ on $\widehat{W}$, which is exactly the statement that distributing an assembled vector and then averaging the copies returns the original assembled vector [12]. In particular, R_D is used to distribute an assembled residual into a product-space right-hand side that is already partitioned among the subdomains, and R_D^T is used to average product-space corrections back into a single-valued assembled correction [1].

With these operators fixed, the BDDC preconditioner is defined in the standard two-level form

$$M_{\text{BDDC}}^{-1} := R_D^T (T_{\text{sub}} + T_0) R_D.$$

Here, T_{sub} is the local (subdomain) correction operator and T_0 is the coarse (primal) correction operator [1], [11]. Both act naturally in the product interface representation and are built from the same block-diagonal Schur operator $S = \text{diag}(S^{(i)})$ as the assembled operator A_{BDDC} , but they differ in how they enforce the primal constraints [1].

The local correction T_{sub} is the BDDC analogue of a one-level Neumann–Neumann type correction restricted to the non-primal interface [17]. In the notation of Chapter 4, after selecting the primal set Γ_c we have the split $W = W_c \oplus W_\Delta$ and the corresponding block structure of each subdomain Schur operator. The local BDDC correction is required to have a vanishing primal component, so that the coarse layer is solely responsible for communicating primal/global information. Algebraically, this is expressed by restricting to the Δ -part and applying local inverses there:

$$T_{\text{sub}} := R_\Delta^T S_{\Delta\Delta}^{-1} R_\Delta,$$

where R_Δ denotes the natural restriction/injection between the full product interface and its Δ -part (consistent with the primal / Δ maps already constructed in Chapter 4), and $S_{\Delta\Delta}^{-1}$ is the block-diagonal inverse of the local $\Delta\Delta$ Schur blocks (implemented by independent constrained subdomain solves). Operationally, this term takes the scaled distributed residual $R_D r$, extracts its Δ -component on each subdomain, solves a local Schur-type problem to produce a product-space interface correction with zero primal values, and returns the assembled correction by scaled averaging R_D^T . Its effect is predominantly local: it quickly reduces error components that are well represented by independent subdomain responses, but it cannot by itself transmit smooth global components efficiently, which is the reason the coarse term is necessary [1].

The coarse correction T_0 is built from the coarse basis matrix Ψ , whose columns are discrete energy-minimizing extensions associated with the primal DOFs [16]. The purpose of Ψ is to provide an explicit set of global coarse modes that are compatible with the subdomain Schur energies and that realize the same “global communication backbone” supplied by Γ_c in the dual formulation of FETI-DP.

Each column of Ψ corresponds to one primal (coarse) DOF and represents the interface trace that results when that primal variable is set to one (and all other primal variables to zero) while all remaining interface values on the involved subdomains are chosen so as to minimize the local interface energy. In other words, for each subdomain Ω_i , the corresponding local block $\Psi^{(i)}$ is obtained by solving a constrained minimization problem of the form

$$\min_{w^{(i)} \in W_i} \frac{1}{2} (w^{(i)})^T S^{(i)} w^{(i)} \quad \text{subject to } C_i w^{(i)} = e_\ell.$$

i.e. subject to the primal constraints on Γ_c selecting the prescribed coarse values, so that the non-primal interface values become the unique minimal-energy (harmonic) completion consistent with the imposed primal data. This construction is a “basis build” step rather than a response to a load.

In the algebraic implementation, this constrained minimization is realized by a small saddle-point solve on each subdomain, with constraints encoding the primal DOFs; only those columns of $\Psi^{(i)}$ are nonzero that correspond to primal DOFs supported on Ω_i .

Once Ψ is assembled from its subdomain blocks, the coarse correction operator is defined as [16]

$$T_0 := \Psi (\Psi^T S \Psi)^{-1} \Psi^T.$$

The matrix $S_0 := \Psi^T S \Psi$ is the coarse (primal) stiffness induced by the Schur energy and is of a small dimension equal to the number of primal DOFs [1].

The action of T_0 on a product-space residual can be interpreted as follows: $\Psi^T r$ extracts the residual components in the coarse basis subspace (coarse “generalized forces”), $(\Psi^T S \Psi)^{-1}$ solves the corresponding coarse problem to determine the amplitudes of the coarse modes needed to reduce these components, and multiplication by Ψ lifts the resulting coarse amplitudes back to a product-space interface correction that is globally consistent in the primal variables and energy-minimizing elsewhere. After the final averaging by R_D^T , this yields an assembled correction that transmits the global information across the entire interface [1].

Putting the pieces together, one application $z = M_{\text{BDDC}}^{-1} r$ proceeds conceptually by (i) distributing and partitioning the assembled residual r to the subdomains by R_D , (ii) computing a sum of a local constrained correction $T_{\text{sub}}(R_D r)$ and a coarse correction $T_0(R_D r)$ in the product interface representation, and (iii) averaging the result back into an assembled correction by R_D^T . This two-level structure is the BDDC counterpart of the “local part + coarse/primal part” decomposition already present in the FETI-DP development of section 5.3 [17]: the coarse problem

is again induced by the primal set Γ_c and reused across iterations, while the remaining work is performed by independent local subdomain solves. The difference is that in FETI-DP the Krylov iteration runs in multiplier space and continuity is enforced explicitly by the constraint operator B_Δ , whereas in BDDC the iteration runs directly on a single-valued assembled interface variable and continuity on the non-primal interface is maintained implicitly by the scaled averaging built into the preconditioner.

5.4.3 PCG method for the BDDC interface system

The BDDC solver is applied to the SPD assembled interface system on the hat space

$$A_{BDDC}\hat{u} = \hat{f}, \quad A_{BDDC} := R^T S R,$$

derived in section 5.4.1. The system is solved by the same PCG wrapper introduced in section 5.2; therefore the generic PCG recurrence is not repeated here. The only method-specific ingredients are the matrix-free action of the BDDC interface operator and the action of the BDDC preconditioner.

For a given assembled interface vector $x \in \widehat{W}$, the operator action $y = A_{BDDC}x$ is evaluated as

$$x \mapsto Rx \mapsto S(Rx) \mapsto R^T S(Rx).$$

Thus, the assembled vector is first distributed to the product interface space, the block-diagonal Schur complement operator is applied locally on subdomains, and the resulting interface reactions are assembled back to the hat space.

The BDDC preconditioner used in PCG is the two-level operator

$$M_{BDDC}^{-1} := R_D^T (T_{\text{sub}} + T_0) R_D,$$

defined in section 5.4.2. Hence, for a residual $\hat{r} \in \widehat{W}$, the preconditioned vector $\hat{z} = M_{BDDC}^{-1}\hat{r}$ is obtained by

$$t := R_D \hat{r}, \quad u := T_{\text{sub}} t + T_0 t, \quad \hat{z} := R_D^T u.$$

Here T_{sub} denotes the local constrained Δ -correction on the non-primal part, while T_0 denotes the coarse correction generated by the BDDC coarse basis Ψ . In particular,

$$T_0 = \Psi(\Psi^T S \Psi)^{-1} \Psi^T.$$

Equivalently, the coarse part is evaluated through the small coarse solve

$$c := \Psi^T t, \quad S_0 \alpha = c, \quad u_0 := \Psi \alpha, \quad S_0 := \Psi^T S \Psi.$$

The PCG iterate is therefore the assembled interface vector $\hat{u}_k$, and the residual monitored by the stopping criterion is

$$\hat{r}_k = \hat{f} - A_{BDDC}\hat{u}_k.$$

This residual has the same meaning as in the assembled interface formulation: it measures the mismatch between the assembled interface reactions induced by the current trace $\hat{u}_k$ and the reduced assembled load vector $\hat{f}$. Thus, convergence of PCG corresponds to convergence to interface equilibrium.

In comparison with the FETI-DP iteration of section 5.3.5, the PCG algorithm itself is unchanged. The difference is the space and the operators on which it acts: FETI-DP iterates on the multiplier λ and applies the dual operator $B_\Delta \tilde{S}^{-1} B_\Delta^T$, whereas BDDC iterates directly on the assembled interface trace $\hat{u}$ and applies the primal operator $R^T S R$ together with the averaging-based two-level preconditioner $R_D^T (T_{\text{sub}} + T_0) R_D$.

After convergence, the vector $\hat{u} \in \widehat{W}$ is the assembled interface trace of the discrete solution. The corresponding product-space trace is obtained by distribution

$$w = R\hat{u} \in W.$$

The interior DOFs on each subdomain are then reconstructed independently by the standard substructuring relation from Chapter 4, with the interface trace fixed. The resulting local subdomain solutions are finally assembled into the global finite element solution by the same assembly principle used throughout the thesis.

5.4.4 BDDC solver - Octave implementation

Implementation principles

The Octave realization of the BDDC solver follows the same matrix-free design as the FETI-DP part. The PCG iteration targets the assembled (hat-space) interface system

$$A_{BDDC}\hat{u} = \hat{f}, \quad A_{BDDC} := R^T S R, \quad \hat{f} := R^T g,$$

and neither A_{BDDC} nor the $BDDC$ preconditioner is assembled as a global matrix. All expensive preprocessing (index maps, local factorizations, and the coarse operator) is performed once in a setup phase and then reused throughout the iteration.

Module layout and main entry points

The solver-specific code is located in the directory `bddc/` and is organized into:

- `bddc/setup/setup_bddc.m`: builds/stores all BDDC objects (distribution/averaging operator, scaling weights, local Δ factorizations, coarse basis and coarse factorization);

- `bddc/operators/`: matrix-free actions (`applyA_hat.m`, `applyM_bddc.m`);
- `bddc/solve/`: PCG driver and reconstruction (`solve_bddc.m`, `reconstruct_bddc_solution.m`).

As in the FETI-DP case, these routines assume that the common structure `data` (mesh, decomposition, interface bookkeeping, primal maps, and local Schur data) has already been built (typically by `build_problem_data.m`).

Setup phase (`setup_bddc.m`)

The setup routine constructs a sub-structure `data.bddc` containing:

- *Distribution/assembly operator R .*

The sparse operator $R : \widehat{W} \rightarrow W$ is built by `build_assembly_operator_R(prod)` and stored as `data.bddc.R`. It maps a hat-space vector into the product interface space by duplicating shared values to all local copies.

- *Multiplicity scaling ω .*

The product-space weights ω are computed by `multiplicity_scaling(prod)` and stored as `data.bddc.omega`. In the implementation, the scaled operators $R_D = DR$ and $R_D^T = R^T D$ are applied without forming D explicitly:

$$R_D x \equiv \omega \odot (R x), \quad R_D^T y \equiv R^T (\omega \odot y),$$

where $\odot$ denotes elementwise multiplication.

- *Condensed interface right-hand side.*

The product-space vector g is assembled as `data.bddc.g_prod` by scattering each local condensed interface right-hand side `sub(i).g` into the global product indexing `sub(i).prod_idx`.

- *Local Δ factorizations.*

For each subdomain, the local $\Delta\Delta$ block $S_{\Delta\Delta}^{(i)}$ is extracted from the explicit local Schur matrix `sub(i).S` using the local index set `sub(i).idx_d` and factorized by Cholesky. The triangular factors are stored as `data.bddc.Sdd_R{i}` and are used for repeated local solves with $S_{\Delta\Delta}^{(i)}$. The corresponding global product indices of local Δ unknowns are stored as `sub(i).prod_idx_d`.

- *Coarse basis Ψ and coarse matrix K_0 .*

The coarse basis $\Psi \in \mathbb{R}^{n_{prod} \times n_C}$ is constructed in product space by energy-minimizing extensions of primal corner values: on each subdomain, the primal values are prescribed on the

local c -indices, and the Δ -part is obtained by solving

$$S_{\Delta\Delta}^{(i)} w_{\Delta}^{(i)} = -S_{\Delta c}^{(i)} w_c^{(i)}.$$

The coarse matrix is then assembled as $K_0 = \Psi^T S \Psi$ using repeated applications of the block-diagonal Schur operator in product space, and factorized once; the factor is stored as `data.bddc.K0_R`.

Matrix-free operator actions

The hat-space operator is applied by `applyA_hat.m` as

$$y = A_{BDDC} \hat{x} = R^T (S(R\hat{x})),$$

where the block-diagonal action of $S = \text{diag}(S^{(i)})$ in product space is performed by the Chapter 4 routine `apply_blockdiag_S`.

The baseline BDDC preconditioner is applied by `applyM_bddc.m` in the form

$$z = R_D^T (T_{\text{sub}} + T_0) R_D r,$$

where T_{sub} is the block-local Δ -correction (independent solves with $S_{\Delta\Delta}^{(i)}$ on each subdomain) and T_0 is the coarse correction

$$T_0 r = \Psi K_0^{-1} \Psi^T r.$$

Both local and coarse solves reuse the stored Cholesky factors `textttdata.bddc.Sdd_R{i}` and `data.bddc.K0_R`.

PCG driver and reconstruction

The routine `solve_bddc.m` forms the hat-space right-hand side $\hat{f} = R^T g$ from the stored product vector `data.bddc.g_prod` and runs PCG via the common wrapper `pcg_wrap.m` with callbacks `applyA_hat` and `applyM_bddc`. After PCG, the routine `reconstruct_bddc_solution.m` reconstructs the product interface trace $w = Ru$, solves for subdomain interiors by local back-substitution (reusing the interior factors prepared in the Chapter 4 Schur's setup), and assembles the global free-DOF solution by averaging repeated contributions on the interface.

Chapter 6

Spectral comparison and numerical experiments

In Chapter 5, the unified dual–primal interface framework from Chapter 4 was developed into two concrete sequential solvers, FETI-DP and BDDC, and their action was implemented in Octave in a matrix-free manner via routines applying the relevant interface operators and preconditioners. The purpose of the present Chapter is to complement these algorithmic constructions with a spectral viewpoint, motivated by the fact that the convergence of the PCG method is governed by the eigenvalue distribution of the preconditioned operator that is actually iterated on [1].

The main spectral comparison in this Chapter is therefore performed in the solver-loop sense: we compare the spectra of the SPD iteration operators seen by PCG. In the notation of the previous Chapters, this means that for FETI-DP we study the spectrum of the preconditioned multiplier-space operator

$$M_{\text{FETI-DP}}^{-1}A_{\text{FETI-DP}},$$

while for BDDC we study the spectrum of the preconditioned assembled interface operator

$$M_{\text{BDDC}}^{-1}A_{\text{BDDC}},$$

These preconditioned spectra are the quantities directly related to the PCG convergence behavior reported later in this Chapter. In addition to this PCG-relevant comparison, we also compute the spectra of the corresponding non-preconditioned reduced operators

$$A_{\text{FETI-DP}}, \quad A_{\text{BDDC}}.$$

These spectra are included as diagnostic information about the raw FETI-DP and BDDC interface formulations before the action of the preconditioners. They should not be interpreted as the primary predictors of the PCG iteration counts, since the actual PCG loops are applied to

the preconditioned systems. Rather, they provide a useful reference point for observing how the preconditioners change the spectral properties of the reduced interface operators.

Since our actual implementation is matrix-free, the full spectra are obtained by constructing explicit matrix representations of the operators from their action on vectors. For a linear mapping $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$, its matrix in the standard basis is assembled column-by-column as $T_{\text{mat}}(:, j) = T(e_j)$, where e_j denotes the j -th unit vector. This procedure, often referred to here as operator probing, yields explicit matrices consistent with the implemented `matvec` and preconditioner routines. The same probing principle is used both for the preconditioned operators and for the non-preconditioned operators; in the former case the probed mapping is the complete preconditioned action, while in the latter case only the raw operator action is probed.

6.1 Basic notions of spectral theory for domain decomposition operators

6.1.1 Eigenvalues and spectrum

In this section, we work exclusively with linear operators that arise after the finite element discretization and the subsequent Schur-complement reduction to subdomain interfaces. In particular, both the FETI-DP and the BDDC method lead to linear mappings acting on finite-dimensional vector spaces (multiplier space in the FETI-DP case and assembled interface space in the BDDC case). For such operators, all spectral concepts can be introduced in the standard matrix setting.

Let $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be a linear operator. A nonzero vector $x \in \mathbb{R}^n$ is called an eigenvector of T if there exists a scalar $\lambda \in \mathbb{R}$ such that the relation

$$Tx = \lambda x$$

holds. The scalar λ is then called an eigenvalue of T , and the pair (λ, x) is called an eigenpair. In the finite-dimensional setting, once a basis is fixed, the operator T is represented by a matrix (denoted by the same symbol), and the eigenpair relation is equivalent to the homogeneous system

$$(T - \lambda I)x = 0,$$

where I denotes the identity matrix in $\mathbb{R}^n$. Nontrivial solutions exist if and only if $\det(T - \lambda I) = 0$ (so called characteristic polynomial), and hence the eigenvalues are precisely the roots of this characteristic polynomial.

The spectrum of T , denoted by $\sigma(T)$, is the set of all eigenvalues of T . In this thesis, it will be convenient to regard the spectrum as a multiset, i.e., to keep track of algebraic multiplicities of eigenvalues. In particular, if an eigenvalue λ occurs as a root of $\det(T - \lambda I)$ with multiplicity m , then it appears in $\sigma(T)$ with multiplicity m . When we speak about the “full spectrum” in the

numerical part of this section, we therefore mean the complete list of all n eigenvalues of T (counted with multiplicities), typically presented in sorted order or in a histogram form.

Finally, we emphasize that in the subsequent sections we will not study spectra of the original saddle-point formulations, but of the reduced operators that are directly iterated on by the PCG method. Consequently, the spectral objects of interest will be the eigenvalues of the corresponding preconditioned iteration operators.

6.1.2 Symmetry and positive definiteness

In the present thesis, the spectral analysis is performed for the concrete interface operators produced by the FETI-DP and BDDC methods. A key common feature of these operators (after the dual-primal reduction described in the previous sections) is that they are designed so that the Krylov iteration is carried out on a SPD (SPD) problem. This is important for two reasons: first, it justifies the use of the PCG (PCG) method, and second, it guarantees that the eigenvalues relevant for the spectral comparison are real (and, in the positive definite case, strictly positive).

Let $T \in \mathbb{R}^{n \times n}$ be a matrix representing a linear operator on $\mathbb{R}^n$. We say that T is symmetric if

$$T = T^\top.$$

A symmetric matrix T is called positive definite if its quadratic form is strictly positive for every nonzero vector,

$$x^\top T x > 0 \quad \text{for all } x \in \mathbb{R}^n, x \neq 0.$$

(If only $x^\top T x \geq 0$ holds, T is positive semidefinite.) In the following, we focus on the positive definite case, since it implies invertibility and excludes nontrivial nullspaces in the iteration space; in particular, the dual-primal constructions used in this thesis yield PCG iteration operators that are SPD on the corresponding iteration spaces.

The symmetry assumption has an immediate spectral consequence: symmetric matrices have only real eigenvalues and admit an orthonormal eigenbasis. Moreover, if T is SPD, then all its eigenvalues are strictly positive,

$$\lambda_i(T) > 0, \quad i = 1, \dots, n.$$

This is the basic reason why it makes sense, in the numerical part of this section, to compare spectra by plotting sorted eigenvalues (as real numbers) and by discussing their spread and clustering.

These general notions apply directly to the two operators studied in this section. For FETI-DP, after enforcing primal continuity and forming the reduced dual formulation, PCG is applied to the reduced multiplier-space operator $A_{\text{FETI-DP}}$. In the standard setting (i.e., after eliminating redundancies so that the constraints are non-degenerate), this operator is SPD on the multiplier space used by the iteration. For BDDC, the interface problem is formulated in the primal viewpoint,

and PCG is applied to the assembled interface operator A_{BDDC} (constructed from local Schur complements and the assembly operator). This operator is again SPD on the assembled interface space.

Finally, since we compare preconditioned operators, it is useful to recall how SPD structure is retained in the spectral viewpoint. If both the operator A and the preconditioner M are SPD, then the eigenvalues relevant for the preconditioned iteration can be characterized in an SPD-consistent way through the generalized eigenvalue problem

$$Ax = \lambda Mx,$$

whose eigenvalues are real and positive. Equivalently, one may view the spectrum via the symmetric similarity form $M^{-1/2}AM^{-1/2}$. This observation will later allow us to compute full spectra in a numerically robust manner while staying consistent with the matrix-free implementation used in this thesis.

6.1.3 Spectral indicators relevant to PCG

In this thesis, both FETI-DP and BDDC are applied in such a way that the interface solve is performed by the PCG method on an SPD operator. Consequently, the behavior of the iteration is closely related to the spectrum of the corresponding preconditioned iteration operator. While a complete convergence theory is beyond the scope of this section, it is useful to recall several basic spectral indicators that will later be reported and used for interpretation of the numerical results.

Let A denote an SPD operator and let M be an SPD preconditioner, both acting on the iteration space. The preconditioned system can be viewed through the generalized eigenvalue problem

$$Ax = \lambda Mx,$$

whose eigenvalues coincide with those of the preconditioned operator $M^{-1}A$ and are real and positive. In the numerical part of this section, these eigenvalues will be computed explicitly (for sufficiently small instances) and analyzed as a set $\{\lambda_i\}_{i=1}^n$.

A classical coarse indicator of difficulty for CG-type iterations is the spectral condition number of the preconditioned operator, defined as

$$\kappa(M^{-1}A) := \frac{\lambda_{\max}}{\lambda_{\min}}, \quad \lambda_{\min} := \min_i \lambda_i, \quad \lambda_{\max} := \max_i \lambda_i.$$

A smaller value of κ typically suggests faster convergence, whereas a large eigenvalue spread may lead to slower reduction of the residual. For this reason, in addition to plotting full spectra, we will report the extremal eigenvalues $\lambda_{\min}$ and $\lambda_{\max}$ and the ratio κ as a compact summary measure.

However, for practical interpretation, the distribution of eigenvalues often provides more information than the condition number alone. In particular, if most eigenvalues form a tight cluster and only a few eigenvalues appear as outliers, the iteration may converge significantly faster than what would be suggested by κ alone. This motivates presenting the full spectrum in a sorted form (and, for selected cases, also as a histogram), which makes it possible to identify clustering, gaps, and outliers. In the experiments below, these qualitative spectral features will be compared between FETI-DP and BDDC in a consistent way.

Finally, since the goal of this section is to connect spectral properties with observed solver performance, the eigenvalue-based indicators will be accompanied by standard PCG diagnostics obtained from the implementation: the iteration count, the achieved relative residual, and the residual history. This allows the numerical discussion to relate the measured spectral distributions of the preconditioned operators directly to the convergence behavior observed in practice.

6.2 Definition of preconditioned iteration operators

6.2.1 Common interface settings and the operators for PCG

In this section we keep exactly the same notation and terminology as in Chapters 4–5 and recall only those objects that are needed to define the two operators whose spectra are compared later. The starting point is the standard interface setting of nonoverlapping domain decomposition: each subdomain Ω_i contributes its local interface space W_i , and we work in the product (duplicated) interface space

$$W = \prod_{i=1}^N W_i.$$

Continuity across subdomain interfaces is expressed by the jump operator B , so that a vector $w \in W$ represents a globally continuous interface trace precisely when

$$Bw = 0, \quad \text{and hence} \quad \widehat{W} = \ker(B).$$

As in Chapter 4, we use the dual–primal viewpoint: a chosen set of primal interface DOFs Γ_c is treated by enforcing continuity directly, which induces the splitting

$$W = W_c \oplus W_\Delta.$$

The remaining continuity conditions are then written only for the Δ -part, in the form

$$B_\Delta w_\Delta = 0.$$

These objects (in particular W , $\widehat{W}$, B , Γ_c , the decomposition $W = W_c \oplus W_\Delta$, and the reduced constraint operator B_Δ) will be used in the next sections when defining the concrete FETI-DP and BDDC operators.

For the spectral comparison, it is crucial to specify what is meant by the “operator” whose spectrum is measured. In this section we always consider the operator that is actually iterated on by PCG, i.e. a preconditioned SPD operator. Concretely, if A is the SPD operator used in the PCG loop and M is the SPD preconditioner, then the relevant iteration mapping is

$$T = M^{-1}A, \quad x \mapsto M^{-1}(Ax).$$

Equivalently (and in a form that preserves symmetry), the same eigenvalues can be obtained from the generalized eigenvalue problem

$$Ax = \lambda Mx.$$

This viewpoint matches the matrix-free implementation: Ax is computed by the operator-application routine and $M^{-1}(\cdot)$ by the preconditioner-application routine. In the following two sections we will define, using the above interface setting, the specific choices of A and M for the FETI-DP and BDDC methods, and thus precisely specify which spectra are computed and compared.

6.2.2 FETI-DP operator

In the FETI-DP method, continuity on the non-primal interface is enforced weakly by Lagrange multipliers. After eliminating the trace variables in the product space and using the dual–primal split introduced earlier, the interface solve reduces to a symmetric system posed directly in the multiplier space. Concretely, the PCG iteration targets the reduced multiplier operator

$$A_{FETI-DP} \lambda = d,$$

with the reduced dual operator defined by

$$A_{FETI-DP} := B_\Delta \tilde{S}^{-1} B_\Delta^\top.$$

Here B_Δ is the reduced jump operator acting on the non-primal trace part, and $\tilde{S}^{-1}$ denotes the corrected inverse arising from the local–coarse decomposition of the Schur complement operator (as introduced in the FETI-DP development in the previous section). In the standard non-redundant setting, $A_{FETI-DP}$ is SPD on the multiplier space used by the iteration; in particular, strict positive definiteness is ensured by taking B_Δ with full row rank (i.e., by removing dependent jump constraints).

The operator is applied in a matrix-free form: for a multiplier vector x , one PCG matvec

evaluates

$$y = A_{FETI-DP}x = B_{\Delta}(\tilde{S}^{-1}(B_{\Delta}^{\top}x)),$$

which emphasizes that one application consists of (i) a sparse constraint transpose $B_{\Delta}^{\top}$, (ii) the action of $\tilde{S}^{-1}$ realized by local subdomain solves plus one coarse correction, and (iii) a sparse constraint application of B_{Δ} .

In addition to the preconditioned operator used in the PCG iteration, we also compute the spectrum of the non-preconditioned FETI-DP operator

$$A_{FETI-DP}.$$

This spectrum describes the conditioning of the raw multiplier-space interface formulation before the action of the FETI-DP preconditioner. It is included as diagnostic information and should not be interpreted as the direct predictor of the PCG iteration counts, since the actual Krylov iteration is applied to the preconditioned operator introduced below.

To accelerate PCG, we introduce an SPD preconditioner $M_{FETI-DP}$, chosen (for the purpose of this thesis) as a scaled lumped preconditioner

$$M_{FETI-DP}^{-1} := B_{\Delta} D S_{\Delta\Delta} D B_{\Delta}^{\top},$$

where $S_{\Delta\Delta}$ is the $\Delta\Delta$ -block of the block-diagonal Schur operator and D is a diagonal scaling on W_{Δ} assembled from the multiplicity weights (restricted to Δ -indices). Its application to a residual r is again purely operator-based,

$$\begin{aligned} t &:= B_{\Delta}^{\top}r, & t &\leftarrow Dt, & u &:= S_{\Delta\Delta}t, \\ u &\leftarrow Du, & z &:= B_{\Delta}u, \end{aligned}$$

so that the preconditioner fits the same matrix-free workflow as the operator $A_{FETI-DP}$ and requires no explicit global matrix assembly.

In this part an algorithmic notation was used, the symbol $x \leftarrow y$ means that the current value of the variable x is replaced by the value y . Thus, $:=$ is used when a quantity is defined, while $\leftarrow$ is used to indicate an assignment or overwrite step inside the computational procedure. In other words, the temporary vector t is first formed as $B_{\Delta}^{\top}r$ and is then replaced by its scaled version Dt ; analogously, u is first formed as $S_{\Delta\Delta}t$ and is then replaced by Du . Therefore, the preconditioner fits the same matrix-free workflow as the operator $A_{FETI-DP}$ and requires no explicit global matrix assembly.

In this section, we will compare the spectra of the preconditioned iteration operator

$$M_{FETI-DP}^{-1}A_{FETI-DP},$$

or, equivalently, spectra of the generalized eigenvalue problem

$$A_{FETI-DP}x = \lambda M_{FETI-DP}x.$$

Finally, we note that in the actual Octave implementation both $A_{FETI-DP}$ and $M_{FETI-DP}^{-1}$ are available only through their action on vectors (operator-application and preconditioner-application routines), i.e. no explicit global matrices are formed. For this reason, the full spectral computations in this section are performed by constructing explicit matrix representations from these operator actions; the corresponding procedure is described in detail in the following section.

6.2.3 BDDC operator

In the BDDC method, the interface problem is treated entirely in the primal (assembled) framework: the Krylov iteration is performed directly on a single-valued assembled interface unknown $\hat{u} \in \widehat{W}$, and continuity is built into the iteration space itself. The interface problem therefore takes the form of the assembled (hat-space) system

$$A_{BDDC}\hat{u} = \hat{f},$$

where the assembled interface operator is defined from the same block-diagonal Schur operator as in the preceding sections. Using the assembly (distribution) mapping $R : \widehat{W} \rightarrow W$ and the product-space Schur operator $S = \text{diag}(S^{(1)}, \dots, S^{(N)})$ acting on W , the BDDC iteration operator is

$$A_{BDDC} := R^\top S R.$$

This operator is SPD on $\widehat{W}$ in the present setting, and it is applied in a matrix-free way: for a hat-space vector x , one PCG matvec evaluates

$$y = A_{BDDC}x = R^\top(S(Rx)),$$

i.e. one distributes by R , applies the local Schur complements independently through the block-diagonal operator S , and assembles the resulting product-space reactions by $R^\top$.

Analogously to the FETI-DP case, we also compute the spectrum of the non-preconditioned BDDC operator

$$A_{BDDC}.$$

This spectrum describes the conditioning of the raw assembled interface formulation before the action of the BDDC preconditioner. It is useful as a diagnostic counterpart to the raw FETI-DP multiplier-space spectrum, but it is not the spectrum of the operator actually iterated on by PCG.

To accelerate PCG, we used the BDDC preconditioner in a symmetric two-level form. In the

baseline variant used in this thesis, the preconditioner is applied as

$$M_{\text{BDDC}}^{-1} := R_D^T (T_{\text{sub}} + T_0) R_D,$$

where R_D denotes a scaled distribution (built from the multiplicity scaling on the product interface space). The operator T_{sub} is the block-local Δ -correction, consisting of independent subdomain solves with the local $\Delta\Delta$ blocks $S_{\Delta\Delta}^{(i)}$, and T_0 is the coarse correction induced by the primal set Γ_c . Both the local and coarse solves are realized by repeatedly applying precomputed factorizations, so that the preconditioner fits naturally into the matrix-free workflow of the PCG iteration.

Therefore, for the BDDC method, we will compare the spectra of the preconditioned iteration operator

$$M_{\text{BDDC}}^{-1} A_{\text{BDDC}},$$

or, equivalently, spectra of the generalized eigenvalue problem

$$A_{\text{BDDC}} x = \lambda M_{\text{BDDC}} x.$$

Finally, as in the FETI-DP case, we emphasize that in the actual Octave implementation neither A_{BDDC} nor M_{BDDC}^{-1} is assembled as a global matrix; both are available only through their action on vectors (operator-application and preconditioner-application routines). For this reason, the full spectral computations in this section are performed by constructing explicit matrix representations from these operator actions; the corresponding procedure is described in detail in the following section.

6.3 Spectral analysis of FETI-DP and BDDC operators – Octave implementation

In sections 6.2.2 and 6.2.3 we have formulated, in an implementation-consistent way, the two SPD operators whose spectra we want to compare, namely the preconditioned multiplier-space operator $M_{\text{FETI-DP}}^{-1} A_{\text{FETI-DP}}$ in the FETI-DP case and the preconditioned assembled (hat-space) operator $M_{\text{BDDC}}^{-1} A_{\text{BDDC}}$ in the BDDC case. At the same time, both formulations were implemented in a matrix-free way: neither the interface operators $A_{\text{FETI-DP}}$, A_{BDDC} nor the preconditioners $M_{\text{FETI-DP}}^{-1}$, M_{BDDC}^{-1} were assembled as global matrices, but are available only through their action on vectors in the PCG loop. The purpose of the present section is therefore to describe a compact auxiliary implementation layer that makes the full spectral computations possible in this matrix-free setting. For sufficiently small test instances, explicit matrix representations of the operator actions are constructed directly from repeated calls to the corresponding operator-application and preconditioner-application routines, and the full spectra are then obtained by a symmetric eigenvalue computation that is numerically stable and compatible with the SPD structure of the problem.

In addition to the PCG-relevant spectra of the preconditioned operators, the same analysis layer is also used to compute spectra of the corresponding non-preconditioned operators

$$A_{\text{FETI-DP}}, \quad A_{\text{BDDC}}.$$

These additional spectra are not used to explain the PCG iteration counts directly, since the PCG method is applied to the preconditioned systems. They are included as diagnostic information about the raw reduced interface formulations before the action of the corresponding preconditioners.

The implementation described in this section is deliberately minimal: it does not introduce any new solver components, but only reuses the matrix-free operator and preconditioner actions that are already present in the PCG loops of Chapter 5. In particular, the spectral routines take as their only “inputs” the dimension of the corresponding iteration space and two callable actions (matvec and preconditioner apply). For FETI-DP, these actions are the already implemented routines `applyA_lambda.m` and `applyM_lambda.m` that are passed as callbacks to the common PCG wrapper `pcg_wrap.m` inside `solve_fetidp.m`. For BDDC, the same role is played by `applyA_hat.m` and `applyM_bddc.m`, which realize the hat-space matvec and the two-level BDDC preconditioner and are used analogously by `solve_bddc.m`. The purpose of the present section is therefore only to describe a small “analysis layer” built on top of these callbacks: for sufficiently small test instances, it constructs explicit matrix representations of the two actions and then performs a full eigenvalue computation.

For the non-preconditioned spectra, the same idea is used in a simpler form. Only the operator-application callback is probed: in the FETI-DP case and in the BDDC case these actions are implemented by `applyA_lambda.m` and `applyA_hat.m` respectively. Thus, the raw FETI-DP matrix represents the mapping $x \mapsto A_{\text{FETI-DP}}x$ in the multiplier space, while the raw BDDC matrix represents the mapping $x \mapsto A_{\text{BDDC}}x$ in the assembled interface space. No preconditioner action is used in this auxiliary computation.

The central technical step in the spectral analysis layer is the explicit construction of operator matrices from the already available matrix-free callbacks. This is implemented by a small helper routine `assemble_from_apply.m`, which takes as input a function handle representing a linear action $T(\cdot)$ on $\mathbb{R}^n$ and the corresponding dimension n , and assembles the matrix representation column-by-column by evaluating the action on the canonical basis vectors e_j , $j = 1, \dots, n$. Concretely, the j -th column is obtained as

$$T_{\text{mat}}(:, j) = T(e_j), \quad j = 1, \dots, n.$$

In the present setting, this procedure is applied twice for each method and each test case: first to assemble the explicit operator matrix A from the matvec callback, and second to assemble the explicit preconditioner M^{-1} from the preconditioner-application callback. Since the cost and memory

of this assembly scale is $O(n^2)$, the implementation enforces a prescribed maximal dimension $n_{\max}$ of the PCG iteration space; if $n > n_{\max}$, the case is still solved by PCG and its convergence history is recorded, but the full matrix assembly and full spectrum computation are skipped.

The full spectrum is computed by the routine `full_spectrum_precond.m`, which takes the explicitly assembled matrices A and M^{-1} (obtained as described above) and converts the eigenvalue computation into a symmetric form that is numerically robust. Instead of forming the product $M^{-1}A$ directly, the implementation first computes a Cholesky factorization of the assembled preconditioner action

$$M^{-1} = R^T R,$$

and then forms the symmetric matrix

$$K = R A R^T.$$

The eigenvalues of the preconditioned operator are then obtained by a standard dense eigendecomposition

$$\{\lambda_i\}_{i=1}^n = \text{eig}(K),$$

and stored in sorted order together with the derived indicators $\lambda_{\min}$, $\lambda_{\max}$ and $\kappa = \lambda_{\max}/\lambda_{\min}$. In practice, this symmetric transformation serves two purposes: it preserves the intended SPD structure in the finite precision setting, and it provides a natural consistency check, since the Cholesky factorization exists only if the assembled M^{-1} is positive definite. For completeness, the implementation also records basic symmetry diagnostics of the assembled matrices (and, if requested, applies a simple symmetrization step) so that the reported spectra correspond to the symmetric operator that the PCG theory assumes.

The spectra of the non-preconditioned operators are computed by `full_spectrum_operator.m` (as the auxiliary routine). This routine uses the same matrix construction procedure based on `assemble_from_apply.m`, but it assembles only the raw operator matrix A . Since the operators considered here are symmetric in exact arithmetic, the assembled matrix is symmetrized in the same diagnostic spirit as above, and its eigenvalues are then computed directly by a dense eigendecomposition. The routine stores the sorted eigenvalue list together with the corresponding values $\lambda_{\min}$, $\lambda_{\max}$, and $\kappa = \lambda_{\max}/\lambda_{\min}$. In this way, the implementation produces raw spectral indicators that are directly comparable with the preconditioned spectral indicators reported later, although they have a different interpretation.

The execution of a single numerical test case is encapsulated in the wrapper `ch6_run_case.m`. For a prescribed configuration (mesh size and subdomain partition), the routine first constructs the model problem and initializes the method-specific data structures using the already existing setup functions. It then runs the sequential PCG solve for FETI-DP and BDDC (in this order), recording the standard convergence diagnostics returned by `pcg_wrap.m`, in particular the iteration count, the

final relative residual, and the residual history vector. Based on the corresponding iteration-space dimensions n_λ and $n_{\hat{u}}$, the wrapper decides whether the full spectral computation is feasible; if the dimension exceeds the prescribed threshold $n_{\max}$, the case is still solved and logged, but the explicit assembly and dense eigendecomposition are skipped. Otherwise, the wrapper constructs the explicit matrices from the callbacks using `assemble_from_apply.m`, calls `full_spectrum_precond.m` to obtain the full eigenvalue list and derived indicators, and returns all collected information in a single structured record `out` that is later processed by the experiment module.

In the extended version of the experiment wrapper, the output structure therefore contains two spectral records for each method. The field `spec` stores the spectrum of the preconditioned operator seen by PCG, while the field `spec_A` stores the spectrum of the corresponding non-preconditioned operator. Thus, for FETI-DP the record `out.fetidp.spec` corresponds to $M_{\text{FETI-DP}}^{-1}A_{\text{FETI-DP}}$, whereas `out.fetidp.spec_A` corresponds to $A_{\text{FETI-DP}}$. Analogously, `out.bddc.spec` corresponds to $M_{\text{BDDC}}^{-1}A_{\text{BDDC}}$, while `out.bddc.spec_A` corresponds to A_{BDDC} .

The complete set of experiments is executed by the driver `run_ch6_spectral_experiments.m`, which simply loops over a predefined list of configurations returned by `ch6_define_cases.m`. For each case, the driver calls `ch6_run_case.m`, stores the returned record `out` to a per-case MAT-file for reproducibility, and exports the basic visualizations used later in section 6.4.1: a sorted eigenvalue overlay of FETI-DP versus BDDC, a histogram overlay of the eigenvalue distribution, and an overlay of the PCG residual histories. Finally, after all cases are processed, a compact cross-case summary is generated by `export_ch6_table_tex.m`, which writes a ready-to-`\input{\}` table containing the main spectral indicators ($\lambda_{\min}$, $\lambda_{\max}$, κ) and PCG diagnostics (iterations and final relative residual) for both methods, while respecting the fixed presentation order (FETI-DP first, BDDC second).

The implementation described above produces, for each prescribed test case, all data required for the spectral comparison of the FETI-DP and BDDC operators as well as the corresponding PCG convergence diagnostics. The resulting spectra, plots and summary tables are presented and discussed in section 6.4.1.

The same experiment driver also exports the additional data associated with the non-preconditioned spectra. For each case, it can produce sorted-spectrum and histogram overlays based on the eigenvalue lists stored in `spec_A`. After all cases are processed, the routine `export_ch6_raw_table_tex.m` writes a separate LaTeX table containing the spectral indicators of $A_{\text{FETI-DP}}$ and A_{BDDC} . This separation keeps the interpretation clear: the table generated by `export_ch6_table_tex.m` contains the preconditioned spectra and PCG convergence diagnostics, while the raw table contains only the non-preconditioned spectral indicators.

6.4 Results and discussion of spectral comparison and PCG convergence

6.4.1 Experimental set and numerical results

This section presents the results of the sequential Octave experiments carried out for the model problem introduced in the previous sections, namely the Poisson equation on the unit square with homogeneous Dirichlet boundary conditions on two opposite sides and homogeneous Neumann boundary conditions on the remaining two sides. The finite element discretization, the substructuring framework, and the dual–primal solver constructions are exactly the same as in Chapters 4 and 5; in the present section, only the global mesh parameter and the subdomain partition are varied.

The computations in this section were performed on a laptop with an Intel(R) Core(TM) Ultra 9 185H processor running at 2.30 GHz and with 64.0 GB of installed RAM. The operating system was Windows 11 Pro, version 25H2, 64-bit. The sequential implementation was executed in GNU Octave 8.4.0. The Octave configuration reported the BLAS and LAPACK libraries as `-lblas` and `-llapack`, respectively. No parallel execution is considered in this section; parallel performance is discussed separately in the PETSc benchmark part of Section 6.5.

In all experiments in this section, the computational domain is

$$\Omega = [0, 1] \times [0, 1].$$

The load is chosen as the constant function

$$f(x, y) = 1.$$

The domain is discretized by a structured triangular P_1 finite element mesh obtained from a uniform $n \times n$ subdivision of the unit square. Thus, before the elimination of Dirichlet boundary nodes, the mesh contains

$$(n + 1)^2$$

nodes and

$$2n^2$$

triangular elements. The Dirichlet boundary is formed by the left and right sides of the square, so that $2(n + 1)$ nodal values are prescribed and eliminated from the discrete system. The remaining DOFs form the reduced finite element space used in all subsequent substructuring constructions.

The mesh is partitioned into a structured $n_{\text{sub},x} \times n_{\text{sub},y}$ array of rectangular subdomains aligned with the finite element grid. The parameter n is always chosen divisible by both $n_{\text{sub},x}$ and $n_{\text{sub},y}$, so that all subdomain interfaces coincide with mesh lines. Interface DOFs are identified after Dirichlet

elimination as those free DOFs that are shared by at least two subdomains. The primal DOFs are selected as the cross-points of the structured subdomain decomposition that remain among the free interface DOFs. In the present setting, because the left and right boundary nodes are Dirichlet nodes and are eliminated, the primal DOFs are the interface cross-points lying on internal vertical subdomain lines and on horizontal subdomain boundary lines. The remaining interface DOFs form the Δ part. In the notation used below, $n_{\widehat{W}}$ denotes the number of assembled interface DOFs, n_c denotes the number of primal DOFs, and n_{Δ} denotes the number of assembled Δ -interface DOFs.

Case	Local cells per subdomain	Nodes	Triangles	Dirichlet DOFs	Free DOFs	Interior DOFs
$(16, 2 \times 2)$	8×8	289	512	34	255	224
$(24, 2 \times 2)$	12×12	625	1152	50	575	528
$(32, 2 \times 2)$	16×16	1089	2048	66	1023	960
$(48, 2 \times 2)$	24×24	2401	4608	98	2303	2208
$(32, 4 \times 4)$	8×8	1089	2048	66	1023	840
$(48, 4 \times 4)$	12×12	2401	4608	98	2303	2024
$(64, 8 \times 8)$	8×8	4225	8192	130	4095	3248

Table 6.1: Mesh and reduced finite element dimensions used in the sequential Octave experiments.

For all cases, both methods were solved by PCG applied to the corresponding preconditioned operator. The initial vector was the zero vector. The stopping criterion was based on the relative residual,

$$\frac{\|r_k\|}{\|r_0\|} \leq 10^{-10},$$

and the maximum number of PCG iterations was set to 300. Consequently, the residual reported in Table 6.3 is the final relative residual returned by the PCG iteration. The experiment driver used the seed value equal to 1; however, with the constant load and the zero initial vector, the individual solves are deterministic.

For FETI-DP, the preconditioned operator relevant for the PCG iteration is the multiplier-space operator

$$M_{\text{FETI-DP}}^{-1} A_{\text{FETI-DP}},$$

where the preconditioner is the multiplicity-scaled lumped FETI-DP preconditioner described in Section 6.2. The jump operator used in the multiplier formulation is reduced to a full-row-rank form using the rank tolerance 10^{-12} . For BDDC, the corresponding PCG-relevant operator is the preconditioned assembled interface operator

$$M_{\text{BDDC}}^{-1} A_{\text{BDDC}}.$$

The BDDC preconditioner uses the same multiplicity scaling on the product interface space together with local Δ -block solves and the coarse correction generated by the primal cross-point DOFs.

In addition to these preconditioned operators, we also compute the spectra of the non-preconditioned reduced operators

$$A_{\text{FETI-DP}}, \quad A_{\text{BDDC}}.$$

These raw spectra are included as diagnostic information about the interface formulations before the action of the preconditioners. They are not used to explain the PCG iteration counts directly, since the PCG method is applied to the preconditioned systems.

For all cases considered in this section, the spectra were obtained by the explicit assembly procedure described in Section 6.3. The operator and preconditioner were first available only through their actions on vectors. For the purpose of spectral diagnostics, explicit matrix representations were constructed column by column by applying the corresponding matrix-free routines to the canonical basis vectors. For the preconditioned spectra, the matrices were then symmetrized and transformed to the symmetric eigenvalue problem by means of the Cholesky factorization of the assembled preconditioner action. For the non-preconditioned spectra, only the raw operator action was assembled and the resulting symmetric matrix was diagonalized directly.

The full spectral computation was enabled for iteration-space dimensions not exceeding

$$n_{\max} = 1200.$$

Since the largest spaces in the present experiments have dimensions $n_\lambda = 784$ for FETI-DP and $n_{\hat{u}} = 847$ for BDDC, the full spectra were computed for all cases reported in Tables 6.2 and 6.3.

The tested set consists of seven cases:

$$(n, \text{sub}) = (16, 2 \times 2), (24, 2 \times 2), (32, 2 \times 2), (48, 2 \times 2), (32, 4 \times 4), (48, 4 \times 4), (64, 8 \times 8).$$

This choice makes it possible to observe several effects within a single compact experimental set. At fixed 2×2 decomposition, it shows the influence of mesh refinement. At comparable global resolutions, it shows the effect of increasing the number of subdomains. Finally, the pair $(32, 4 \times 4)$ and $(64, 8 \times 8)$ provides a weak-scaling-like numerical comparison in which the local subdomain resolution is preserved, namely 8×8 cells per subdomain. In this setting, the classical theory predicts that the condition number of the preconditioned operators should remain bounded independently of the total number of subdomains. Although the present sequential Octave experiments do not constitute a parallel scaling study, they allow us to examine whether the measured condition numbers, and consequently also the PCG convergence behavior, remain of comparable quality when the total problem size increases through a larger number of subdomains.

For each case, the reported spectral quantities are the dimension of the corresponding iteration space, the extreme eigenvalues $\lambda_{\min}$ and $\lambda_{\max}$, and the condition number

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

Case	n_λ	$A_{\text{FETI-DP}}$				$n_{\hat{u}}$	A_{BDDC}			
		$\lambda_{\min}$	$\lambda_{\max}$	$\kappa_{\text{FETI-DP}}$			$\lambda_{\min}$	$\lambda_{\max}$	κ_{BDDC}	
16 2x2	28	7.263e-01	9.312e+00	1.282e+01		31	1.760e-01	5.882e+00	3.341e+01	
24 2x2	44	7.158e-01	1.531e+01	2.139e+01		47	1.167e-01	5.882e+00	5.040e+01	
32 2x2	60	7.120e-01	2.154e+01	3.026e+01		63	8.723e-02	5.882e+00	6.744e+01	
32 4x4	168	7.251e-01	1.706e+01	2.353e+01		183	4.414e-02	5.899e+00	1.336e+02	
48 2x2	92	7.093e-01	3.448e+01	4.861e+01		95	5.792e-02	5.882e+00	1.016e+02	
48 4x4	264	7.154e-01	3.042e+01	4.252e+01		279	2.902e-02	5.885e+00	2.028e+02	
64 8x8	784	7.248e-01	1.875e+01	2.588e+01		847	1.073e-02	5.904e+00	5.500e+02	

Table 6.2: Summary of spectral indicators for the non-preconditioned FETI-DP and BDDC operators.

Case	n_λ	FETI-DP						$n_{\hat{u}}$	BDDC					
		$\lambda_{\min}$	$\lambda_{\max}$	$\kappa_{\text{FETI-DP}}$	it	relres			$\lambda_{\min}$	$\lambda_{\max}$	κ_{BDDC}	it	relres	
16 2x2	28	1.000e+00	1.376e+00	1.376e+00	5	8.575e-11		31	1.000e+00	1.376e+00	1.376e+00	1	3.143e-15	
24 2x2	44	1.000e+00	1.511e+00	1.511e+00	6	1.182e-13		47	1.000e+00	1.511e+00	1.511e+00	1	4.955e-15	
32 2x2	60	1.000e+00	1.617e+00	1.617e+00	6	1.637e-12		63	1.000e+00	1.617e+00	1.617e+00	1	7.795e-15	
32 4x4	168	1.000e+00	2.257e+00	2.257e+00	7	3.500e-11		183	1.000e+00	2.257e+00	2.257e+00	8	3.016e-12	
48 2x2	92	1.000e+00	1.779e+00	1.779e+00	6	5.603e-11		95	1.000e+00	1.779e+00	1.779e+00	1	2.187e-14	
48 4x4	264	1.000e+00	2.683e+00	2.683e+00	8	7.575e-12		279	1.000e+00	2.683e+00	2.683e+00	9	8.422e-12	
64 8x8	784	1.000e+00	2.458e+00	2.458e+00	14	4.713e-11		847	1.000e+00	2.458e+00	2.458e+00	16	3.083e-11	

Table 6.3: Summary of spectral indicators and PCG convergence for the preconditioned FETI-DP and BDDC operators.

For the preconditioned operators, we additionally report the number of PCG iterations and the final relative residual. For representative cases, the full sorted spectrum, the histogram of eigenvalue distribution, and the normalized residual history are also shown. Because FETI-DP and BDDC iterate in different spaces, the dimensions n_λ and $n_{\hat{u}}$ need not coincide, and therefore the sorted spectral curves compared later need not have the same length.

We first report the spectra of the non-preconditioned reduced operators. These data describe the raw FETI-DP and BDDC interface formulations before the action of the preconditioners.

We then report the spectra of the corresponding preconditioned operators together with the PCG convergence data. These are the quantities directly related to the iterative solver behavior, since they describe the operators actually seen by PCG.

The comparison of Tables 6.2 and 6.3 separates two different questions. The non-preconditioned spectra show the conditioning of the raw reduced interface operators before preconditioning, whereas the preconditioned spectra show the operators used in the actual PCG iterations.

The spectra of the non-preconditioned operators are not expected to coincide. The reason is that the two raw operators represent different algebraic formulations of the interface problem. The FETI-DP operator $A_{\text{FETI-DP}}$ acts in the multiplier space, whereas the BDDC operator A_{BDDC} acts in the assembled interface space. Consequently, the unknowns have different meanings and the corresponding spaces generally have different dimensions. The two raw operators are therefore not simply two matrix representations of the same linear mapping in different bases, and there is no reason for their full spectra to be identical.

The theoretical spectral relation between FETI-DP and BDDC concerns the corresponding preconditioned iteration operators, not the raw reduced operators themselves. The preconditioners introduce the scaling, local subdomain corrections, and coarse primal correction that compensate for the different dual and primal formulations. Therefore, the spectra of

$$M_{\text{FETI-DP}}^{-1}A_{\text{FETI-DP}} \quad \text{and} \quad M_{\text{BDDC}}^{-1}A_{\text{BDDC}}$$

are the relevant spectra for the PCG convergence comparison. This distinction is important for the interpretation of the numerical results discussed below.

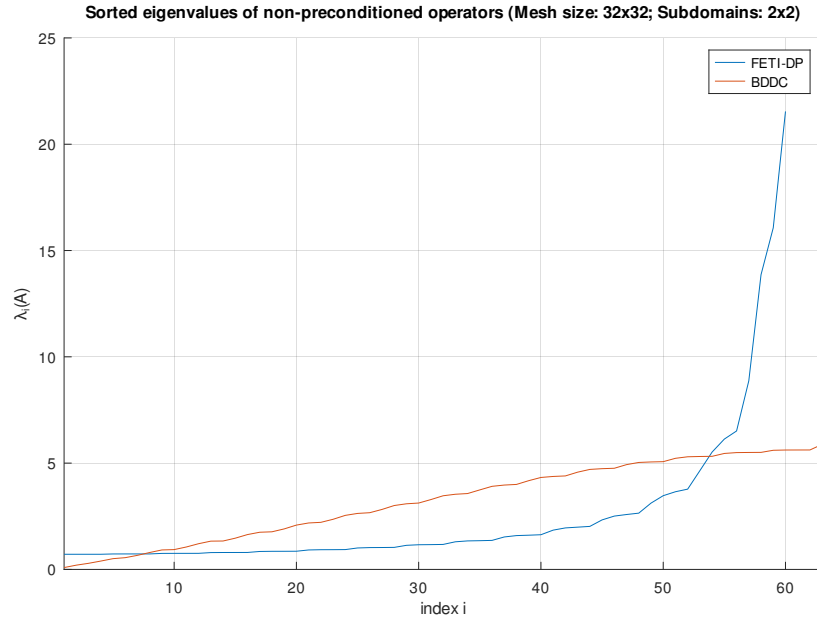
6.4.2 Comparison of spectral properties and convergence rates for representative cases

The complete set of numerical results is summarized in Tables 6.2 and 6.3, while in this section only selected representative cases are discussed in detail. The purpose of this subsection is to illustrate the main spectral phenomena visible in the figures. We first compare the non-preconditioned and preconditioned spectra for the same small representative case, and then consider two larger cases that show how the preconditioned spectral distribution and the PCG residual histories behave when the interface problem becomes larger.

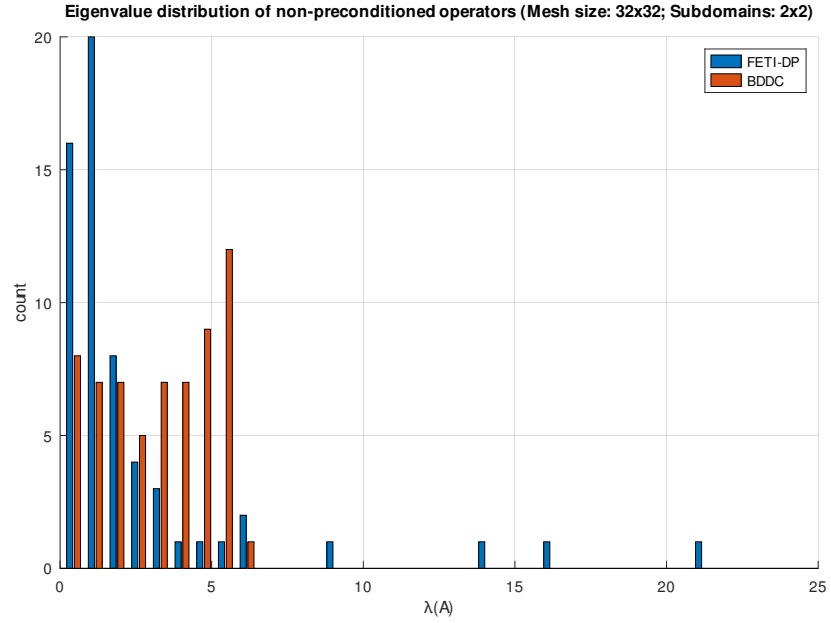
The distinction between the two tables is important for the interpretation of the figures. Table 6.2 describes the raw operators $A_{\text{FETI-DP}}$ and A_{BDDC} , which act in different spaces and are therefore not expected to have identical spectra. Table 6.3, on the other hand, describes the preconditioned operators actually used in the PCG iterations. Consequently, the representative non-preconditioned Figure below should be read as a diagnostic illustration of the raw formulations, while the preconditioned figures and residual histories are the quantities directly connected with solver convergence.

A first representative example is provided by the case $n = 32$ with the 2×2 decomposition. Figure 6.1 shows the spectra of the non-preconditioned operators for this case. The two distributions are visibly different. This is consistent with Table 6.2, where the raw FETI-DP operator has condition number approximately $3.026 \cdot 10^1$, while the raw BDDC operator has condition number approximately $6.744 \cdot 10^1$. The sorted plot shows that the FETI-DP spectrum contains a more pronounced upper tail, whereas the BDDC spectrum is bounded above by a smaller largest eigenvalue and starts closer to zero. The histogram therefore illustrates the main diagnostic message of the raw spectra: before preconditioning, the two formulations have different spectral distributions.

For the same mesh and subdomain decomposition, Figure 6.2 shows the spectra after applying the corresponding preconditioners. The contrast with Figure 6.1 is substantial. Instead of the wider raw spectral intervals, the preconditioned eigenvalues are strongly clustered near 1, and both methods have the same reported condition number, approximately 1.617. Thus, this pair of figures



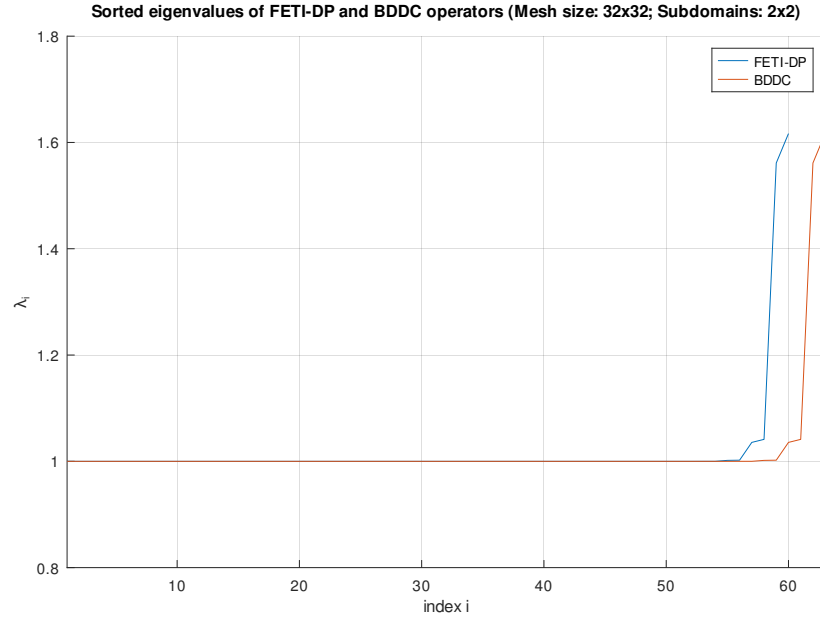
(a) Sorted spectra of the non-preconditioned operators for the case $n = 32$ with 2×2 subdomains



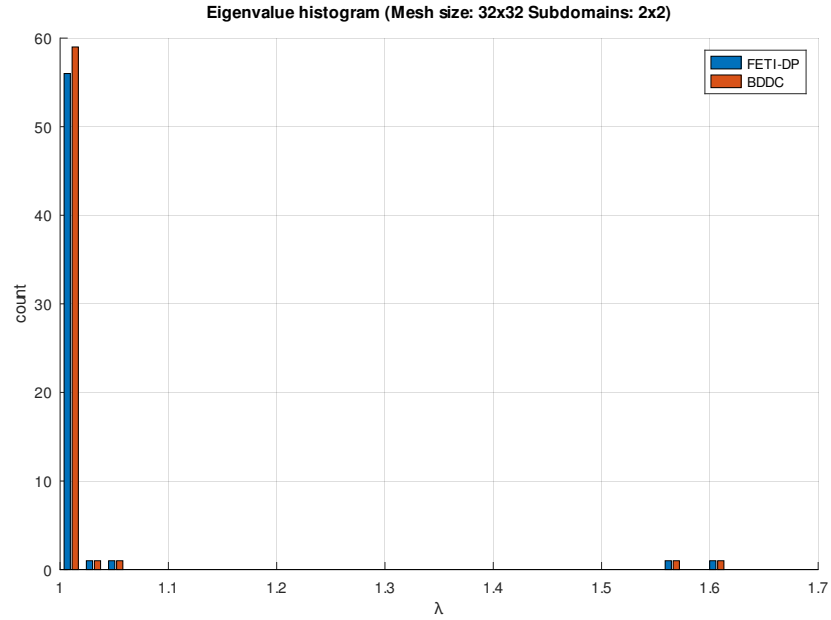
(b) Histogram overlay of the eigenvalue distributions of the non-preconditioned operators for the case $n = 32$ with 2×2 subdomains

Figure 6.1: Spectral comparison of the non-preconditioned FETI-DP and BDDC operators for the case $n = 32$ with 2×2 subdomains

provides a direct before-and-after comparison: the raw operators are spectrally different, while the preconditioned operators become closely comparable in the sense relevant for PCG convergence.



(a) Sorted spectra of the preconditioned iteration operators for the case $n = 32$ with 2×2 subdomains



(b) Histogram overlay of the eigenvalue distributions for the case $n = 32$ with 2×2 subdomains

Figure 6.2: Spectral comparison for the case $n = 32$ with 2×2 subdomains

The two plots in Figure 6.2 show that, after preconditioning, the situation is very different from the raw spectra in Figure 6.1. For both methods the eigenvalues are strongly concentrated near the value 1, with only a short upper tail. This is consistent with Table 6.3, where both methods have the same extremal eigenvalues $\lambda_{\min} = 1$ and $\lambda_{\max} \approx 1.617$, and therefore also the same condition number $\kappa \approx 1.617$. At the same time, the sorted curves do not have the same length, because FETI-DP iterates in the multiplier space associated with the reduced jump constraints, whereas BDDC iterates in the assembled interface space of single-valued traces.

In this small case, the PCG iteration counts in Table 6.3 are not identical: FETI-DP requires 6 iterations, while BDDC reaches the prescribed tolerance in one iteration. This does not contradict the close spectral relation of the preconditioned methods. The condition number describes only the extreme eigenvalues, whereas the actual PCG convergence also depends on the distribution of the initial residual among the corresponding eigenspaces. In this particular small assembled interface problem, the right-hand side and the initial residual evidently produce a very favorable situation for the BDDC iteration. The main conclusion from the spectral plots remains that the preconditioned spectra are compact and closely comparable, even though the raw spectra shown before are visibly different.

A second and more demanding representative case is the decomposition $n = 48$ with 4×4 subdomains. Figure 6.3 shows the sorted spectra of the preconditioned operators. Compared with the previous 2×2 example, the spectra are visibly broader, which reflects the more difficult interface problem associated with the finer subdomain partition. This broadening is consistent with Table 6.3: both methods now reach $\lambda_{\max} \approx 2.683$, and hence $\kappa \approx 2.683$. Nevertheless, the spectrum remains compact and no isolated large outliers appear. The figure therefore shows a moderate widening of the preconditioned spectral interval, but not a qualitative deterioration of the preconditioned operators.

The practical effect of this spectral behavior is visible in the corresponding residual histories shown in Figure 6.4. In both cases the normalized residual decreases rapidly, and the prescribed tolerance is reached after only a small number of PCG iterations, namely 8 steps for FETI-DP and 9 steps for BDDC. Thus, even though increasing the number of subdomains produces a wider preconditioned spectral interval than in the 2×2 case, the convergence remains fast for both methods. This illustrates the main point emphasized in Section 6.1: favorable PCG behavior is connected not only with a small condition number, but also with the overall distribution of eigenvalues.

The largest tested sequential case, $n = 64$ with the 8×8 decomposition, is most conveniently viewed through the histogram of the preconditioned eigenvalues and the corresponding residual histories in Figure 6.5. In this case, the iteration-space dimensions are already substantially larger, namely $n_\lambda = 784$ for FETI-DP and $n_u = 847$ for BDDC. Nevertheless, the reported preconditioned spectral indicators remain in a favorable regime. Table 6.3 gives $\kappa \approx 2.458$ for both methods, with 14 PCG iterations for FETI-DP and 16 for BDDC.

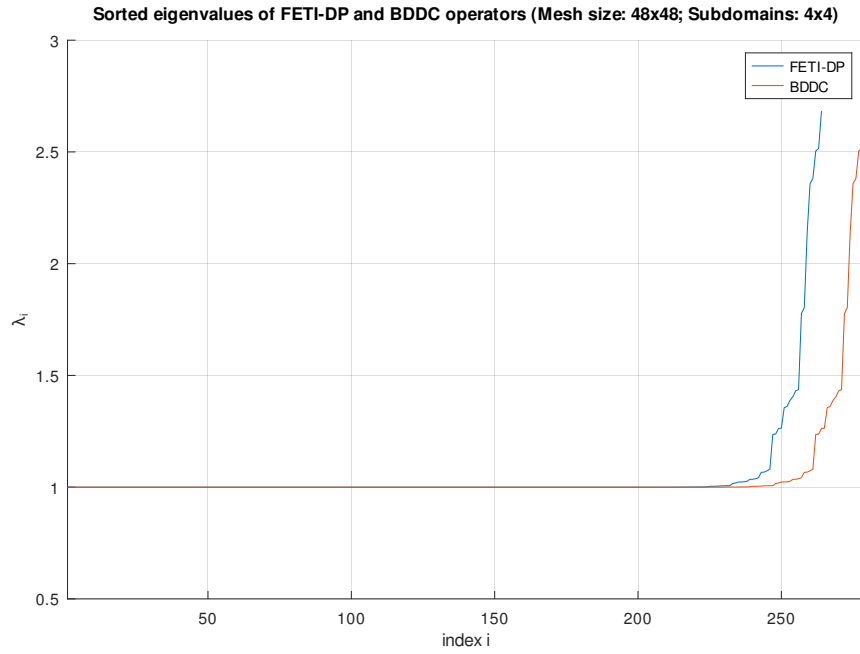


Figure 6.3: Sorted spectra of the preconditioned iteration operators for the case $n = 48$ with 4×4 subdomains

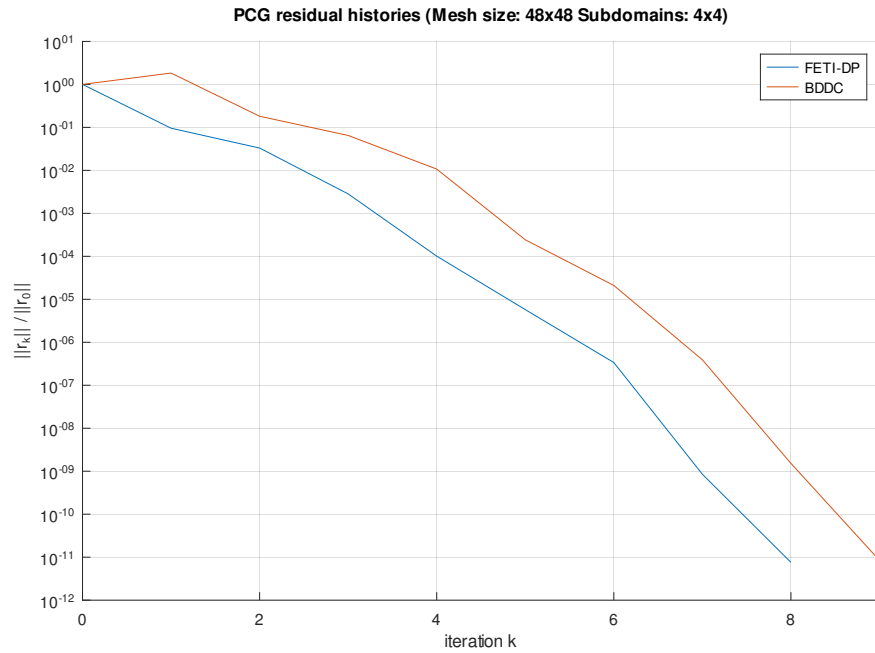
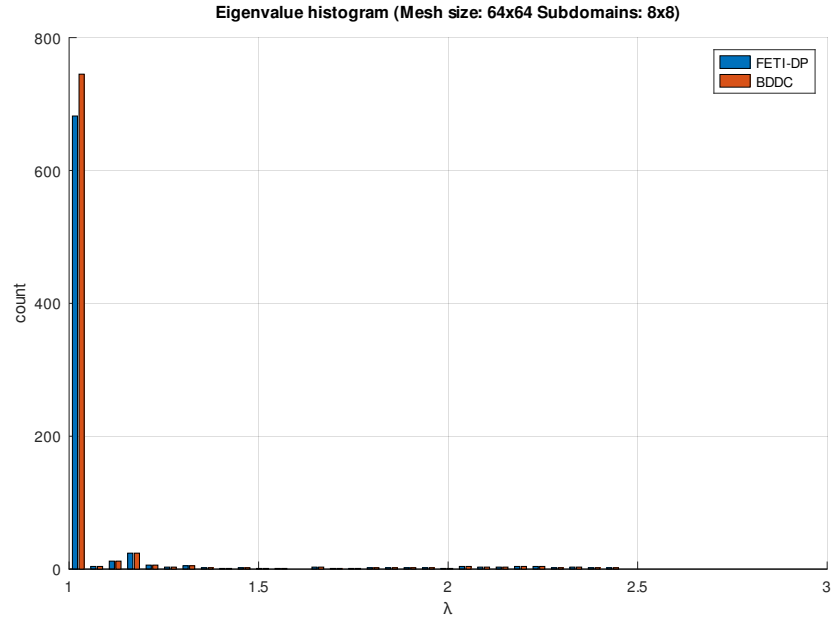
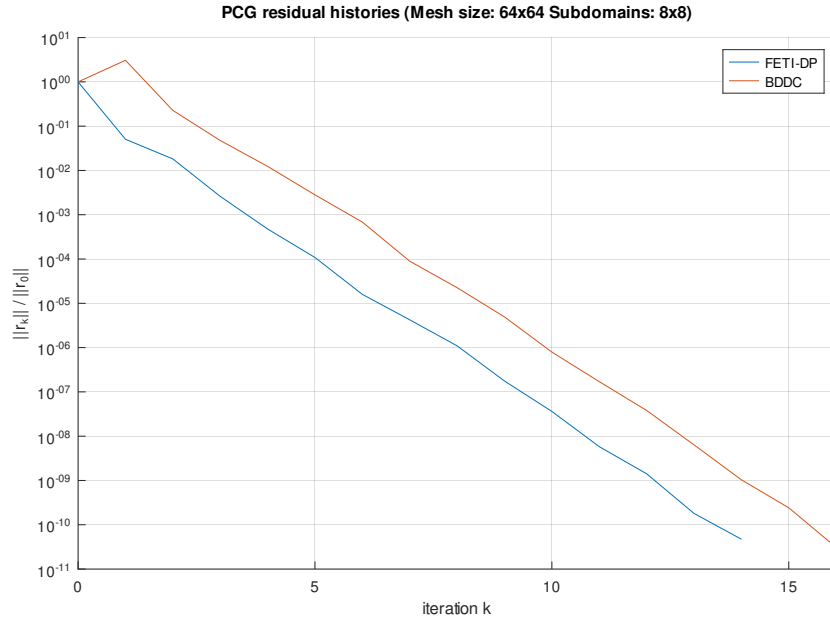


Figure 6.4: Normalized PCG residual histories for the case $n = 48$ with 4×4 subdomains



(a) Histogram overlay of the eigenvalue distributions for the case $n = 64$ with 8×8 subdomains



(b) Normalized PCG residual histories for the case $n = 64$ with 8×8 subdomains

Figure 6.5: Spectral distribution and PCG convergence for the case $n = 64$ with 8×8 subdomains

The histogram is useful here because it shows that even for the largest tested configuration the eigenvalues remain concentrated in a relatively narrow interval and are strongly clustered near 1. Thus, this case does not reveal any qualitative breakdown of the preconditioners; instead, it preserves the same overall pattern already seen in the smaller examples. The residual histories confirm this conclusion on the level of the actual PCG process, since both methods still converge rapidly despite the larger interface dimensions.

Taken together, the representative figures confirm the distinction suggested by Tables 6.2 and 6.3. The non-preconditioned spectra in Figure 6.1 illustrate that the raw FETI-DP and BDDC interface formulations are not spectrally identical before preconditioning. This is expected, since the corresponding operators act in different spaces. In contrast, Figures 6.2–6.5 show that the preconditioned operators have compact spectral distributions and lead to rapid PCG convergence. The remaining visible differences are naturally connected with the different iteration spaces, different dimensions, and different multiplicities of the eigenvalue $\lambda = 1$. A more systematic interpretation of these trends, including their relation to mesh refinement, increasing number of subdomains, and weak-scaling-like behavior, is given in the following section.

6.4.3 Discussion of observed trends and numerical scalability signatures

The representative cases in the previous subsection illustrate the main spectral phenomena visually. We now interpret the complete data from Tables 6.2 and 6.3 from the viewpoint of observed trends and numerical scalability. The distinction between the two tables is essential. Table 6.2 describes the spectra of the raw reduced interface operators, whereas Table 6.3 describes the spectra of the preconditioned operators that are actually used in the PCG iterations.

The non-preconditioned spectra should not be expected to satisfy the same estimates as the preconditioned spectra. For Schur-complement-type interface operators arising from second-order elliptic problems, the unpreconditioned condition number grows algebraically with the ratio between the subdomain size and the mesh size; in the usual notation this behavior is of order

$$\kappa(S) = \mathcal{O}(H/h),$$

up to constants depending on the precise setting and geometry [18]. This is fundamentally different from the polylogarithmic estimates associated with scalable dual–primal preconditioners. In addition, the raw FETI-DP and BDDC operators are not two matrix representations of the same operator. The operator $A_{\text{FETI-DP}}$ acts in the multiplier space, while A_{BDDC} acts in the assembled interface space. Therefore, their raw condition numbers may differ substantially and should be interpreted only as diagnostic information about the unpreconditioned interface formulations.

This distinction is visible already in the fixed 2×2 decomposition sequence. When the mesh is refined from $n = 16$ to $n = 48$, the raw FETI-DP condition number increases from approximately $1.282 \cdot 10^1$ to $4.861 \cdot 10^1$, while the raw BDDC condition number increases from approximately

$3.341 \cdot 10^1$ to $1.016 \cdot 10^2$. This behavior is consistent with the fact that no preconditioner has yet been applied: the raw interface problems become more ill-conditioned under mesh refinement. At the same time, the two methods deteriorate in different ways. In the raw FETI-DP case, the smallest eigenvalue remains close to $7 \cdot 10^{-1}$, and the growth of the condition number is mainly caused by the increasing largest eigenvalue. In the raw BDDC case, the largest eigenvalue remains close to 5.9, while the smallest eigenvalue decreases as the problem becomes larger. This again confirms that the two raw spectra are not expected to coincide.

The effect of increasing the number of subdomains is also much more visible before preconditioning. For example, at $n = 32$, changing the decomposition from 2×2 to 4×4 changes the raw FETI-DP condition number from approximately $3.026 \cdot 10^1$ to $2.353 \cdot 10^1$, while the raw BDDC condition number increases from approximately $6.744 \cdot 10^1$ to $1.336 \cdot 10^2$. At $n = 48$, the corresponding raw BDDC condition number increases from approximately $1.016 \cdot 10^2$ to $2.028 \cdot 10^2$. The largest raw value in the table is obtained for BDDC in the case $n = 64$ with 8×8 subdomains, where $\kappa(A_{\text{BDDC}}) \approx 5.500 \cdot 10^2$.

The situation changes substantially after preconditioning. For the preconditioned dual-primal operators, the relevant theoretical expectation is a bound of the form

$$\kappa \leq C(1 + \log(H/h))^2,$$

where the constant C is independent of the number of subdomains and of the mesh size, provided that the primal constraints and scaling are chosen appropriately [10, 2, 1, 11]. This estimate applies to the preconditioned iteration operators

$$M_{\text{FETI-DP}}^{-1} A_{\text{FETI-DP}}, \quad M_{\text{BDDC}}^{-1} A_{\text{BDDC}},$$

and not to $A_{\text{FETI-DP}}$ and A_{BDDC} themselves. It is precisely this distinction which explains why Table 6.2 contains relatively large and method-dependent condition numbers, whereas Table 6.3 contains small and identical condition numbers for the two methods.

In the fixed 2×2 refinement sequence, the preconditioned condition number grows only mildly, from approximately 1.376 for $n = 16$ to approximately 1.779 for $n = 48$. This should be compared with the much stronger growth of the corresponding non-preconditioned condition numbers. The PCG iteration counts remain small throughout this sequence: FETI-DP requires between 5 and 6 iterations, while BDDC reaches the prescribed tolerance in one iteration in all tested 2×2 cases. As already discussed in the previous subsection, the exceptional one-step convergence of BDDC in these small cases should be understood as a feature of the particular right-hand side and eigenspace distribution, not as a separate asymptotic conclusion.

A second trend is obtained by increasing the number of subdomains at comparable global mesh sizes. In the preconditioned results, the condition number increases from approximately 1.617 to

2.257 when comparing $(32, 2 \times 2)$ with $(32, 4 \times 4)$, and from approximately 1.779 to 2.683 when comparing $(48, 2 \times 2)$ with $(48, 4 \times 4)$. Thus, a finer subdomain partition does lead to a more demanding interface problem, but the deterioration remains moderate after preconditioning. The contrast with the raw BDDC condition numbers, which increase much more strongly in the same comparisons, illustrates the stabilizing role of the dual–primal preconditioners.

The weak-scaling-like comparison is represented by the pair $(32, 4 \times 4)$ and $(64, 8 \times 8)$. In this comparison, the local number of cells per subdomain is preserved, and therefore the local ratio H/h remains approximately constant. The word “weak-scaling-like” is used here only in the numerical sense: the local problem size per subdomain is kept comparable while the number of subdomains and the global problem size increase. It is not a parallel weak-scaling experiment, since the Octave implementation presented and tested in this thesis is sequential and no processor count is varied.

For the preconditioned operators, this weak-scaling-like comparison shows the expected numerical signature. The condition number changes only from approximately 2.257 to 2.458, and both methods remain in the same favorable convergence regime. The PCG iteration counts increase from 7 to 14 for FETI-DP and from 8 to 16 for BDDC. This increase is natural because the iteration spaces are substantially larger, but it remains moderate and does not indicate a qualitative loss of robustness. In contrast, the raw BDDC condition number changes from approximately $1.336 \cdot 10^2$ to $5.500 \cdot 10^2$, showing again that the non-preconditioned operators do not display the same scalability signature as the preconditioned ones.

Taken together, the sequential Octave experiments support two complementary conclusions. First, the non-preconditioned spectra show that the raw FETI-DP and BDDC interface formulations may have substantially different and considerably less favorable conditioning. This is consistent with the algebraic conditioning expected for unpreconditioned interface problems and with the fact that the two raw operators act in different spaces. Second, after applying the corresponding dual–primal preconditioners, the spectral indicators become small, the reported condition numbers are identical for the two methods in all tested cases, and the PCG convergence remains fast. Therefore, the numerical scalability conclusions of this section concern the preconditioned operators and the corresponding PCG iterations. The raw spectra are useful diagnostic information, but they are not the quantities that determine the convergence behavior of the implemented solvers.

Consequently, the sequential experiments provide numerical evidence of robustness and numerical scalability signatures, but not a study of parallel scalability. They show that the preconditioned FETI-DP and BDDC operators remain well conditioned on the tested model problems and that their behavior is consistent with the standard dual–primal theory. The question of actual parallel efficiency, including wall-clock times, speedup, and implementation overhead, is treated separately in the PETSc benchmark comparison.

6.5 Parallel PETSc benchmark comparison of FETI-DP and BDDC

The results of the previous section showed that, within the sequential Octave framework developed in Chapters 4 and 5, the implemented FETI-DP and BDDC solvers produce well-conditioned preconditioned operators, compact spectra, and correspondingly fast convergence of the PCG method. These results provide evidence of numerical robustness in the sequential setting, but they do not constitute a benchmark of official parallel implementations. We therefore conclude this chapter by a compact PETSc benchmark comparison of FETI-DP and BDDC.

The purpose of the PETSc part is different from the purpose of the Octave implementation. The Octave code developed in this thesis is used to expose and understand the algebraic structure of the methods, to assemble diagnostic spectra, and to compare the resulting operators in a controlled sequential environment. The PETSc part, on the other hand, uses an official PETSc tutorial example in order to compare the available parallel implementations of the same solver families. The PETSc experiments are therefore not intended as a direct timing comparison with the Octave code and they are not a reimplement of the same custom discretization pipeline. Their role is to complement the previous spectral study by iteration counts, residual histories, spectral indicators reported by PETSc, and representative timing data from an established parallel solver library.

The PETSc benchmark is carried out using PETSc built from source from the official PETSc GitLab repository. The version used in the computations was PETSc 3.24.5, with the Git revision v3.24.5-8-g14db6046566. The MPI implementation and BLAS/LAPACK libraries used by this PETSc build were downloaded and configured as part of the PETSc build process. The computations were performed under Ubuntu 24.04.3 LTS running in WSL2 on the same workstation as the sequential experiments, namely a laptop with an Intel(R) Core(TM) Ultra 9 185H processor and 64 GB of RAM.

6.5.1 PETSc experimental set and benchmark configuration

In the present section, the sequential viewpoint of the previous experiments is complemented by a compact benchmark study carried out in the PETSc environment. PETSc is used here as a software framework providing official parallel implementations of Krylov subspace methods and domain decomposition preconditioners. From the mathematical point of view, the same dual-primal ideas remain relevant as in Chapters 4 and 5: FETI-DP is understood as the dual multiplier-space formulation, whereas BDDC is understood as the primal assembled-space formulation. The purpose of this subsection is therefore to specify the actual PETSc benchmark setting, the relation of the PETSc example to the model problem used earlier in the thesis, and the precise meaning of the benchmark labels used in the numerical tables.

In PETSc, the BDDC method is represented in this benchmark by the solver branch with the options prefix `physical_`. This branch acts on the original discretized problem in the physical degrees

of freedom and uses PCBDDC as the domain decomposition preconditioner. The FETI-DP method is represented by the solver branch with the options prefix `fluxes_`. In the PETSc implementation used here, this branch is created as `KSPFETIDP`; PETSc then constructs the corresponding FETI-DP system automatically by means of an internal PCBDDC object. Consequently, quantities later denoted by `physical_ksp_*` refer to the BDDC branch, while quantities denoted by `fluxes_ksp_*` refer to the FETI-DP branch. This also explains why the two PETSc implementations are closely related on the software level: the FETI-DP branch is generated within the same dual-primal infrastructure rather than being a completely unrelated code path [4, 5].

The model problem used in Chapters 2–5 was the Poisson equation on the unit square with homogeneous mixed Dirichlet–Neumann boundary conditions. It was discretized in the Octave implementation by piecewise linear finite elements on a structured triangular mesh and partitioned into rectangular subdomains aligned with the grid. In contrast, the PETSc part uses the official tutorial example `src/ksp/ksp/tutorials/ex59.c`. This example solves a discrete Laplacian problem using spectral elements and illustrates the use of PCBDDC and `KSPFETIDP`. In the present thesis only the two-dimensional version of this example is used. The subdomain decomposition is prescribed by the runtime parameters `-npx` and `-npy`, the global element counts by `-nex` and `-ney`, and the spectral polynomial degree by `-p`. The pure Neumann variant of the PETSc example was not used; instead the default setting with Dirichlet boundary conditions on one side of the domain was used [3]. This means that the PETSc benchmark problem is not identical to the custom model problem used in the sequential Octave part of the thesis. This difference is not introduced because a different model would be preferable in itself, but rather because the thesis aims to compare official parallel PETSc implementations on a controlled benchmark example that is actually available within the library. Nevertheless, the comparison remains meaningful for the purposes of this thesis, because both parts concern scalar second-order elliptic problems on structured non-overlapping decompositions, and both use the same dual-primal domain decomposition principles: local subdomain problems, interface coupling, a coarse continuity mechanism, and iterative solution of the resulting preconditioned systems.

The PETSc benchmark uses a fixed spectral degree

$$p = 2$$

in all cases, i.e. the PETSc parameters `-nex` and `-ney` prescribe the number of high-order finite elements, in the two coordinate directions. Each such element carries a polynomial approximation of degree p , and therefore contributes $p + 1$ local interpolation points in one coordinate direction. Therefore, the global spectral grid associated with the parameters `-nex` and `-ney` has

$$(pn_{\text{ex}} + 1) \times (pn_{\text{ey}} + 1)$$

grid points. In the default Dirichlet setting used here, the nodes on one side of the domain are not iterated on. Therefore, the number of active physical degrees of freedom in the BDDC branch is

$$(p n_{\text{ex}} + 1)(p n_{\text{ey}} + 1) - (p n_{\text{ey}} + 1).$$

These counts describe the physical problem solved by the `physical_` branch. The FETI-DP branch acts in a different, internally generated multiplier space; its dimension is therefore not identical to the number of physical degrees of freedom and is reported separately in the PETSc output.

The tested benchmark set consists of nine cases obtained by combining three subdomain layouts,

$$2 \times 2, \quad 4 \times 2, \quad 4 \times 4,$$

with three local element resolutions per subdomain edge,

$$S = 16, \quad M = 32, \quad L = 48.$$

The resulting case labels are denoted by $A-S, \dots, C-L$, where the letter identifies the subdomain layout and the suffix identifies the local resolution. The full correspondence between labels and PETSc runtime parameters is shown in Table 6.4.

Label	Layout	MPI ranks	-npx	-npy	Local level	-nex	-ney	Global spectral grid	Active physical DOFs
$A-S$	2×2	4	2	2	$S = 16$	32	32	65×65	4160
$A-M$	2×2	4	2	2	$M = 32$	64	64	129×129	16512
$A-L$	2×2	4	2	2	$L = 48$	96	96	193×193	37056
$B-S$	4×2	8	4	2	$S = 16$	64	32	129×65	8320
$B-M$	4×2	8	4	2	$M = 32$	128	64	257×129	33024
$B-L$	4×2	8	4	2	$L = 48$	192	96	385×193	74112
$C-S$	4×4	16	4	4	$S = 16$	64	64	129×129	16512
$C-M$	4×4	16	4	4	$M = 32$	128	128	257×257	65792
$C-L$	4×4	16	4	4	$L = 48$	192	192	385×385	147840

Table 6.4: PETSc benchmark design and meaning of the case labels. The active physical DOF count refers to the `physical_` BDDC branch of the PETSc example.

This benchmark set was chosen to keep the PETSc part compact while still showing several relevant numerical effects. For a fixed subdomain layout, increasing the suffix from S to M and L increases the local element resolution. Comparing cases with comparable global sizes but different layouts shows the effect of changing the number of subdomains and MPI ranks. Finally, cases with the same local level but different layouts provide a weak-scaling-like numerical comparison, because the local number of elements per subdomain edge is preserved while the global problem size and the number of MPI ranks increase. Since all computations were performed on one workstation and only a compact set of cases was used, this should not be interpreted as a full parallel weak-scaling study in the strict performance-engineering sense.

In the final benchmark configuration, the stopping parameters were set explicitly and identically for both PETSc solver branches. The relative tolerance was prescribed as

$$\text{physical_ksp_rtol} = \text{fluxes_ksp_rtol} = 10^{-10},$$

with absolute tolerance 10^{-50} and maximum number of Krylov iterations equal to 300. The value 10^{-10} was chosen in order to impose a strict convergence requirement of the same numerical order as in the sequential experiments of Section 6.4.1. This should not, however, be interpreted as an identical stopping rule for the plotted residual curves. The sequential Octave experiments and the PETSc benchmark use different solver implementations and different residual quantities.

More precisely, PETSc applies its convergence test to the residual norm selected internally by the corresponding KSP solver. If this norm is denoted by ρ_k , the relative tolerance controls the required relative decrease of this KSP convergence norm. With the default PETSc convergence test, the iteration is stopped when this selected norm satisfies a condition of the form

$$\rho_k \leq \max\{\text{rtol } \rho_{\text{ref}}, \text{atol}\},$$

where ρ_{ref} is the corresponding reference residual norm used by PETSc. Since in the present runs $\text{atol} = 10^{-50}$, the absolute tolerance is negligible, and the stopping rule is effectively governed by the relative decrease prescribed by $\text{rtol} = 10^{-10}$. The PETSc diagnostic output shows that the BDDC branch `physical_` uses the preconditioned norm type for its convergence test, and the FETI-DP branch is solved through an internal KSP object which also uses a preconditioned norm type. Thus, the stopping criterion is applied to the PETSc KSP convergence norm, not necessarily to the true residual plotted later in the figures.

For the residual-history figures, the PETSc true-residual monitor is used for both branches,

`-physical_ksp_monitor_true_residual,` `-fluxes_ksp_monitor_true_residual.`

The plotted quantity is therefore the monitored true relative residual

$$\frac{\|b - Ax_k\|}{\|b\|}.$$

This quantity is useful for comparing the actual residual decrease in the original linear systems, but it is not necessarily the same norm as the one used internally by PETSc to decide convergence. For this reason, the residual figures are interpreted as convergence histories, while the actual stopping information is given by the PETSc convergence reason and the reported iteration count.

Each main benchmark case is executed three times. The iteration counts and spectral indicators are stable for a fixed case, while the wall-clock time may vary slightly because the computations are performed on a workstation. For this reason, the timing reported in the PETSc summary table

is the median over the three runs. The total time is taken from the maximum time reported by PETSc in the `Time (sec)` line of the performance summary. In addition to these final benchmark runs, an auxiliary representative BDDC profiling diagnostic is performed for the cases *C-M* and *C-L*. This diagnostic uses the PETSc profiling events `PCSetUp`, `KSPSolve`, and `PCApply` to illustrate the distribution of time between setup and iterative solution phases.

6.5.2 Numerical results

The PETSc benchmark results obtained with the configuration described in the previous subsection are summarized in Table 6.5. In all final benchmark runs, the stopping parameters were set explicitly and identically for the two PETSc solver branches,

$$\text{physical_ksp_rtol} = \text{fluxes_ksp_rtol} = 10^{-10},$$

with absolute tolerance 10^{-50} and maximum number of iterations equal to 300. Both solver branches converged with the PETSc convergence flag `CONVERGED_RTOL` in all reported cases. This means that the selected PETSc KSP convergence norm satisfied the prescribed relative decrease criterion. It does not mean that the plotted true relative residual was itself used as the stopping norm. In the table, the FETI-DP branch corresponds to the PETSc `fluxes_` solver, while the BDDC branch corresponds to the PETSc `physical_` solver.

Case	FETI-DP				BDDC				$\tilde{t}$ [s]
	n_λ	it	$\lambda_{\max}$	κ	n_{phys}	it	$\lambda_{\max}$	κ	
A-S	123	6	1.6	1.6	4160	6	1.5	1.5	2.162e-02
A-M	251	7	2.0	2.0	16512	6	1.8	1.8	8.718e-02
A-L	379	7	2.3	2.3	37056	7	2.0	2.0	4.400e-01
B-S	309	9	1.7	1.7	8320	8	1.7	1.7	3.435e-02
B-M	629	10	2.2	2.2	33024	9	2.0	2.0	1.509e-01
B-L	949	11	2.5	2.5	74112	10	2.2	2.2	5.216e-01
C-S	741	11	1.8	1.8	16512	10	1.7	1.7	4.171e-02
C-M	1509	12	2.2	2.2	65792	10	2.0	2.0	2.340e-01
C-L	2277	13	2.5	2.5	147840	11	2.2	2.2	7.747e-01

Table 6.5: Summary of PETSc benchmark results for FETI-DP and BDDC.

The table shows that both PETSc implementations remain in a favorable convergence regime over the whole benchmark set. The FETI-DP iteration counts range from 6 to 13, while the BDDC iteration counts range from 6 to 11. Thus, even when the active physical problem size grows from 4160 degrees of freedom in case *A-S* to 147840 degrees of freedom in case *C-L*, the number of Krylov iterations remains small. The BDDC branch requires slightly fewer iterations in most of the larger cases, but the difference remains modest and should not be interpreted as a general superiority statement. The PETSc benchmark uses a different discretization and a different implementation

framework than the sequential Octave experiments, and the two PETSc branches operate in different algebraic spaces.

The spectral indicators printed by the PETSc example show the same qualitative picture. In all cases, PETSc reports $\lambda_{\min} = 1.0$ for both methods. The largest reported eigenvalue remains between approximately 1.6 and 2.5 for FETI-DP, and between approximately 1.5 and 2.2 for BDDC. Since the minimum reported eigenvalue is equal to 1.0, these values also represent the corresponding condition-number indicators. The PETSc spectral estimates are therefore consistent with the main conclusion from the sequential part of the chapter: after preconditioning, both dual–primal methods lead to well-conditioned iteration operators.

The median total time reported in the last column of Table 6.5 is the total wall-clock time of the complete PETSc tutorial run, including both solver branches and all associated setup and solve work performed by the example. It is therefore not a method-separated timing measurement. The values are nevertheless useful as a basic reproducibility and benchmark-size indicator: they show that the chosen benchmark set remains feasible on the workstation used for the thesis, while still reaching substantially larger problem sizes than the sequential Octave spectral experiments. A more detailed interpretation of PETSc profiling events is given separately below.

For a more direct view of convergence, Figures 6.6 and 6.7 show the monitored true relative residual histories for the two largest 4×4 benchmark cases *C-M* and *C-L*. The plotted quantity is

$$\frac{\|b - Ax_k\|}{\|b\|},$$

as reported by the PETSc true-residual monitor. These curves are therefore useful for interpreting the actual decrease of the true residual, but they should not be confused with the PETSc KSP convergence norm used for the stopping decision.

In the *C-M* case, BDDC converges in 10 iterations and FETI-DP in 12 iterations. The final monitored true relative residuals are approximately $2.164 \cdot 10^{-11}$ for BDDC and $3.618 \cdot 10^{-12}$ for FETI-DP. Thus, the true residuals are reduced to a very small level for both methods. The fact that the plotted true residual may already be below 10^{-10} before the final PETSc stopping iteration is not a contradiction: PETSc stops according to the selected KSP convergence norm, whereas the figure displays the monitored true relative residual.

The same qualitative behavior is observed in the larger *C-L* case. Here, BDDC converges in 11 iterations and FETI-DP in 13 iterations. The final monitored true relative residuals are approximately $1.951 \cdot 10^{-11}$ for BDDC and $5.822 \cdot 10^{-13}$ for FETI-DP. The residual histories therefore confirm a strong decrease of the true residual for both methods. As in the *C-M* case, the stopping decision itself should be interpreted through the PETSc convergence flag `CONVERGED_RTOL`, which refers to the KSP convergence norm and not necessarily to the monitored true residual plotted in the figure.

Overall, the PETSc results support the same qualitative conclusion as the sequential Octave

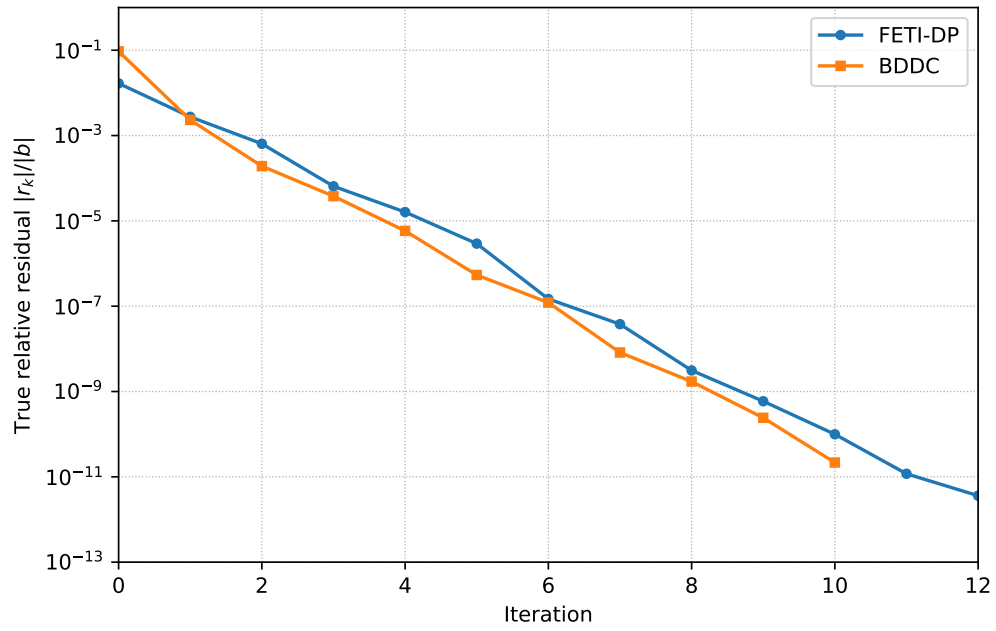


Figure 6.6: Normalized true residual histories for the representative PETSc case *C-M*.

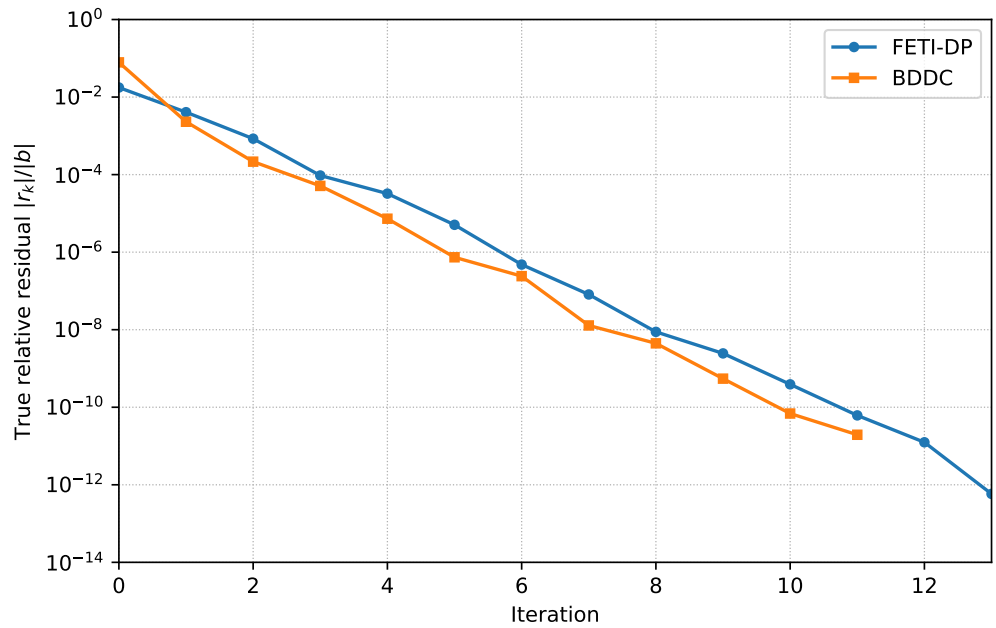


Figure 6.7: Normalized true residual histories for the representative PETSc case *C-L*.

experiments, but from a different computational viewpoint. The Octave part demonstrated spectral consistency and PCG convergence for the thesis implementation on small problems where full spectra could be computed. The PETSc part shows that the official library implementations also maintain small iteration counts, favorable reported spectral indicators, and rapidly decreasing true residual histories on a larger benchmark set. The iteration counts in Table 6.5 are determined by PETSc’s `CONVERGED_RTOL` stopping flag, while Figures 6.6 and 6.7 show the monitored true relative residuals for interpretation. These experiments should still be interpreted as a compact workstation benchmark, not as a full parallel scaling study. The detailed decomposition of PETSc timing data is discussed next.

In addition to the convergence and spectral results reported above, Table 6.6 gives an auxiliary representative timing-event decomposition for the BDDC branch. This profiling diagnostic is presented to illustrate the relative size of selected PETSc profiling events. The Table 6.6 shows how the measured BDDC time is distributed between setup and solve-related PETSc events in representative cases. The events `PCSetUp`, `KSPSolve`, and `PCApply` are PETSc profiling events associated here with the representative BDDC runs. The reason why the detailed timing decomposition was extracted only for representative BDDC run is that in the PETSc benchmark example used here the BDDC branch provides the most directly interpretable profiling events, whereas the FETI-DP branch is internally generated within the same dual-primal infrastructure through an internal `PCBDDC` object.

The event `PCSetUp` denotes the one-time preparation of the preconditioner, `PCApply` measures repeated applications of the preconditioner during the iterative process, and `KSPSolve` represents the overall Krylov solve phase. For the representative case *C-M*, the recorded total time was approximately $2.099 \cdot 10^{-1}$ s, with `PCSetUp` contributing approximately $1.175 \cdot 10^{-1}$ s. For the larger case *C-L*, the total time was approximately $5.841 \cdot 10^{-1}$ s, with `PCSetUp` contributing approximately $4.560 \cdot 10^{-1}$ s.

Case	total [s]	<code>PCSetUp</code> [s]	<code>KSPSolve</code> [s]	<code>PCApply</code> [s]
<i>C-M</i>	2.099e-01	1.175e-01	2.298e-02	2.118e-02
<i>C-L</i>	5.841e-01	4.560e-01	7.914e-02	7.554e-02

Table 6.6: Auxiliary PETSc timing-event decomposition for representative BDDC runs.

The auxiliary timing-event decomposition reported in Table 6.6 adds a limited practical implementation viewpoint that was not present in the purely sequential Octave study. In the two representative BDDC profiling runs shown there, the measured setup event `PCSetUp` is larger than the subsequent solve-related events `KSPSolve` and `PCApply`. This observation is useful for interpreting the cost structure of these particular one-shot PETSc runs, where the preconditioner setup is paid once and is not reused for several right-hand sides. It should not be interpreted as a systematic timing comparison between FETI-DP and BDDC, nor as a strong or weak parallel scaling result. The timing data therefore provide only implementation-level context, while the main convergence

and spectral conclusions of the PETSc part are based on Table 6.5 and Figures 6.6–6.7. Taken together, the PETSc benchmark comparison complements the previous sequential part of the thesis in the intended way: the PETSc timings are not directly comparable with the Octave timings because a different official benchmark problem and a different software environment are used, but the qualitative behavior of FETI-DP and BDDC remains consistent with the overall framework of the thesis.

6.5.3 Discussion of the parallel PETSc benchmark results

The PETSc benchmark results presented in Table 6.5 show a coherent numerical picture across the entire tested set. For both PETSc implementations, the reported spectral intervals of the preconditioned operators remain short, the corresponding condition-number indicators stay moderate, and the reduced problems are solved in a small number of Krylov iterations. In all reported cases PETSc gives $\lambda_{\min} = 1.0$, so the values of $\lambda_{\max}$ in Table 6.5 also represent the corresponding condition-number indicators. For FETI-DP, these values remain between approximately 1.6 and 2.5, while for BDDC they remain between approximately 1.5 and 2.2. The iteration counts are also small: FETI-DP requires between 6 and 13 iterations, while BDDC requires between 6 and 11 iterations. Thus, the PETSc benchmark confirms, using the official parallel PETSc implementations, the same qualitative numerical robustness that was already observed in the sequential Octave part of this chapter. This conclusion concerns the numerical behavior of the solvers; the present benchmark is not intended as a systematic parallel speedup or efficiency study.

The comparison between the PETSc FETI-DP and PETSc BDDC branches is also consistent across the tested benchmark set. Both methods display the same general convergence quality, and no result indicates a qualitative failure of either approach. At the same time, the BDDC branch requires slightly fewer iterations in most of the larger cases. This difference should not be overinterpreted as a general superiority statement. The two methods are implemented in PETSc through different algebraic formulations and act on different iteration spaces: the FETI-DP branch corresponds to the `fluxes_` solver and works in the multiplier setting, while the BDDC branch corresponds to the `physical_` solver and works on the physical degrees of freedom. As in the sequential spectral comparison, the Krylov convergence is influenced not only by the extremal eigenvalues, but also by the distribution of the residual with respect to the relevant eigenspaces and by implementation details of the concrete solver branch. The PETSc results therefore support the conclusion that the two dual-primal methods remain closely comparable on the numerical level, while still allowing moderate formulation-dependent differences in iteration counts and residual decay.

The representative true-residual histories in Figures 6.6 and 6.7 support the same interpretation. In the *C-M* case, BDDC converges in 10 iterations and FETI-DP in 12 iterations, with final monitored true relative residuals approximately $2.164 \cdot 10^{-11}$ and $3.618 \cdot 10^{-12}$, respectively. In the larger *C-L* case, BDDC converges in 11 iterations and FETI-DP in 13 iterations, with final monitored

true relative residuals approximately $1.951 \cdot 10^{-11}$ and $5.822 \cdot 10^{-13}$. These figures show that the true residual decreases rapidly for both methods in the representative larger cases. At the same time, the figures should be interpreted together with the stopping-criterion explanation from the previous subsection: PETSc reports convergence through the flag `CONVERGED_RTOL`, which refers to the selected KSP convergence norm, whereas the figures display the monitored true relative residual $\|b - Ax_k\|/\|b\|$. Therefore, the residual plots are used here as convergence-history diagnostics, while the stopping information itself is represented by the PETSc convergence reason and the iteration counts in Table 6.5.

A more systematic reading of Table 6.5 leads to the same three viewpoints that motivated the benchmark design. First, for a fixed subdomain decomposition, passing from the small to the medium and large cases leads only to a moderate increase of the reported spectral indicators and iteration counts. For example, within the 2×2 group, the FETI-DP condition-number indicator increases from about 1.6 to 2.3, while the BDDC indicator increases from about 1.5 to 2.0. Second, increasing the number of subdomains produces a more demanding interface problem, but the observed deterioration remains moderate. This is visible, for instance, when comparing the groups *A*, *B*, and *C*, where the iteration counts increase but remain small in all cases. Third, the benchmark includes weak-scaling-like comparisons in which the local subdomain resolution is preserved while the number of subdomains increases. These comparisons do not constitute a parallel weak-scaling study in the timing sense, but they are useful for observing numerical scalability signatures. The measured condition-number indicators and iteration counts grow only moderately, which is consistent with the classical expectation for dual-primal methods that, under suitable primal constraints, the preconditioned condition number is controlled mainly by the local ratio H/h and satisfies an estimate of the form

$$\kappa \leq C(1 + \log(H/h))^2, \quad (6.1)$$

where the constant C is independent of the number of subdomains. The present PETSc benchmark is too compact to verify this estimate asymptotically, but it displays the expected qualitative behavior: increasing the global problem size and the number of subdomains does not lead to a qualitative loss of robustness.

The timing information should be interpreted more cautiously than the spectral and iteration-count data. The total time $\tilde{t}$ in Table 6.5 is the median total wall-clock time of the complete PETSc run (of both methods) over three repetitions. It is useful as a reproducibility and problem-size indicator, but it is not a method-separated timing measurement. The auxiliary timing-event decomposition in Table 6.6 is retained only for representative BDDC runs and serves as an implementation-level diagnostic. In those two one-shot profiling runs, the measured setup event `PCSetUp` is larger than the subsequent solve-related events `KSPSolve` and `PCApply`. This indicates that, for the representative BDDC runs *C-M* and *C-L*, a significant part of the measured time is spent in preparing the preconditioner. However, this observation should not be interpreted as a universal asymptotic statement.

For larger problems, different processor counts, or repeated solves with the same preconditioner setup, the balance between setup and solve-related costs may change.

Taken together, the PETSc benchmark comparison complements the sequential Octave experiments in the intended way. The Octave part provides a controlled implementation in which full spectra of both non-preconditioned and preconditioned operators can be computed for relatively small problems. The PETSc part, on the other hand, uses official parallel implementations on a larger benchmark set and confirms that both dual-primal methods retain small iteration counts, moderate reported spectral indicators, and rapidly decreasing true residual histories. The PETSc timings are not directly comparable with the Octave timings because a different official benchmark problem, a different discretization, and a different software environment are used. Nevertheless, the qualitative conclusion remains consistent across both parts of the thesis: FETI-DP and BDDC behave as closely related and numerically robust dual-primal domain decomposition methods, with no qualitative breakdown observed in the tested cases.

The official parallel PETSc implementations thus confirm, on a controlled benchmark set, the same main conclusion already suggested by the sequential experiments: both dual-primal solvers behave in a numerically robust and closely comparable manner, while exhibiting the expected numerical scalability signatures of dual-primal DDM.

Chapter 7

Conclusion

This bachelor thesis was devoted to the comparison of the dual–primal domain decomposition methods FETI-DP and BDDC for the solution of linear systems arising from finite element discretizations of elliptic boundary value problems. The objectives of the work were to understand the mathematical foundations of the two methods, to formulate them within a common framework, to implement their sequential versions in Octave, to compare the spectra of the corresponding operators, and to complement this study by a benchmark comparison of the official parallel PETSc implementations.

The first part of the thesis established the analytical and discretization background required for the later domain decomposition methods. A model Poisson problem with homogeneous mixed Dirichlet–Neumann boundary conditions on the unit square was introduced, motivated by the deflection of a tensioned membrane. Its weak formulation was derived in Sobolev spaces, the role of trace operators and boundary conditions was explained, and the existence and uniqueness of the weak solution were discussed through the Lax–Milgram framework. The variational interpretation of the homogeneous model problem then led naturally to the Ritz method. On this basis, the finite element discretization was constructed using piecewise linear basis functions on a structured triangular mesh. The resulting stiffness matrix was shown to be sparse, symmetric, and positive definite after the homogeneous Dirichlet degrees of freedom were eliminated.

The second part of the thesis reformulated the finite element system in a non-overlapping substructuring setting. The domain was partitioned into structured rectangular subdomains, and the local finite element spaces were described by separating interior and interface degrees of freedom. By eliminating the interior unknowns exactly, the original discrete problem was reduced to an equivalent interface problem involving local Schur complement operators. This reduction was then expressed in a product interface framework, where each subdomain carries its own copy of interface unknowns and global continuity is represented explicitly by a jump operator. The introduction of primal degrees of freedom Γ_c and the splitting

$$W = W_c \oplus W_\Delta$$

provided the common dual–primal structure used by both methods.

Within this common framework, the thesis derived and implemented the two solvers. FETI-DP was formulated as a multiplier method in which the reduced continuity constraints on W_Δ are enforced by Lagrange multipliers. This led to a reduced SPD multiplier-space system suitable for PCG, together with a multiplicity-scaled preconditioner and a coarse correction associated with the primal unknowns. BDDC was formulated in the assembled interface space, using scaled distribution and averaging operators, local Δ -block corrections, and a coarse correction generated by primal basis functions. Both methods were implemented in Octave in a modular matrix-free way, reusing the same finite element assembly, subdomain decomposition, interface bookkeeping, primal maps, PCG wrapper, and reconstruction routines.

The sequential Octave experiments showed that the distinction between non-preconditioned and preconditioned operators is essential. The spectra of the raw reduced FETI-DP and BDDC operators were useful diagnostic information, but their conditioning was substantially less favorable and the two raw operators acted in different spaces. The PCG-relevant comparison was therefore based on the preconditioned operators. In all tested Octave cases, the preconditioned spectra were compact, the smallest eigenvalue was equal to 1, and the largest eigenvalue remained small, between approximately 1.376 and 2.683. Consequently, the corresponding condition-number indicators remained small as well. The PCG iteration counts stayed low: FETI-DP required between 5 and 14 iterations, while BDDC required between 1 and 16 iterations in the tested cases, with final relative residuals below the prescribed tolerance. The sequential experiments therefore support the conclusion that both implemented solvers are numerically robust on the considered model problems.

The Octave experiments also displayed the expected numerical scalability signature of dual–primal methods. In the weak-scaling-like comparison where the local subdomain resolution was preserved and the global problem size was increased by using more subdomains, the deterioration of the condition-number indicators and iteration counts remained moderate. This observation concerns numerical scalability of the preconditioned operators, not parallel scalability of the Octave implementation. The Octave code was sequential and was used primarily to expose the algebraic structure of the methods, compute full diagnostic spectra, and verify the solver behavior in a controlled setting.

The PETSc benchmark comparison provided a complementary viewpoint based on official parallel implementations. The BDDC branch was represented by the PETSc `physical_` solver using PCBDDC, whereas the FETI-DP branch was represented by the `fluxes_` solver created by KSPFETIDP. The PETSc experiments used the official tutorial example `ex59.c`, which is not identical to the previous Octave implementation, but it belongs to the same class of structured elliptic problems and uses the same dual–primal domain decomposition principles. The tested benchmark set ranged from 4160 to 147840 active physical degrees of freedom.

The PETSc results confirmed the qualitative conclusions obtained from the sequential Octave experiments. For both methods, the reported PETSc spectral intervals of the preconditioned oper-

ators remained short, with $\lambda_{\min} = 1.0$ in all cases. The largest reported eigenvalue stayed between approximately 1.6 and 2.5 for FETI-DP and between approximately 1.5 and 2.2 for BDDC. The Krylov iteration counts also remained small: FETI-DP required between 6 and 13 iterations, while BDDC required between 6 and 11 iterations. BDDC often required slightly fewer iterations in the larger PETSc cases, but this difference was moderate and should not be interpreted as a general proof of superiority. The two PETSc branches operate in different algebraic spaces and are implemented through different formulations within the same dual–primal infrastructure.

The representative PETSc residual histories for the larger 4×4 cases further confirmed rapid convergence for both methods. In case designated as *C-M*, BDDC converged in 10 iterations and FETI-DP in 12 iterations, while in case *C-L*, BDDC converged in 11 iterations and FETI-DP in 13 iterations. The monitored true relative residuals were reduced to very small values in both cases. The additional PETSc timing information showed that, in the representative BDDC profiling runs, the setup event `PCSetUp` was larger than the solve-related events `KSPSolve` and `PCApply`. This provides useful implementation-level context, but it should not be interpreted as a systematic method-separated timing comparison or as a complete strong- or weak-scaling study.

Overall, the thesis shows that FETI-DP and BDDC can be understood as two closely related realizations of the same dual–primal substructuring idea. They share the same fundamental components: local subdomain solves, Schur complement reduction to the interface, primal coarse degrees of freedom, and a preconditioned Krylov iteration on the reduced interface problem. Their difference lies mainly in the enforcement mechanism for the remaining non-primal continuity: FETI-DP uses dual Lagrange multipliers, whereas BDDC uses a primal assembled formulation with averaging and coarse correction. The numerical results obtained in both the Octave and PETSc parts of the thesis support the same main conclusion: on the tested model and benchmark problems, both methods are numerically robust and closely comparable, with no qualitative breakdown observed for either method.

Several possible extensions remain beyond the scope of this bachelor thesis. A more extensive parallel study could investigate strong and weak scaling on a larger computing cluster, including method-separated setup and solve timings. Further experiments could also consider more general geometries, unstructured decompositions, heterogeneous coefficients, nonuniform meshes, multiple right-hand sides, and richer primal constraints such as edge or face averages. Nevertheless, within the scope of the present work, the thesis successfully constructs a unified theoretical and implementation framework for FETI-DP and BDDC and demonstrates their close numerical relationship on representative elliptic problems.

The implementation developed as part of this thesis is submitted together with the thesis as an electronic appendix in IS EDISON and archived in a fixed public repository release. The exact submitted version corresponds to the repository https://github.com/Musiljuraj/bc_ddm, release tag `Thesis-final`, and commit `52830ed61a7a75a2b5858b341bdb6332a7ba3fda`. This makes it possible to inspect the implementation corresponding to the presented results and, within the

limitations described in the numerical chapters, to reproduce the Octave experiments and PETSc benchmark workflow.

Literatura

1. LI, Jing; WIDLUND, Olof B. FETI-DP, BDDC, and Block Cholesky Methods. *International Journal for Numerical Methods in Engineering*. 2006, vol. 66, no. 2, pp. 250–271. Published version of the preprint dated December 11, 2004.
2. MANDEL, Jan; DOHRMANN, Clark R.; TEZAUR, Radek. An Algebraic Theory for Primal and Dual Substructuring Methods by Constraints. *Applied Numerical Mathematics*. 2005, vol. 54, no. 2, pp. 167–193. Available from DOI: 10.1016/j.apnum.2004.09.022.
3. PETSC DEVELOPERS. *PETSc tutorial example ex59.c: PCBDDC/FETI-DP* [<https://petsc.org/main/src/ksp/ksp/tutorials/ex59.c.html>]. 2026. Official PETSc source documentation.
4. PETSC DEVELOPERS. *PCBDDC: Balancing Domain Decomposition by Constraints* [<https://petsc.org/release/manualpages/PC/PCBDDC/>]. 2026. Official PETSc manual page.
5. PETSC DEVELOPERS. *KSPFETIDP: FETI-DP Krylov solver* [<https://petsc.org/main/manualpages/KSP/KSPFETIDP/>]. 2026. Official PETSc manual page.
6. REKTORYS, Karel. *Variační metody : v inženýrských problémech a v problémech matematické fyziky*. 2nd revised edition in Czech. Praha: Academia, 1999. ISBN 80-200-0714-8.
7. LE TALLEC, P.; DE ROECK, Y. H.; VIDRASCU, M. Domain decomposition methods for large linearly elliptic three-dimensional problems. *Journal of Computational and Applied Mathematics*. 1991, vol. 34, pp. 93–117. ISSN 0377-0427. North-Holland imprint.
8. DOSTÁL, Zdeněk. Rozděl a slep aneb jak řešit soustavu s bilionem lineárních rovnic. *Pokroky Matematiky, Fyziky a Astronomie*. 2018, vol. 63, no. 1, pp. 28–40.
9. DOLEAN, Victorita; JOLIVET, Pierre; NATAF, Frédéric. *An Introduction to Domain Decomposition Methods: Algorithms, Theory, and Parallel Implementation*. Vol. 144. Philadelphia: Society for Industrial and Applied Mathematics, 2015. Other Titles in Applied Mathematics. ISBN 978-1-61197-405-8.

10. FARHAT, Charbel; LESOINNE, Michel; LE TALLEC, Patrick; PIERSON, Kendall; RIXEN, Daniel. FETI-DP: A dual-primal unified FETI method — Part I: A faster alternative to the two-level FETI method. *International Journal for Numerical Methods in Engineering*. 2001, vol. 50, no. 7, pp. 1523–1544.
11. MANDEL, Jan; SOUSEDÍK, Bedřich. BDDC and FETI-DP under Minimalist Assumptions. *Computing*. 2007, vol. 81, no. 4, pp. 269–280. Available from DOI: 10.1007/s00607-007-0254-y.
12. MANDEL, Jan. Balancing Domain Decomposition. *Communications in Numerical Methods in Engineering*. 1993, vol. 9, pp. 233–241. Available from DOI: 10.1002/cnm.1640090307.
13. DOSTÁL, Zdeněk; HORÁK, David; KUČERA, Radek. Total FETI – An easier implementable variant of the FETI method for numerical solution of elliptic PDE. *Preprint*. 2005, pp. 1–8. Supported by grants AVOZ2076919 and GACR 101/04/1145.
14. FARHAT, Charbel; ROUX, François-Xavier. A Method of Finite Element Tearing and Interconnecting and Its Parallel Solution Algorithm. *International Journal for Numerical Methods in Engineering*. 1991, vol. 32, no. 6, pp. 1205–1227. Available from DOI: 10.1002/nme.1620320604.
15. DOHRMANN, Clark R. *A Study of Two Domain Decomposition Preconditioners*. 2003-11. Tech. rep., SAND2003-4391. Sandia National Laboratories. Available from DOI: 10.2172/918238.
16. DOHRMANN, Clark R. Preconditioning of Saddle Point Systems by Substructuring and a Penalty Approach. In: WIDLUND, Olof B.; KEYES, David E. (eds.). *Domain Decomposition Methods in Science and Engineering XVI*. Berlin, Heidelberg: Springer, 2007, vol. 55, pp. 53–64. Lecture Notes in Computational Science and Engineering. Available from DOI: 10.1007/978-3-540-34469-8_5.
17. COWSAR, Lawrence C.; MANDEL, Jan; WHEELER, Mary F. Balancing Domain Decomposition for Mixed Finite Elements. *Mathematics of Computation*. 1995, vol. 64, no. 211, pp. 989–1015. Available from DOI: 10.1090/S0025-5718-1995-1297465-9.
18. BRENNER, Susanne C. The condition number of the Schur complement in domain decomposition. *Numerische Mathematik*. 1999, vol. 83, no. 2, pp. 187–203. Available from DOI: 10.1007/s002110050446.

Appendix A

Source Code and Reproducibility Archive

The source code developed as part of this thesis is submitted as an electronic appendix in IS EDISON. The complete source code is not printed in the thesis text, because the implementation is extensive and is more appropriately submitted in electronic form.

The electronic appendix contains the final source-code archive corresponding to this thesis. In particular, it contains the Octave implementation of the finite element assembly, the domain decomposition infrastructure, the sequential FETI-DP and BDDC solvers, experiment scripts, test runners, selected input/output data, and files related to the PETSc benchmark workflow.

The submitted archive corresponds to the following fixed public repository release:

Repository: https://github.com/Musiljuraj/bc_ddm

Release tag: Thesis-final

Commit hash: 52830ed61a7a75a2b5858b341bdb6332a7ba3fda

Short commit hash: 52830ed

The electronic appendix is intended as a reproducibility supplement to the thesis. It identifies the exact implementation version corresponding to the numerical experiments and makes it possible to inspect and, subject to the software requirements described in the repository, reproduce the Octave experiments and the PETSc benchmark workflow.